

JAVA 17 OCP · YDS STUDY SERIES

Unit 14 · I/O · Bilingual Notes

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu ana kaynak, [OCP_Java_SE17_Chapter1den_Itibaren.pdf](#) içindeki ilgili chapter gövdesini ve Appendix resmî cevaplarını kaynak sırasını koruyan English → Türkçe paragraf çiftleriyle bir araya getirir. Kod ve terminal çıktıları çevrilmeden, bir kez ve kaynak konumunda gösterilir.

[Vocabulary](#) · [Grammar notes](#) · [Teknik hafıza notu](#)

Kaynak kapsam manifesti

- Kaynak: [exam_lecture/OCP_Java_SE17_Chapter1den_Itibaren.pdf](#)
- Chapter: 14 · I/O
- Chapter PDF sayfaları: 785–862
- Appendix cevap sayfaları: 955–959
- Beklenen sayfa marker'ı: 78
- Beklenen resmî cevap: 25
- Eşleme biçimi: English paragraf → Türkçe çeviri → varsa kod

İçindekiler

- 1 [Conceptualizing the File System](#)
- 2 [Creating a File or Path](#)
- 3 [Operating on File and Path](#)
- 4 [Introducing I/O Streams](#)
- 5 [Reading and Writing Files](#)
- 6 [Serializing Data](#)
- 7 [Interacting with Users](#)
- 8 [Working with Advanced APIs](#)
- 9 [Summary](#)
- 10 [Exam Essentials](#)
- 11 [Review Questions](#)
- 12 [Official Review Question Answers / Resmî Cevaplar](#)

Chapter 14 · I/O · Eksiksiz çift dilli kaynak

Chapter 14 · I/O

English: OCP exam objectives covered in this chapter: Using the Java I/O API.

Türkçe: Bu bölümde ele alınan OCP sınav hedefi: Java I/O API'sini kullanmak.

English: Read and write console and file data using I/O streams.

Türkçe: I/O stream'leri kullanarak console ve file verilerini okuyun ve yazın.

English: Serialize and deserialize Java objects.

Türkçe: Java object'lerini serialize ve deserialize edin.

English: Create, traverse, read, and write Path objects and their properties using the java.nio.file API.

Türkçe: `java.nio.file` API'sini kullanarak `Path` object'lerini ve property'lerini oluşturun, dolaşın, okuyun ve yazın.

English: What can Java applications do outside the scope of managing objects and attributes in memory? How can they save data so that information is not lost every time the program is terminated? They use files, of course! You can design code that writes the current state of an application to a file every time the application is closed and then reloads the data when the application is executed the next time. In this manner, information is preserved between program executions.

Türkçe: Java uygulamaları bellekteki nesneleri ve nitelikleri yönetme kapsamı dışında neler yapabilir? Program her sonlandırıldığında bilgilerin kaybolmaması için verileri nasıl kaydedebilirler? Tabii ki dosyaları kullanıyorlar! Uygulama her kapatıldığında bir uygulamanın mevcut durumunu bir dosyaya yazan ve daha sonra uygulama bir dahaki sefere çalıştırıldığında verileri yeniden yükleyen kodlar tasarlayabilirsiniz. Bu şekilde, bilgi program yürütmeleri arasında korunur.

English: This chapter focuses on using I/O (input/output) and NIO.2 (non-blocking I/O) APIs to interact with files and I/O streams. The preferred approach for working with files and directories with newer software applications is to use NIO.2 rather than I/O where possible. However, you'll see that the two relate, and both are in wide use.

Türkçe: Bu bölüm, dosyalar ve I/O stream'leriyle çalışmak için I/O (input/output) ve NIO.2 (New I/O) API'lerinin kullanımına odaklanır. Yeni uygulamalarda dosya ve izin işlemleri için mümkün olduğunda I/O yerine NIO.2 tercih edilir. Bununla birlikte, bu iki API birbirine ilişkilidir ve ikisi de yaygın olarak kullanılmaktadır.

English: We start by describing how files and directories are organized within a file system and show how to access them with the File class and Path interface. Then we show how to work with files and directories. We conclude this chapter with advanced topics like serializing data, discussing ways of reading user input at runtime using the Console class, and interacting with file attributes.

Türkçe: Önce dosyaların ve dizinlerin bir file system (dosya sistemi) içinde nasıl düzenlendiğini açıklıyor; bunlara `File` class'ı ve `Path` interface'i ile nasıl erişileceğini gösteriyoruz. Ardından dosya ve izin işlemlerini ele alıyoruz. Bölümün sonunda ise verilerin serialization'ı, `Console` class'ı ile çalışma zamanında kullanıcı girdisi okuma ve file attribute'larla çalışma gibi ileri konuları inceliyoruz.

English: NIO stands for non-blocking input/output API and is sometimes referred to as new I/O. The exam covers NIO version 2. There was a version 1 that covered channels, but it is not on the exam.

Türkçe: NIO, engelleyici olmayan giriş/çıkış API'si anlamına gelir ve bazen yeni I/O olarak adlandırılır. Sınav, NIO sürüm 2'yi kapsar. Kanalları kapsayan bir sürüm 1 vardı, ancak sınavda değil.

Java 17 editör notu: NIO ailesi **New I/O** adıyla da bilinir. NIO.2 `Path/Files` operation'larının genel olarak non-blocking olduğu sonucu bu açılmadan çıkarılamaz; ordinary file-system operation'ları blocking olabilir.

Referencing Files and Directories

English: We begin this chapter by reviewing what files and directories are within a file system. We also present the `File` class and `Path` interface along with how to create them.

Türkçe: Bölüme, file system içindeki dosya ve dizin kavramlarını gözden geçirerek başlıyoruz. Ayrıca `File` class'ını ve `Path` interface'ini tanıtıyor, bunların nasıl oluşturulacağını gösteriyoruz.

Conceptualizing the File System

English: We start with the basics. Data is stored on persistent storage devices, such as hard disk drives and memory cards. A file within the storage device holds data. Files are organized into hierarchies using directories. A directory is a location that can contain files as well as other directories. When working with directories in Java, we often treat them like files. In fact, we use many of the same classes and interfaces to operate on files and directories. For example, a file and directory both can be renamed with the same Java method. Note that we often say file to mean file or directory in this chapter.

Türkçe: Temel kavramlarla başlayalım. Veriler, sabit disk ve hafıza kartı gibi kalıcı depolama aygıtlarında tutulur; aygıt içindeki bir file veri içerir. File'lar directory kullanılarak hiyerarşik biçimde düzenlenir. Bir directory, hem file'ları hem de başka directory'leri barındırabilen bir konumdur. Java'da directory'ler üzerinde çalışırken çoğu zaman onlara file gibi davranırız; aynı class ve interface'lerin birçoğu her iki tür üzerinde de kullanılır. Örneğin bir file ile bir directory aynı Java method'uyla yeniden adlandırılabilir. Bu bölümde “file” sözcüğünün kimi zaman hem file hem de directory anlamında kullanıldığını unutmayın.

English: To interact with files, we need to connect to the file system. The file system is in charge of reading and writing data within a computer. Different operating systems use different file systems to manage their data. For example, Windows-based systems use a different file system than Unix-based ones. For the exam, you just need to know how to issue commands using the Java APIs. The JVM will automatically connect to the local file system, allowing you to perform the same operations across multiple platforms.

Türkçe: File'larla etkileşim kurmak için file system'e bağlanmamız gerekir. Bilgisayardaki verilerin okunup yazılmasından file system sorumludur. Farklı operating system'ler, verilerini yönetmek için farklı file system'ler kullanır; örneğin Windows ve Unix tabanlı sistemlerin file system'leri farklıdır. Sınav için Java API'leriyle gerekli operation'ları nasıl çağıracağınızı bilmeniz yeterlidir. JVM yerel file system'e otomatik olarak bağlanır; böylece aynı operation'ları farklı platformlarda gerçekleştirebilirsiniz.

English: Next, the root directory is the topmost directory in the file system, from which all files and directories inherit. In Windows, it is denoted with a drive letter such as `C:\`, while on Linux, it is denoted with a single forward slash, `/`.

Türkçe: Root directory, file system içindeki en üst directory'dir ve bütün file ile directory'ler buradan türetilir. Windows'ta `C:\` gibi drive letter ile, Linux'ta ise tek forward slash (`/`) ile gösterilir.

English: A path is a representation of a file or directory within a file system. Each file system defines its own path separator character that is used between directory entries. The value to the left of a separator is the parent of the value to the right of the separator. For example, the path value `/user/home/zoo.txt` means that the file `zoo.txt` is inside the home directory, with the home directory inside the user directory.

Türkçe: Path, file system içindeki file veya directory'nin temsilidir. Her file system, directory entry'leri arasında kullanılacak path separator'ı tanımlar. Separator'ın solundaki value sağındakinin parent'ıdır.

Örneğin `/user/home/zoo.txt`, `zoo.txt` file'ının `home` directory'sinde, `home`un da `user` directory'sinde olduğunu gösterir.

English: Operating System File Separators Different operating systems vary in their format of pathnames. For example, Unix-based systems use the forward slash, `/`, for paths, whereas Windows-based systems use the backslash, `\`, character. That said, many programming languages and file systems support both types of slashes when writing path statements. Java offers a system property to retrieve the local separator character for the current environment:

Türkçe: Operating System File Separator'ları — Farklı operating system'ler pathname formatında farklılaşır. Unix tabanlı sistemler forward slash (`/`), Windows tabanlı sistemler backslash (`\`) kullanır. Bununla birlikte birçok programming language ve file system path statement'larında iki slash türünü de destekler. Java current environment'ın local separator character'ını almak için system property sunar:

```
System.out.print(System.getProperty("file.separator"));
```

English: We show how a directory and file system is organized in a hierarchical manner in FIGURE 14.1.

Türkçe: FIGURE 14.1, directory ve file'ların bir file system içinde nasıl hiyerarşik biçimde düzenlendiğini gösterir.

English: FIGURE 14.1 — Directory and file hierarchy

```
c:\
├─ app\
│   ├── animals\
│   │   ├── Bear.java
│   │   └── Bear.class
│   ├── employees\
│   └── java.exe
├─ zoo\
└─ info.txt
```

Türkçe: **ŞEKİL 14.1 — Dizin ve dosya hiyerarşisi.** Root `c:\`; `app` ve `zoo` directory'leriyle `info.txt` file'ını içerir. `app` altında `animals`, `employees` ve `java.exe`; `animals` altında ise `Bear.java` ile `Bear.class` bulunur.

English: This diagram shows the root directory, `c:`, as containing two directories, `app` and `zoo`, along with the file `info.txt`. Within the `app` directory, there are two more folders, `animals` and `employees`, along with the file `java.exe`. Finally, the `animals` directory contains two files, `Bear.java` and `Bear.class`.

Türkçe: Diyagramda root directory `c:`, `app` ve `zoo` adlı iki directory ile `info.txt` file'ını içerir. `app` directory'sinin içinde `animals` ve `employees` adlı iki directory ile `java.exe` file'ı bulunur. Son olarak `animals` directory'si `Bear.java` ve `Bear.class` adlı iki file içerir.

English: We use both absolute and relative paths to the file or directory within the file system. The absolute path of a file or directory is the full path from the root directory to the file or directory, including all subdirectories that contain the file or directory. Alternatively, the relative path of a file or directory is the path from the current working directory to the file or directory. For example, the following is an absolute path to the `Bear.java` file:

Türkçe: File system içindeki bir file veya directory'ye absolute path (mutlak yol) ya da relative path (görelî yol) ile erişebiliriz. Absolute path, root directory'den hedefe kadar hedefi barındıran bütün alt directory'leri içeren tam yoldur. Relative path ise current working directory'den hedefe giden yoldur. Örneğin `Bear.java` için aşağıdaki değer bir absolute path'tir:

```
C:\app\animals\Bear.java
```

English: The following is a relative path to the same file, assuming the user's current directory is set to `C:\app`:

Türkçe: Kullanıcının current directory'sinin `C:\app` olduğunu varsayarsak aynı file'a giden relative path şöyledir:

```
animals\Bear.java
```

English: Determining whether a path is relative or absolute is file-system dependent. To match the exam, we adopt the following conventions:

Türkçe: Bir path'in relative mı yoksa absolute mu sayılacağı file system'e bağlıdır. Sınav kapsamında aşağıdaki convention'ları (kuralları) kabul ediyoruz:

English: If a path starts with a forward slash (/), it is absolute, with / as the root directory, such as `/bird/parrot.png`.

Türkçe: Path forward slash (/) ile başlıyorsa absolute'dur; root / olur. Örnek: `/bird/parrot.png`.

English: If a path starts with a drive letter (c:), it is absolute, with the drive letter as the root directory, such as `C:/bird/info`.

Türkçe: Path `c:` gibi drive letter ile başlıyorsa absolute'dur ve drive letter root sayılır. Örnek: `C:/bird/info`.

English: Otherwise, it is a relative path, such as `bird/parrot.png`.

Türkçe: Diğer durumlarda `bird/parrot.png` gibi bir relative path'tir.

English: Absolute and relative paths can contain path symbols. A path symbol is one of a reserved series of characters with special meaning in some file systems. For the exam, there are two path symbols you need to know, as listed in TABLE 14.1.

Türkçe: Absolute ve relative path'ler path symbol'ları içerebilir. Path symbol, bazı file system'lerde özel anlamı olan ayrılmış karakter dizilerinden biridir. Sınav için TABLE 14.1'deki iki path symbol'ı bilmeniz gerekir.

English: TABLE 14.1 — File-system symbols

Symbol	Description
.	A reference to the current directory
..	A reference to the parent of the current directory

Türkçe: TABLO 14.1 — File-system sembolleri

Sembol	Açıklama
.	Current directory'ye referans
..	Current directory'nin parent'ına referans

English: Looking at FIGURE 14.2, suppose the current directory is `/fish/shark/hammerhead`. In this case, `../swim.txt` is a valid relative path equivalent to `/fish/shark/swim.txt`. Likewise, `./play.png` refers to `play.png` in the current directory. These symbols can also be combined for greater effect. For example, `../../clownfish` is a relative path equivalent to `/fish/clownfish` within the file system.

Türkçe: FIGURE 14.2'de current directory'nin `/fish/shark/hammerhead` olduğunu varsayalım. `../swim.txt`, `/fish/shark/swim.txt` ile eşdeğer geçerli relative path'tir. Benzer şekilde `./play.png`, current directory'deki `play.png` file'ını gösterir. Symbol'lar birlikte de kullanılabilir: `../../clownfish`, file system içinde `/fish/clownfish` ile eşdeğerdir.

English: Sometimes you'll see path symbols that are redundant or unnecessary. For example, the absolute path `/fish/clownfish/../shark../swim.txt` can be simplified to `/fish/shark/swim.txt`. We see how to handle these redundancies later in the chapter when we cover `normalize()`.

Türkçe: Bazen redundant veya gereksiz path symbol'ları görülür. Örneğin absolute `/fish/clownfish/../shark../swim.txt` path'i `/fish/shark/swim.txt` olarak sadeleştirilebilir. Bu redundancy'lerin `normalize()` ile nasıl giderildiği ileride ele alınır.

English: FIGURE 14.2 — Relative paths using path symbols

```

fish\
├─ shark\
│   ├── swim.txt
│   └─ hammerhead\
│       └─ play.png
└─ clownfish\

```

`../..`
`../swim.txt`
`.` (current directory)
`./play.png`
`../../clownfish`

Türkçe: ŞEKİL 14.2 — Path sembolleri kullanan relative path'ler. Başlangıç noktası `hammerhead` directory'sidir. `.` burayı, `..` parent `shark` directory'sini, `../..` ise `fish` directory'sini gösterir.

English: A symbolic link is a special file within a file system that serves as a reference or pointer to another file or directory. Suppose we have a symbolic link from `/zoo/user/favorite` to `/fish/shark`. The `shark` folder and its elements can be accessed directly or via the symbolic link. For example, the following paths reference the same file:

Türkçe: Symbolic link, file system içinde başka bir file veya directory'ye reference görevi gören özel file'dir. `/zoo/user/favorite` path'inin `/fish/shark`a symbolic link olduğunu varsayalım. `shark` directory'sine ve entry'lerine doğrudan ya da link üzerinden erişilebilir. Örneğin aşağıdaki path'ler aynı file'i gösterir:

```
/fish/shark/swim.txt
/zoo/user/favorite/swim.txt
```

English: In general, symbolic links are transparent to the user, as the operating system takes care of resolving the reference to the actual file. While the I/O APIs do not support symbolic links, NIO.2 includes full support for creating, detecting, and navigating symbolic links within the file system.

Türkçe: Genel olarak, symbolic links kullanıcıya şeffaftır, çünkü işletim sistemi gerçek dosyaya yapılan referansı çözmeye özen gösterir. I/O API'leri symbolic links'yi desteklemezken, NIO.2 file system içinde symbolic links oluşturmak, tespit etmek ve gezinmek için tam destek içerir.

Creating a File or Path

English: In order to do anything useful, you first need an object that represents the path to a particular file or directory on the file system. Using legacy I/O, this is the `java.io.File` class, whereas with NIO.2, it is the `java.nio.file.Path` interface. The `File` class and `Path` interface cannot read or write data within a file, although they are passed as a reference to other classes, as you see in this chapter.

Türkçe: Yararlı operation'lar için önce file system'deki belirli file veya directory'nin path'ini temsil eden object gerekir. Legacy I/O'da bu `java.io.File` class'ı, NIO.2'de `java.nio.file.Path` interface'idir. `File` ve `Path`, başka class'lara reference olarak geçirilebilse de file content'ini okuyup yazamaz.

English: Remember, a `File` or `Path` can represent a file or a directory.

Türkçe: Bir `File` veya `Path`in file ya da directory gösterebileceğini unutmayın.

Creating a File

English: The `File` class is created by calling its constructor. This code shows three different constructors:

Türkçe: `File` class'ı constructor çağırısıyla oluşturulur. Bu kod üç farklı constructor gösterir:

```
File zooFile1 = new File("/home/tiger/data/stripes.txt");
File zooFile2 = new File("/home/tiger", "data/stripes.txt");
```

```
File parent = new File("/home/tiger");
File zooFile3 = new File(parent, "data/stripes.txt");
System.out.println(zooFile1.exists());
```

English: All three create a File object that points to the same location on disk. If we passed null as the parent to the final constructor, it would be ignored, and the method would behave the same way as the single String constructor. For fun, we also show how to tell if the file exists on the file system.

Türkçe: Üçü de disk'teki aynı location'ı gösteren File object'i oluşturur. Son constructor'a parent olarak null verilirse value göz ardı edilir ve tek String alan constructor gibi davranır. Ayrıca file'in file system'de varlığı da kontrol edilir.

Creating a Path

English: Since Path is an interface, we can't create an instance directly. After all, interfaces don't have constructors! Java provides a number of classes and methods that you can use to obtain Path objects.

Türkçe: Path bir interface olduğundan doğrudan instance oluşturulamaz; interface'lerin constructor'ı yoktur. Java, Path object'i elde etmek için çeşitli class ve method'lar sağlar.

English: The simplest and most straightforward way to obtain a Path object is to use a static factory method defined on Path or Paths. All four of these examples point to the same reference on disk:

Türkçe: Path object'i elde etmenin en basit yolu Path veya Paths üzerinde tanımlı static factory method kullanmaktır. Dört örnek de disk'teki aynı reference'ı gösterir:

```
Path zooPath1 = Path.of("/home/tiger/data/stripes.txt");
Path zooPath2 = Path.of("/home", "tiger", "data", "stripes.txt");
Path zooPath3 = Paths.get("/home/tiger/data/stripes.txt");
Path zooPath4 = Paths.get("/home", "tiger", "data", "stripes.txt");
System.out.println(Files.exists(zooPath1));
```

English: Both methods allow passing a varargs parameter to pass additional path elements. The values are combined and automatically separated by the operating system–dependent file separator. We also show the Files helper class, which can check if the file exists on the file system.

Türkçe: İki method da ek path element'leri için varargs parameter kabul eder. Value'lar birleştirilir ve operating-system-dependent file separator ile otomatik ayrılır. Files helper class'ı file'in file system'de olup olmadığını denetleyebilir.

English: As you can see, there are two ways of doing the same thing here. The Path.of() method was introduced in Java 11 as a static method on the interface. The Paths factory class also provides a get() method to do the same thing. Note the s at the end of the Paths class to distinguish it from the Path interface. We use Path.of() and Paths.get() interchangeably in this chapter.

Türkçe: Aynı operation'ın iki yolu vardır. Path.of() Java 11'de interface'e eklenen static method'dur; Paths factory class'ı aynı iş için get() sağlar. Paths'in sonundaki s, onu Path interface'inden ayırır. Bu bölümde Path.of() ile Paths.get() birbirinin yerine kullanılır.

English: You might notice that both the I/O and NIO.2 classes can interact with a URI. A uniform resource identifier (URI) is a string of characters that identifies a resource. It begins with a schema that indicates the resource type, followed by a path value such as file:// for local file systems and http://, https://, and ftp:// for remote file systems.

Türkçe: Hem I/O hem de NIO.2 class'ları URI ile çalışabilir. Uniform resource identifier (URI), resource'u tanımlayan character sequence'tir. Resource type'ını belirten scheme ile başlar; ardından local file system için file://, remote resource için http://, https:// veya ftp:// gibi path value gelir.

Switching between File and Path

English: Since File and Path both reference locations on disk, it is helpful to be able to convert between them. Luckily, Java makes this easy by providing methods to do just that:

Türkçe: `File` ve `Path` disk'teki location'ları gösterdiğinden aralarında conversion yapabilmek kullanışlıdır. Java bunun için doğrudan method'lar sağlar:

```
File file = new File("rabbit");
Path nowPath = file.toPath();
File backToFile = nowPath.toFile();
```

English: Many older libraries use File, making it convenient to be able to get a File from a Path and vice versa. When working with newer applications, you should rely on NIO.2's Path interface, as it contains a lot more features. For example, only NIO.2 provides FileSystem support, as we are about to discuss.

Türkçe: Pek çok legacy library `File` kullandığından `Path`'ten `File`, `File`'den da `Path` elde etmek kolaydır. Yeni application'larda daha fazla özellik sunan NIO.2 `Path` interface'i tercih edilmelidir. Örneğin `FileSystem` desteğini yalnız NIO.2 sağlar.

Obtaining a Path from the FileSystems Class

English: NIO.2 makes extensive use of creating objects with factory classes. The FileSystems class creates instances of the abstract FileSystem class. The latter includes methods for working with the file system directly. Both Paths.get() and Path.of() are shortcuts for this FileSystem method. Let's rewrite our earlier examples one more time to see how to obtain a Path instance the long way:

Türkçe: NIO.2, fabrika sınıfları ile nesnelerin yaratılmasını kapsamlı bir şekilde kullanır. FileSystems sınıfı, abstract FileSystem sınıfının örneklerini oluşturur. İkincisi, doğrudan file system ile çalışma yöntemlerini içerir. Hem Paths.get() hem de Path.of() bu FileSystem yönteminin kısayollarıdır. Path örneğinin long yolunu nasıl elde edeceğini görmek için önceki örneklerimizi bir kez daha yeniden yazalım:

```
Path zooPath1 = FileSystems.getDefault()
    .getPath("/home/tiger/data/stripes.txt");
Path zooPath2 = FileSystems.getDefault()
    .getPath("/home", "tiger", "data", "stripes.txt");
```

Reviewing I/O and NIO.2 Relationships

English: The model for I/O is smaller, and you only need to understand the File class. In contrast, NIO.2 has more features and makes extensive use of the factory pattern. You should become comfortable with this approach. Many of your interactions with NIO.2 will require two types: an abstract class or interface and a factory or helper class. FIGURE 14.3 shows the relationships among the classes and interface we have used in this chapter so far.

Türkçe: I/O modeli daha küçüktür ve yalnızca `File` class'ını anlamanız gerekir. Buna karşılık, NIO.2 daha fazla özelliğe sahiptir ve fabrika desenini kapsamlı bir şekilde kullanır. Bu yaklaşımla rahat olmalısınız. NIO.2 ile etkileşimlerinizin çoğu iki tür gerektirecektir: bir abstract sınıfı veya arayüzü ve bir fabrika veya yardımcı sınıfı. FIGURE 14.3, bu bölümde şimdiye kadar kullandığımız sınıflar ve arayüz arasındaki ilişkileri gösterir.

English: **FIGURE 14.3 — I/O and NIO.2 class and interface relationships**

```

FileSystems —creates—> FileSystem —creates—> Path
Paths —creates—> Path
java.io.File —converts—> Path
java.net.URI —converts—> Path
Files —uses—> Path

```

Türkçe: ŞEKİL 14.3 — I/O ile NIO.2 class ve interface ilişkileri. `FileSystems`, `FileSystem`, `FileSystem` ve `Paths`, `Path` oluşturur. `Path`, `File` ve `URI` ile dönüştürülebilir; `Files` operation'ları `Path` kullanır.

English: Review FIGURE 14.3 carefully. In particular, keep an eye on whether the class name is singular or plural. Classes with plural names include methods to create or operate on class/interface instances with singular names. Remember, as a convenience (and source of confusion), a `Path` can also be created from the `Path` interface using the static factory `of()` method.

Türkçe: FIGURE 14.3'ü dikkatlice gözden geçirin. Özellikle, sınıf adının tekil veya çoğul olup olmadığına dikkat edin. Çoğul isimlere sahip sınıflar, tekil isimlere sahip class/interface örneklerini oluşturma veya çalıştırma yöntemlerini içerir. Unutmayın, bir kolaylık (ve karışıklık kaynağı) olarak, static fabrika `of()` yöntemini kullanarak `Path` arayüzünden de bir `Path` oluşturulabilir.

English: The `java.io.File` is the I/O class, while `Files` is an NIO.2 helper class.

Türkçe: `java.io.File` bir I/O class'ı, `Files` ise bir NIO.2 yardımcı class'ıdır.

English: `Files` operates on `Path` instances, not `java.io.File` instances. We know this is confusing, but they are from completely different APIs!

Türkçe: `Files`, `java.io.File` instance'larıyla değil `Path` instance'larıyla çalışır. Adları kafa karıştırılabilir de bunlar tamamen farklı API'lere aittir.

English: TABLE 14.2 reviews the APIs we have covered for creating `java.io.File` and `java.nio.file.Path` objects. When reading the table, remember that static methods operate on the class/interface, while instance methods require an instance of an object. Be sure you know this well before proceeding with the rest of the chapter.

Türkçe: TABLE 14.2, `java.io.File` ve `java.nio.file.Path` object'leri oluşturmak için ele aldığımız API'leri özetler. Tabloyu okurken static method'ların class/interface üzerinden çağrıldığını, instance method'ların ise bir object instance'ı gerektirdiğini unutmayın. Bölümün geri kalanına geçmeden önce bu ayrımı iyi bildiğinizden emin olun.

English: TABLE 14.2 — Options for creating `File` and `Path`

Creates	Declared in	Method or constructor
<code>File</code>	<code>File</code>	<code>public File(String pathname)</code>
<code>File</code>	<code>File</code>	<code>public File(File parent, String child)</code>
<code>File</code>	<code>File</code>	<code>public File(String parent, String child)</code>
<code>File</code>	<code>Path</code>	<code>public default File toFile()</code>

Creates	Declared in	Method or constructor
<code>Path</code>	<code>File</code>	<code>public Path toPath()</code>
<code>Path</code>	<code>Path</code>	<code>public static Path of(String first, String... more)</code>
<code>Path</code>	<code>Path</code>	<code>public static Path of(URI uri)</code>
<code>Path</code>	<code>Paths</code>	<code>public static Path get(String first, String... more)</code>
<code>Path</code>	<code>Paths</code>	<code>public static Path get(URI uri)</code>
<code>Path</code>	<code>FileSystem</code>	<code>public Path getPath(String first, String... more)</code>
<code>FileSystem</code>	<code>FileSystems</code>	<code>public static FileSystem getDefault()</code>

Türkçe: TABLO 14.2 — `File` ve `Path` oluşturma seçenekleri. İlk sütun oluşturulan type'ı, ikinci sütun API'nin declare edildiği type'ı gösterir. `File` constructor ile; `Path` ise factory method, conversion method veya `FileSystem.getPath()` ile elde edilir.

Operating on File and Path

English: Now that we know how to create `File` and `Path` objects, we can start using them to do useful things. In this section, we explore the functionality available to us that involves directories.

Türkçe: `File` ve `Path` object'lerinin nasıl oluşturulduğunu bildiğimize göre bunları kullanmaya başlayabiliriz. Bu bölümde directory operation'ları ele alınır.

Using Shared Functionality

English: Many operations can be done using both the I/O and NIO.2 libraries. We present many common APIs in TABLE 14.3 and TABLE 14.4. Although these tables may seem like a lot of methods to learn, many of them are self-explanatory. You can ignore the vararg parameters for now. We explain those later in the chapter.

Türkçe: Birçok işlem hem I/O hem de NIO.2 kütüphaneleri kullanılarak yapılabilir. TABLE 14.3 ve TABLE 14.4'te birçok ortak API sunuyoruz. Bu tablolar öğrenmek için birçok yöntem gibi görünse de, birçoğu kendini açıklayıcıdır. Vararg parametrelerini şimdilik göz ardı edebilirsiniz. Bunları daha sonra bölümde açıklayacağız.

English: TABLE 14.3 — Common `File` and `Path` operations

Description	I/O <code>File</code> instance method	NIO.2 <code>Path</code> instance method
Gets name of file/directory	<code>getName()</code>	<code>getFileName()</code>
Retrieves parent directory, or <code>null</code> if none	<code>getParent()</code>	<code>getParent()</code>

Description	I/O <code>File</code> instance method	NIO.2 <code>Path</code> instance method
Checks whether file/directory is an absolute path	<code>isAbsolute()</code>	<code>isAbsolute()</code>

Türkçe: TABLO 14.3 — Ortak `File` ve `Path` operation'ları

Açıklama	I/O <code>File</code> instance method	NIO.2 <code>Path</code> instance method
File/directory adını alır	<code>getName()</code>	<code>getFileName()</code>
Parent'ı; yoksa <code>null</code> değerini alır	<code>getParent()</code>	<code>getParent()</code>
Absolute path olup olmadığını test eder	<code>isAbsolute()</code>	<code>isAbsolute()</code>

English: TABLE 14.4 — Common `File` and `Files` operations

Description	I/O <code>File</code> instance method	NIO.2 <code>Files</code> static method
Deletes file/directory	<code>delete()</code>	<code>deleteIfExists(Path p)</code> throws <code>IOException</code>
Checks whether file/directory exists	<code>exists()</code>	<code>exists(Path p, LinkOption... o)</code>
Retrieves absolute path	<code>getAbsolutePath()</code>	<code>toAbsolutePath()</code>
Checks whether resource is a directory	<code>isDirectory()</code>	<code>isDirectory(Path p, LinkOption... o)</code>
Checks whether resource is a file	<code>isFile()</code>	<code>isRegularFile(Path p, LinkOption... o)</code>

Türkçe: TABLO 14.4 — Ortak `File` ve `Files` operation'ları

Açıklama	I/O <code>File</code> instance method	NIO.2 <code>Files</code> static method
File/directory siler	<code>delete()</code>	<code>deleteIfExists(Path p)</code> throws <code>IOException</code>
File/directory var mı test eder	<code>exists()</code>	<code>exists(Path p, LinkOption... o)</code>
Absolute path'i alır	<code>getAbsolutePath()</code>	<code>toAbsolutePath()</code>
Resource directory mi test eder	<code>isDirectory()</code>	<code>isDirectory(Path p, LinkOption... o)</code>
Resource regular file mi test eder	<code>isFile()</code>	<code>isRegularFile(Path p, LinkOption... o)</code>

English: **TABLE 14.4 — Common `File` and `Files` operations (continued)**

Description	I/O <code>File</code> instance method	NIO.2 <code>Files</code> static method
Returns last-modified time	<code>lastModified()</code>	<code>getLastModifiedTime(Path p, LinkOption... o)</code> throws <code>IOException</code>
Retrieves number of bytes in file	<code>length()</code>	<code>size(Path p)</code> throws <code>IOException</code>
Lists directory contents	<code>listFiles()</code>	<code>list(Path p)</code> throws <code>IOException</code>
Creates directory	<code>mkdir()</code>	<code>createDirectory(Path p, FileAttribute... a)</code> throws <code>IOException</code>
Creates directory and missing parents	<code>mkdirs()</code>	<code>createDirectories(Path p, FileAttribute... a)</code> throws <code>IOException</code>
Renames/moves denoted file or directory	<code>renameTo(File dest)</code>	<code>move(Path src, Path dest, CopyOption... o)</code> throws <code>IOException</code>

Türkçe: **TABLO 14.4 — Ortak `File` ve `Files` operation'ları (devam)**

Açıklama	I/O <code>File</code> instance method	NIO.2 <code>Files</code> static method
Last-modified time'ı döndürür	<code>lastModified()</code>	<code>getLastModifiedTime(Path p, LinkOption... o)</code> throws <code>IOException</code>
File byte sayısını alır	<code>length()</code>	<code>size(Path p)</code> throws <code>IOException</code>
Directory içeriğini listeler	<code>listFiles()</code>	<code>list(Path p)</code> throws <code>IOException</code>
Directory oluşturur	<code>mkdir()</code>	<code>createDirectory(Path p, FileAttribute... a)</code> throws <code>IOException</code>
Directory ve eksik parent'ları oluşturur	<code>mkdirs()</code>	<code>createDirectories(Path p, FileAttribute... a)</code> throws <code>IOException</code>
File/directory'yi yeniden adlandırır veya taşır	<code>renameTo(File dest)</code>	<code>move(Path src, Path dest, CopyOption... o)</code> throws <code>IOException</code>

English: Now let's try to use some of these APIs. The following is a sample program using only legacy I/O APIs. Given a file path, it outputs information about the file or directory, such as whether it exists, what files are contained within it, and so forth:

Türkçe: Şimdi bu API'lerden bazılarını kullanalım. Aşağıdaki örnek program yalnızca legacy I/O API'lerini kullanır. Verilen bir file path için hedefin var olup olmadığı, file mı directory mi olduğu ve directory ise hangi file'ları içerdiği gibi bilgileri yazdırır:

```
public static void io() {
    var file = new File("C:\\data\\zoo.txt");
    if (file.exists()) {
        System.out.println("Absolute Path: " + file.getAbsolutePath());
        System.out.println("Is Directory: " + file.isDirectory());
        System.out.println("Parent Path: " + file.getParent());
        if (file.isFile()) {
            System.out.println("Size: " + file.length());
            System.out.println("Last Modified: " + file.lastModified());
        } else {
```

```
            for (File subfile: file.listFiles()) {
                System.out.println(" " + subfile.getName());
            } } } }
```

English: If the path provided points to a valid file, the program outputs something similar to the following due to the `if` statement on line 16:

Türkçe: Verilen path geçerli bir file'ı gösteriyorsa program, 16. satırdaki `if` statement'ı nedeniyle aşağıdakine benzer bir çıktı üretir:

```
Absolute Path: C:\data\zoo.txt
Is Directory: false
Parent Path: C:\data
Size: 12382
Last Modified: 1650610000000
```

English: Finally, if the path provided points to a valid directory, such as `C:\data`, the program outputs something similar to the following, thanks to the `else` block:

Türkçe: Verilen path `C:\data` gibi geçerli bir directory'yi gösteriyorsa program, `else` block'u sayesinde aşağıdakine benzer bir çıktı üretir:

```
Absolute Path: C:\data
Is Directory: true
Parent Path: C:\
employees.txt
zoo.txt
zoo-backup.txt
```

English: In these examples, you see that the output of an I/O-based program is completely dependent on the directories and files available at runtime in the underlying file system.

Türkçe: Bu örneklerde, bir I/O-based programının çıktısının, altta yatan file system içinde çalışma zamanında mevcut olan dizinlere ve dosyalara tamamen bağlı olduğunu görürsünüz.

English: On the exam, you might see paths that look like files but are directories or vice versa. For example, `/data/zoo.txt` could be a file or a directory, even though it has a file extension. Don't assume it is either unless the question tells you it is!

Türkçe: Sınavda file'a benzeyen ama directory olan ya da bunun tersi path'ler görebilirsiniz. Örneğin `/data/zoo.txt`, file extension'a benzeyen sonuna rağmen file da directory de olabilir. Soru açıkça söylemedikçe ikisini de varsaymayın.

English: In the previous example, we used two backslashes (\\) in the path String, such as C:\\data\\zoo.txt. When the compiler sees a \\ inside a String expression, it interprets it as a single \ value.

Türkçe: Önceki örnekte path'i temsil eden `String` içinde `C:\\data\\zoo.txt` örneğindeki gibi iki backslash (\\) kullandık. Compiler bir `String` expression'ında \\ gördüğünde bunu tek bir \ karakteri olarak yorumlar.

English: Now, let's write that same program using only NIO.2 and see how it differs:

Türkçe: Şimdi, sadece NIO.2 kullanarak aynı programı yazalım ve nasıl farklılaştığını görelim:

```
public static void nio() throws IOException {
    var path = Path.of("C:\\data\\zoo.txt");
    if (Files.exists(path)) {
        System.out.println("Absolute Path: " + path.toAbsolutePath());
        System.out.println("Is Directory: " + Files.isDirectory(path));
        System.out.println("Parent Path: " + path.getParent());
        if (Files.isRegularFile(path)) {
            System.out.println("Size: " + Files.size(path));
        }
    }
}
```

```
System.out.println("Last Modified: "
    + Files.getLastModifiedTime(path));
} else {
    try (Stream<Path> stream = Files.list(path)) {
        stream.forEach(p ->
            System.out.println(" " + p.getName()));
    } } }
```

English: Most of this example is equivalent and replaces the I/O method calls in the previous tables with the NIO.2 versions. However, there are key differences. First, line 25 declares a checked exception. More APIs in NIO.2 throw `IOException` than the I/O APIs did. In this case, `Files.size()`, `Files.getLastModifiedTime()`, and `Files.list()` throw an `IOException`.

Türkçe: Örneğin büyük bölümü önceki programla eşdeğerdir; tablolardaki I/O method call'larının yerini NIO.2 sürümleri almıştır. Ancak önemli farklar vardır. İlk olarak 25. satır bir checked exception declare eder. NIO.2'de `IOException` fırlatan API sayısı legacy I/O API'lerinden fazladır. Bu örnekte `Files.size()`, `Files.getLastModifiedTime()` ve `Files.list()` bir `IOException` fırlatır.

English: Second, lines 36–39 use a `Stream` and a lambda instead of a loop. Since streams use lazy evaluation, this means the method will load each path element as needed, rather than the entire directory at once.

Türkçe: İkinci olarak 36–39. satırlar, loop yerine bir `Stream` ve lambda kullanır. Stream'ler lazy evaluation (tembel değerlendirme) uyguladığı için method bütün directory'yi bir kerede yüklemek yerine her `Path` element'ini gerektiğinde yükler.

English: Closing the Stream Did you notice that in the last code sample, we put our Stream object inside a try-with-resources? The NIO.2 stream-based methods open a connection to the file system that must be properly closed; otherwise, a resource leak could ensue. A resource leak within the file system means the path may be locked from modification long after the process that used it is completed.

Türkçe: Stream'i Kapatma — Son kod örneğinde `Stream` object'ini bir try-with-resources statement'ı içine koyduğumuzu fark ettiniz mi? NIO.2'nin stream tabanlı method'ları file system'e bir connection açar ve bu connection düzgün biçimde kapatılmalıdır; aksi halde resource leak (kaynak sızıntısı) oluşabilir. File system'deki bir resource leak, onu kullanan process tamamlandıktan çok sonra bile path'in değişikliklere karşı kilitli kalmasına yol açabilir.

English: If you assumed that a stream's terminal operation would automatically close the underlying file resources, you'd be wrong. There was a lot of debate about this behavior when it was first presented; in short, requiring developers to close the stream won out.

Türkçe: Bir stream'in terminal operation'ının altında yatan file resource'larını otomatik olarak kapatacağını varsayarsanız yanılırsınız. Bu davranış ilk tasarlandığında çok tartışıldı; sonuçta stream'i kapatma sorumluluğunun geliştiricide kalmasına karar verildi.

English: On the plus side, not all streams need to be closed: only those that open resources, like the ones found in NIO.2. For instance, you didn't need to close any of the streams you worked with in Chapter 10, "Streams." Finally, the exam doesn't always properly close NIO.2 resources. To match the exam, we sometimes skip closing NIO.2 resources in review and practice questions. Always use try-with-resources statements with these NIO.2 methods in your own code.

Türkçe: Artı tarafta, tüm streams kapalı olması gerekmez: sadece NIO.2'de bulunanlar gibi kaynakları açanlar. Örneğin, 10. Bölümde birlikte çalıştığınız streams 'nin hiçbirini kapatmanıza gerek yoktu, "Streams." Son olarak, sınav her zaman NIO.2 kaynaklarını düzgün bir şekilde kapatmaz. Sınava uymak için, bazen inceleme ve uygulama sorularında NIO.2 kaynaklarını kapatmayı atlıyoruz. Bu NIO.2 yöntemleriyle her zaman try-with-resources statements kullanın.

English: For the remainder of this section, we only discuss the NIO.2 methods, because they are more important. There is also more to know about them, and they are more likely to come up on the exam.

Türkçe: Bu bölümün geri kalanı için, sadece NIO.2 yöntemlerini tartışıyoruz, çünkü bunlar daha önemli. Ayrıca onlar hakkında daha fazla bilgi var ve sınava girme olasılıkları daha yüksektir.

Handling Methods That Declare IOException

English: Many of the methods presented in this chapter declare IOException. Common causes of a method throwing this exception include the following:

Türkçe: Bu bölümde sunulan yöntemlerin çoğu IOException olarak beyan eder. Bu istisnayı atan bir yöntemin yaygın nedenleri şunlardır:

English: Loss of communication to the underlying file system.

Türkçe: Alttaki file system ile iletişim kaybı.

English: File or directory exists but cannot be accessed or modified.

Türkçe: Dosya veya dizin var ancak erişilemiyor veya değiştirilemiyor.

English: File exists but cannot be overwritten.

Türkçe: Dosya var ama üzerine yazılamaz.

English: File or directory is required but does not exist.

Türkçe: Dosya veya dizin gereklidir, ancak mevcut değildir.

English: Methods that access or change files and directories, such as those in the `Files` class, often declare `IOException`. There are exceptions to this rule, as we will see. For example, the method `Files.exists()` does not declare `IOException`. If it did throw an exception when the file did not exist, it would never be able to return `false`! As a rule of thumb, if a NIO.2 method declares an `IOException`, it usually requires the paths it operates on to exist.

Türkçe: `Files` class'ındakiler gibi file ve directory'lere erişen ya da bunları değiştiren method'lar çoğunlukla `IOException` declare eder; ancak bu kuralın istisnaları vardır. Örneğin `Files.exists()` `IOException` declare etmez. File bulunmadığında exception fırlatsaydı hiçbir zaman `false` döndüremezdi. Genel kural olarak, bir NIO.2 method'u `IOException` declare ediyorsa üzerinde çalıştığı path'lerin genellikle var olması gerekir.

Providing NIO.2 Optional Parameters

English: Many of the NIO.2 methods in this chapter include a varargs that takes an optional list of values. TABLE 14.5 presents the arguments you should be familiar with for the exam.

Türkçe: Bu bölümdeki NIO.2 yöntemlerinin birçoğu, isteğe bağlı bir değerler listesi alan bir vararg içerir. TABLE 14.5 sınav için aşına olmanız gereken argümanları sunar.

English: TABLE 14.5 — Common NIO.2 method arguments

Enum type	Interface inherited	Enum value	Details
<code>LinkOption</code>	<code>CopyOption</code> , <code>OpenOption</code>	<code>NOFOLLOW_LINKS</code>	Do not follow symbolic links
<code>StandardCopyOption</code>	<code>CopyOption</code>	<code>ATOMIC_MOVE</code>	Move file as an atomic file-system operation
<code>StandardCopyOption</code>	<code>CopyOption</code>	<code>COPY_ATTRIBUTES</code>	Copy existing attributes to new file
<code>StandardCopyOption</code>	<code>CopyOption</code>	<code>REPLACE_EXISTING</code>	Overwrite file if it already exists

Türkçe: TABLO 14.5 — Yaygın NIO.2 method argument'ları

Enum type	Miras alınan interface	Enum değeri	Ayrıntı
<code>LinkOption</code>	<code>CopyOption</code> , <code>OpenOption</code>	<code>NOFOLLOW_LINKS</code>	Symbolic link'leri izleme
<code>StandardCopyOption</code>	<code>CopyOption</code>	<code>ATOMIC_MOVE</code>	File'ı atomic file-system operation olarak taşı
<code>StandardCopyOption</code>	<code>CopyOption</code>	<code>COPY_ATTRIBUTES</code>	Mevcut attribute'ları yeni file'a kopyala

Enum type	Miras alınan interface	Enum değeri	Ayrıntı
<code>StandardCopyOption</code>	<code>CopyOption</code>	<code>REPLACE_EXISTING</code>	File zaten varsa üzerine yaz

English: TABLE 14.5 — Common NIO.2 method arguments (continued)

Enum type	Interface inherited	Enum value	Details
<code>StandardOpenOption</code>	<code>OpenOption</code>	<code>APPEND</code>	If open for write, append to the end
<code>StandardOpenOption</code>	<code>OpenOption</code>	<code>CREATE</code>	Create a new file if it does not exist
<code>StandardOpenOption</code>	<code>OpenOption</code>	<code>CREATE_NEW</code>	Create only if absent; fail otherwise
<code>StandardOpenOption</code>	<code>OpenOption</code>	<code>READ</code>	Open for read access
<code>StandardOpenOption</code>	<code>OpenOption</code>	<code>TRUNCATE_EXISTING</code>	If open for write, erase the file and write from the beginning
<code>StandardOpenOption</code>	<code>OpenOption</code>	<code>WRITE</code>	Open for write access
<code>FileVisitOption</code>	N/A	<code>FOLLOW_LINKS</code>	Follow symbolic links

Türkçe: TABLO 14.5 — Yaygın NIO.2 method argument'ları (devam). `StandardOpenOption` open/read/write davranışını; `FileVisitOption` directory traversal'ın symbolic link davranışını belirler.

English: With the exceptions of `Files.copy()` and `Files.move()`, we won't discuss these varargs parameters each time we present a method. Their behavior should be straightforward, though. For example, can you figure out what the following call to `Files.exists()` with the `LinkOption` does in the following code snippet?

Türkçe: `Files.copy()` ve `Files.move()` istisnaları dışında, bir yöntem sunduğumuzda bu vararg parametrelerini tartışmayacağız. Davranışları yine de açık olmalıdır. Örneğin, aşağıdaki kod snippet'inde `LinkOption` ile aşağıdaki `Files.exists()` çağrısının ne yaptığını bulabilir misiniz?

```
Path path = Paths.get("schedule.xml");
boolean exists = Files.exists(path, LinkOption.NOFOLLOW_LINKS);
```

English: The `Files.exists()` simply checks whether a file exists. But if the parameter is a symbolic link, the method checks whether the target of the symbolic link exists, instead. Providing `LinkOption.NOFOLLOW_LINKS` means the default behavior will be overridden, and the method will check whether the symbolic link itself exists.

Türkçe: `Files.exists()` basitçe bir dosyanın var olup olmadığını kontrol eder. Ancak parametre symbolic link ise, yöntem bunun yerine symbolic link hedefinin var olup olmadığını kontrol eder. `LinkOption.NOFOLLOW_LINKS` sağlamak, varsayılan davranışın geçersiz olacağı anlamına gelir ve yöntem symbolic link'un kendisinin var olup olmadığını kontrol eder.

English: Note that some of the enums in TABLE 14.5 inherit an interface. That means some methods accept a variety of enum types. For example, the `Files.move()` method takes a `CopyOption` vararg so it can take enums of different types, and more options can be added over time.

Türkçe: TABLE 14.5'teki enumların bazılarının bir arayüze sahip olduğunu unutmayın. Bu, bazı yöntemlerin çeşitli enum türlerini kabul ettiği anlamına gelir. Örneğin, `Files.move()` yöntemi bir `CopyOption` vararg alır, böylece farklı türlerde enumlar alabilir ve zaman içinde daha fazla seçenek eklenebilir.

```
void copy(Path source, Path target) throws IOException {
    Files.move(source, target,
        LinkOption.NOFOLLOW_LINKS,
        StandardCopyOption.ATOMIC_MOVE);
}
```

Interacting with NIO.2 Paths

English: Just like `String` values, `Path` instances are immutable. In the following example, the `Path` operation on the second line is lost since `p` is immutable:

Türkçe: `String` değerleri gibi `Path` örnekleri de değiştirilemez. Aşağıdaki örnekte, ikinci satırdaki `Path` işlemi, `p` değiştirilemez olduğundan kaybolur:

```
Path p = Path.of("whale");
p.resolve("krill");
System.out.println(p); // whale
```

English: Many of the methods available in the `Path` interface transform the path value in some way and return a new `Path` object, allowing the methods to be chained. We demonstrate chaining in the following example, the details of which we discuss in this section of the chapter:

Türkçe: `Path` interface'indeki method'ların çoğu path değerini bir biçimde dönüştürüp yeni bir `Path` object'i döndürür; böylece method call'ları chain edilebilir. Ayrıntılarını bu bölümde açıklayacağımız chaining örneği şöyledir:

```
Path.of("/zoo/../../home").getParent().normalize().toAbsolutePath();
```

Viewing the Path

English: The `Path` interface contains three methods to retrieve basic information about the path representation. The `toString()` method returns a `String` representation of the entire path. In fact, it is the only method in the `Path` interface to return a `String`. Many of the other methods in the `Path` interface return `Path` instances.

Türkçe: `Path` interface'i path gösterimi hakkında temel bilgi veren üç method içerir. `toString()`, path'in tamamının `String` gösterimini döndürür. Hatta `Path` interface'inde `String` döndüren tek method budur; diğer method'ların birçoğu `Path` instance'ı döndürür.

English: The `getNameCount()` and `getName()` methods are often used together to retrieve the number of elements in the path and a reference to each element, respectively. These two methods do not include the root directory as part of the path.

Türkçe: `getNameCount()` ve `getName()` sırasıyla path'teki element sayısını ve her element'e ait referansı almak için genellikle birlikte kullanılır. Bu iki method root directory'yi path'in bir parçası olarak saymaz.

```
Path path = Paths.get("/land/hippo/harry.happy");
System.out.println("The Path Name is: " + path);
for(int i=0; i<path.getNameCount(); i++)
System.out.println(" Element " + i + " is: " + path.getName(i));
```

English: Notice that we didn't call `toString()` explicitly on the second line. Remember, Java calls `toString()` on any Object as part of string concatenation. We use this feature throughout the examples in this chapter.

Türkçe: İkinci satırda `toString()` i açıkça çağırmadığımıza dikkat edin. Java, string concatenation sırasında her Object için `toString()` i kendiliğinden çağırır. Bölüm boyunca örneklerde bu özellikten yararlanıyoruz.

English: The code prints the following:

Türkçe: Kod aşağıdakileri yazdırır:

```
The Path Name is: /land/hippo/harry.happy
Element 0 is: land
Element 1 is: hippo
Element 2 is: harry.happy
```

English: Even though this is an absolute path, the root element is not included in the list of names. As we said, these methods do not consider the root part of the path.

Türkçe: Bu bir absolute path olmasına rağmen, kök eleman isim listesine dahil değildir. Dediğim gibi, bu yöntemler yolun kök kısmını dikkate almaz.

```
var p = Path.of("/");
System.out.print(p.getNameCount()); // 0
System.out.print(p.getName(0)); // IllegalArgumentException
```

English: Notice that if you try to call `getName()` with an invalid index, it will throw an exception at runtime.

Türkçe: `getName()` i geçersiz bir index ile çağırırsanız runtime'da exception fırlatılır.

English: Our examples print `/` as the file separator character because of the system we are using. Your actual output may vary throughout this chapter.

Türkçe: Kullandığımız sistemde file separator karakteri `/` olduğu için örneklerimiz bunu yazdırır. Gerçek çıktınız bölüm boyunca işletim sisteminize göre değişebilir.

Creating Part of the Path

English: The Path interface includes the subpath() method to select portions of a path. It takes two parameters: an inclusive beginIndex and an exclusive endIndex. This should sound familiar as it is how String's substring() method works, as you saw in Chapter 4, "Core APIs." The following code snippet shows how subpath() works. We also print the elements of the Path using getName() so that you can see how the indices are used.

Türkçe: Path arayüzü, bir yolun bölümlerini seçmek için subpath() yöntemini içerir. İki parametre gerektirir: kapsayıcı bir beginIndex ve özel bir endIndex. Bu, String substring() yönteminin nasıl çalıştığı gibi tanıdık gelmelidir, Bölüm 4'te gördüğümüz gibi, "Core API'leri." Aşağıdaki kod snippet'i subpath()'un nasıl çalıştığını gösterir. Endekslerin nasıl kullanıldığını görebilmeniz için Path öğelerini getName() kullanarak da yazdırıyoruz.

```
var p = Paths.get("/mammal/omnivore/raccoon.image");
System.out.println("Path is: " + p);
for (int i = 0; i < p.getNameCount(); i++) {
    System.out.println("Element " + i + " is: " + p.getName(i));
}
System.out.println();
System.out.println("subpath(0,3): " + p.subpath(0, 3));
System.out.println("subpath(1,2): " + p.subpath(1, 2));
System.out.println("subpath(1,3): " + p.subpath(1, 3));
```

English: The output of this code snippet is the following:

Türkçe: Bu kod snippet'inin çıktısı aşağıdaki gibidir:

```
Path is: /mammal/omnivore/raccoon.image
Element 0 is: mammal
Element 1 is: omnivore
Element 2 is: raccoon.image
subpath(0,3): mammal/omnivore/raccoon.image
subpath(1,2): omnivore
subpath(1,3): omnivore/raccoon.image
```

English: Like getNameCount() and getName(), subpath() is zero-indexed and does not include the root. Also like getName(), subpath() throws an exception if invalid indices are provided.

Türkçe: getNameCount() ve getName() gibi, subpath() sıfır indekslidir ve kök içermez. Ayrıca getName() gibi, subpath() de geçersiz endeksler sağlanmışsa bir istisna atar.

```
var q = p.subpath(0, 4); // IllegalArgumentException
var x = p.subpath(1, 1); // IllegalArgumentException
```

English: The first example throws an exception at runtime, since the maximum index value allowed is 3. The second example throws an exception since the start and end indexes are the same, leading to an empty path value.

Türkçe: İlk örnek çalışma zamanında bir istisna atar, çünkü izin verilen maksimum indeks değeri 3'tür. İkinci örnek, başlangıç ve bitiş indeksleri aynı olduğundan bir istisna atar ve bu da boş bir yol değerine yol açar.

Accessing Path Elements

English: The Path interface contains numerous methods for retrieving particular elements of a Path, returned as Path objects themselves. The `getFileName()` method returns the Path element of the current file or directory, while `getParent()` returns the full path of the containing directory. The `getParent()` method returns null if operated on the root path or at the top of a relative path. The `getRoot()` method returns the root element of the file within the file system, or null if the path is a relative path.

Türkçe: `Path` interface'i, bir path'in belirli element'lerini yine `Path` object'i olarak almak için birçok method içerir. `getFileName()` current file veya directory'ye ait son `Path` element'ini; `getParent()` ise onu içeren directory'nin tam path'ini döndürür. `getParent()`, root path'te ya da relative path'in en üstünde çağrılırsa `null` döndürür. `getRoot()` file system'deki root element'i döndürür; path relative ise sonuç `null` olur.

English: Consider the following method, which prints various Path elements:

Türkçe: Çeşitli Path öğelerini yazdıran aşağıdaki yöntemi düşünün:

```
public void printPathInformation(Path path) {
    System.out.println("Filename is: " + path.getFileName());
    System.out.println(" Root is: " + path.getRoot());
    Path currentParent = path;
    while((currentParent = currentParent.getParent()) != null)
        System.out.println(" Current parent is: " + currentParent);
    System.out.println();
}
```

English: The while loop in the `printPathInformation()` method continues until `getParent()` returns null. We apply this method to the following three paths:

Türkçe: `printPathInformation()` yöntemindeki while döngüsü `getParent()` null döndürene kadar devam eder. Bu yöntemi aşağıdaki üç yola uygularız:

```
printPathInformation(Path.of("zoo"));
printPathInformation(Path.of("/zoo/armadillo/shells.txt"));
printPathInformation(Path.of("./armadillo/../shells.txt"));
```

English: This sample application produces the following output:

Türkçe: Bu örnek uygulama aşağıdaki çıktıyı üretir:

```
Filename is: zoo
Root is: null
Filename is: shells.txt
Root is: /
Current parent is: /zoo/armadillo
Current parent is: /zoo
Current parent is: /
Filename is: shells.txt
Root is: null
Current parent is: ./armadillo/..
Current parent is: ./armadillo
Current parent is: .
```

English: Reviewing the sample output, you can see the difference in the behavior of `getRoot()` on absolute and relative paths. As you can see in the first and last examples, the `getParent()` method does not traverse relative paths outside the current working directory.

Türkçe: Örnek çıktı, `getRoot()` un absolute ve relative path'lerdeki davranış farkını gösterir. İlk ve son örneklerde görüldüğü gibi `getParent()`, relative path üzerinde current working directory'nin üstüne çıkmaz.

English: You also see that these methods do not resolve the path symbols and treat them as a distinct part of the path. While most of the methods in this part of the chapter treat path symbols as part of the path, we present one shortly that cleans up path symbols.

Türkçe: Bu method'ların path symbol'ları çözmediğini ve her symbol'a path'in ayrı bir parçası gibi davrandığını da görürsünüz. Bölümün bu kısmındaki method'ların çoğu path symbol'larını path'in bir parçası olarak korur; birazdan bunları temizleyen bir method göreceğiz.

Resolving Paths

English: Suppose you want to concatenate paths in a manner similar to how we concatenate strings. The `resolve()` method provides overloaded versions that let you pass either a `Path` or `String` parameter. The object on which the `resolve()` method is invoked becomes the basis of the new `Path` object, with the input argument being appended onto the `Path`. Let's see what happens if we apply `resolve()` to an absolute path and a relative path:

Türkçe: String'leri concatenate ettiğimiz gibi path'leri birleştirmek istediğimizi varsayalım. `resolve()` un `Path` ya da `String` parameter kabul eden overloaded sürümleri vardır. `resolve()` un çağırıldığı object yeni `Path` object'inin temelini oluşturur; input argument bu path'in sonuna eklenir. `resolve()` u bir absolute path ile bir relative path üzerinde uygularsak ne olduğuna bakalım:

```
Path path1 = Path.of("/cats/../panther");
Path path2 = Path.of("food");
System.out.println(path1.resolve(path2));
```

English: The code snippet generates the following output:

Türkçe: Kod snippet'i aşağıdaki çıktıyı oluşturur:

```
/cats/../panther/food
```

English: Like the other methods we've seen, `resolve()` does not clean up path symbols. In this example, the input argument to the `resolve()` method was a relative path, but what if it had been an absolute path?

Türkçe: Gördüğümüz diğer method'lar gibi `resolve()` da path symbol'larını temizlemez. Bu örnekte `resolve()` a verilen input argument bir relative path'ti. Peki absolute path olsaydı ne olurdu?

```
Path path3 = Path.of("/turkey/food");
System.out.println(path3.resolve("/tiger/cage"));
```

English: Since the input parameter is an absolute path, the output would be the following:

Türkçe: Girdi parametresi bir absolute path olduğundan, çıktı aşağıdaki olacaktır:

```
/tiger/cage
```

English: For the exam, you should be cognizant of mixing absolute and relative paths with the `resolve()` method. If an absolute path is provided as input to the method, that is the value returned. Simply put, you cannot combine two absolute paths using `resolve()`.

Türkçe: Sınav için mutlak ve relative paths ile `resolve()` yöntemini karıştırmanın farkında olmalısınız. Yönteme girdi olarak bir absolute path sağlanıyorsa, bu döndürülen değerdir. Basitçe söylemek gerekirse, `resolve()` kullanarak iki absolute paths birleştiremezsiniz.

English: On the exam, when you see `resolve()`, think concatenation.

Türkçe: Sınavda, `resolve()` gördüğünüzde, birleştirmeyi düşünün.

Relativizing a Path

English: The `Path` interface includes a `relativize()` method for constructing the relative path from one `Path` to another, often using path symbols. What do you think the following examples will print?

Türkçe: `Path` interface'indeki `relativize()` method'u, bir `Path`ten diğerine giden relative path'i çoğu zaman path symbol'larından yararlanarak oluşturur. Aşağıdaki örnekler ne yazdırır?

```
var path1 = Path.of("fish.txt");
var path2 = Path.of("friendly/birds.txt");
System.out.println(path1.relativize(path2));
System.out.println(path2.relativize(path1));
```

English: The examples print the following:

Türkçe: Örnekler aşağıdakileri yazdırır:

```
../friendly/birds.txt
../../fish.txt
```

English: The idea is this: if you are pointed at a path in the file system, what steps would you need to take to reach the other path? For example, to get to `fish.txt` from `friendly/birds.txt`, you need to go up two levels (the file itself counts as one level) and then select `fish.txt`.

Türkçe: Buradaki fikir şudur: File system içinde bir path'ten diğerine ulaşmak için hangi adımlar gerekir? Örneğin `friendly/birds.txt`ten `fish.txt`e gitmek için iki level yukarı çıkmanız (file'ın kendisi de bir level sayılır), ardından `fish.txt`i seçmeniz gerekir.

English: If both path values are relative, the `relativize()` method computes the paths as if they are in the same current working directory. Alternatively, if both path values are absolute, the method computes the relative path from one absolute location to another, regardless of the current working directory. The following example demonstrates this property when run on a Windows computer:

Türkçe: İki path de relative ise `relativize()`, bunları aynı current working directory'de bulunuyormuş gibi hesaplar. İki path de absolute ise current working directory'den bağımsız biçimde bir absolute konumdan diğerine giden relative path'i hesaplar. Aşağıdaki Windows örneği bu özelliği gösterir:

```
Path path3 = Paths.get("E:\\habitat");
Path path4 = Paths.get("E:\\sanctuary\\raven\\poe.txt");
System.out.println(path3.relativize(path4));
System.out.println(path4.relativize(path3));
```

English: This code snippet produces the following output:

Türkçe: Bu kod snippet'i aşağıdaki çıktıyı üretir:

```
..\sanctuary\raven\poe.txt
..\..\..\habitat
```

English: The `relativize()` method requires both paths to be absolute or relative and throws an exception if the types are mixed.

Türkçe: `relativize()` için iki path'in de absolute ya da ikisinin de relative olması gerekir; türler karıştırılırsa method exception fırlatır.

```
Path path1 = Paths.get("/primate/chimpanzee");
Path path2 = Paths.get("bananas.txt");
path1.relativize(path2); // IllegalArgumentException
```

English: On Windows-based systems, it also requires that if absolute paths are used, both paths must have the same root directory or drive letter. For example, the following would also throw an `IllegalArgumentException` on a Windows-based system:

Türkçe: Windows tabanlı sistemlerde absolute path kullanılıyorsa iki path'in de aynı root directory'ye veya drive letter'a sahip olması gerekir. Örneğin aşağıdaki kod Windows'ta `IllegalArgumentException` fırlatır:

```
Path path3 = Paths.get("C:\\primate\\chimpanzee");
Path path4 = Paths.get("D:\\storage\\bananas.txt");
path3.relativize(path4); // IllegalArgumentException
```

Normalizing a Path

English: So far, we've presented a number of examples that included path symbols that were unnecessary. Luckily, Java provides the `normalize()` method to eliminate unnecessary redundancies in a path.

Türkçe: Şimdiye kadar gereksiz path symbol'ları içeren çeşitli örnekler gördük. Java, bir path'teki azaltılabilir fazlalıkları gidermek için `normalize()` method'unu sunar.

English: Remember, the path symbol `..` refers to the parent directory, while the path symbol `.` refers to the current directory. We can apply `normalize()` to some of our previous paths.

Türkçe: `..` path symbol'ı parent directory'yi, `.` ise current directory'yi gösterir. Önceki path'lerimizden bazılarını `normalize()` uygulayabiliriz.

```
var p1 = Path.of("../armadillo/../shells.txt");
System.out.println(p1.normalize()); // shells.txt
```

```
var p2 = Path.of("/cats/../panther/food");
System.out.println(p2.normalize()); // /panther/food
var p3 = Path.of("../../fish.txt");
System.out.println(p3.normalize()); // ../../fish.txt
```

English: The first two examples apply the path symbols to remove the redundancies, but what about the last one? That is as simplified as it can be. The `normalize()` method does not remove all of the path symbols, only the ones that can be reduced.

Türkçe: İlk iki örnekte path symbol'ları uygulanarak fazlalıklar giderilir. Son örnek ise zaten mümkün olan en sade hâldedir. `normalize()` bütün path symbol'larını değil, yalnızca azaltılabilenleri kaldırır.

English: The `normalize()` method also allows us to compare equivalent paths. Consider the following example:

Türkçe: `normalize()` yöntemi de eşdeğer yolları karşılaştırmamızı sağlar. Aşağıdaki örneği ele alalım:

```
var p1 = Paths.get("/pony/../weather.txt");
var p2 = Paths.get("/weather.txt");
System.out.println(p1.equals(p2)); // false
System.out.println(p1.normalize().equals(p2.normalize())); // true
```

English: The `equals()` method returns true if two paths represent the same value. In the first comparison, the path values are different. In the second comparison, the path values have both been reduced to the same normalized value, `/weather.txt`. This is the primary function of the `normalize()` method: to allow us to better compare different paths.

Türkçe: `equals()`, iki path aynı value'yu temsil ediyorsa `true` döndürür. İlk karşılaştırmada path value'ları farklıdır. İkincide iki value da aynı normalized `/weather.txt` değerine indirgenmiştir. `normalize()` in temel amacı farklı path'lerin karşılaştırılmasını kolaylaştırmaktır.

Retrieving the Real File System Path

English: While working with theoretical paths is useful, sometimes you want to verify that the path exists within the file system using `toRealPath()`. This method is similar to `normalize()` in that it eliminates any redundant path symbols. It is also similar to `toAbsolutePath()`, in that it will join the path with the current working directory if the path is relative.

Türkçe: Henüz file system'de var olması gerekmeyen teorik path'lerle çalışmak yararlıdır; ancak bazen path'in gerçekten var olduğunu `toRealPath()` ile doğrulamak isteriz. Bu method, gereksiz path symbol'larını kaldırdığı için `normalize()` a benzer. Path relative ise onu current working directory ile birleştirdiği için `toAbsolutePath()` e de benzer.

English: Unlike those two methods, though, `toRealPath()` will throw an exception if the path does not exist. In addition, it will follow symbolic links, with an optional `LinkOption` varargs parameter to ignore them.

Türkçe: Bu iki method'dan farklı olarak `toRealPath()`, path yoksa exception fırlatır. Ayrıca varsayılan olarak symbolic link'leri takip eder; isteğe bağlı `LinkOption` varargs parameter'ıyla bunların izlenmemesi sağlanabilir.

English: Let's say that we have a file system in which we have a symbolic link from /zebra to /horse. What do you think the following will print, given a current working directory of /horse/schedule?

Türkçe: File sistem'de /zebradan /horsea symbolic link olduğunu varsayalım. Current working directory /horse/schedule iken aşağıdaki kod ne yazdırır?

```
System.out.println(Paths.get("/zebra/food.txt").toRealPath());
System.out.println(Paths.get("../food.txt").toRealPath());
```

English: The output of both lines is the following:

Türkçe: Her iki satırın çıktısı aşağıdaki gibidir:

```
/horse/food.txt
```

English: In this example, the absolute and relative paths both resolve to the same absolute file, as the symbolic link points to a real file within the file system. We can also use the toRealPath() method to gain access to the current working directory as a Path object.

Türkçe: Bu örnekte, mutlak ve relative paths her ikisi de aynı mutlak dosyaya çözülür, çünkü symbolic link file system içindeki gerçek bir dosyaya işaret eder. Mevcut çalışma dizinine Path nesnesi olarak erişmek için toRealPath() yöntemini de kullanabiliriz.

```
System.out.println(Paths.get(".").toRealPath());
```

Reviewing NIO.2 Path APIs

English: We've covered a lot of instance methods on Path in this section. TABLE 14.6 lists them for review.

Türkçe: Bu bölümde Path üzerinde birçok örnek yöntemi ele aldık. TABLE 14.6 gözden geçirmek için onları listeler.

English: TABLE 14.6 — Path APIs

Description	Method
File path as string	<code>public String toString()</code>
Single segment	<code>public Path getName(int index)</code>
Number of segments	<code>public int getNameCount()</code>
Segments in range	<code>public Path subpath(int beginIndex, int endIndex)</code>
Final segment	<code>public Path getFileName()</code>
Immediate parent	<code>public Path getParent()</code>
Top-level segment	<code>public Path getRoot()</code>
Concatenate paths	<code>public Path resolve(String p), public Path resolve(Path p)</code>

Description	Method
Construct path to the provided path	<code>public Path relativize(Path p)</code>
Remove redundant path parts	<code>public Path normalize()</code>
Follow symbolic links and find real path	<code>public Path toRealPath()</code>

Türkçe: TABLO 14.6 — Path API'leri

Açıklama	Method
Path'i string olarak verir	<code>toString()</code>
Tek segment'i alır	<code>getName(int index)</code>
Segment sayısını alır	<code>getNameCount()</code>
Belirli aralıktaki segment'leri alır	<code>subpath(int beginIndex, int endIndex)</code>
Son segment'i alır	<code>getFileName()</code>
Immediate parent'ı alır	<code>getParent()</code>
Root'u alır	<code>getRoot()</code>
Path'leri birleştirir	<code>resolve(String/Path)</code>
Verilen path'e relative path kurar	<code>relativize(Path)</code>
Redundant parçaları kaldırır	<code>normalize()</code>
Symbolic link'leri izleyerek real path'i bulur	<code>toRealPath()</code>

Creating, Moving, and Deleting Files and Directories

English: Since creating, moving, and deleting have some nuance, we flesh them out in this section.

Türkçe: Yaratma, hareket etme ve silme bazı nüanslara sahip olduğundan, onları bu bölümde detaylandırıyoruz.

Making Directories

English: To create a directory, we use these Files methods:

Türkçe: Bir dizin oluşturmak için şu Files yöntemlerini kullanınız:

```
public static Path createDirectory(Path dir,
FileAttribute<?>... attrs) throws IOException
public static Path createDirectories(Path dir,
FileAttribute<?>... attrs) throws IOException
```

English: The `createDirectory()` method will create a directory and throw an exception if it already exists or if the paths leading up to the directory do not exist. The `createDirectories()` method creates the target directory along with any nonexistent parent directories leading up to the path. If all of the directories already exist, `createDirectories()` will simply complete without doing anything. This is useful in situations where you want to ensure a directory exists and create it if it does not.

Türkçe: `createDirectory()` bir directory oluşturur; hedef zaten varsa ya da hedefe giden parent path'ler yoksa exception fırlatır. `createDirectories()` ise hedef directory ile birlikte path üzerindeki eksik parent directory'leri de oluşturur. Bütün directory'ler zaten varsa hiçbir şey yapmadan tamamlanır. Böylece bir directory'nin varlığını güvenceye alabilir, yoksa oluşturabilirsiniz.

English: Both of these methods also accept an optional list of `FileAttribute<?>` values to apply to the newly created directory or directories. We discuss file attributes toward the end of the chapter.

Türkçe: Her iki method da yeni oluşturulan directory ya da directory'lere uygulanmak üzere isteğe bağlı bir `FileAttribute<?>` değer listesi kabul eder. File attribute'ları bölümün sonlarına doğru ele alıyoruz.

English: The following shows how to create directories:

Türkçe: Aşağıdakiler dizinlerin nasıl oluşturulacağını gösterir:

```
Files.createDirectory(Path.of("/bison/field"));
Files.createDirectories(Path.of("/bison/field/pasture/green"));
```

English: The first example creates a new directory, `field`, in the directory `/bison`, assuming `/bison` exists; otherwise, an exception is thrown. Contrast this with the second example, which creates the directory `green` along with any of the following parent directories if they do not already exist, including `bison`, `field`, and `pasture`.

Türkçe: İlk örnek, `/bison` varsa içinde `field` adında yeni directory oluşturur; yoksa exception fırlatır. İkinci örnek ise `green` directory'sini oluştururken bulunmayan parent directory'leri (`bison`, `field`, `pasture`) de oluşturur.

Copying Files

English: The `Files` class provides a method for copying files and directories within the file system.

Türkçe: `Files` class'ı, file system içindeki file ve directory'leri kopyalamak için bir method sunar.

```
public static Path copy(Path source, Path target,
    CopyOption... options) throws IOException
```

English: The method copies a file or directory from one location to another using `Path` objects. The following shows an example of copying a file and a directory:

Türkçe: Bu method, `Path` object'lerini kullanarak bir file veya directory'yi bir konumdan diğerine kopyalar. Aşağıda bir file ve bir directory kopyalama örneği vardır:

```
Files.copy(Paths.get("/panda/bamboo.txt"),
    Paths.get("/panda-save/bamboo.txt"));
Files.copy(Paths.get("/turtle"), Paths.get("/turtleCopy"));
```

English: When directories are copied, the copy is shallow. A shallow copy means that the files and subdirectories within the directory are not copied. A deep copy means that the entire tree is copied, including all of its content and subdirectories. A deep copy typically requires recursion, where a method calls itself.

Türkçe: Directory kopyalandığında varsayılan işlem shallow copy'dir (sığ kopya); içindeki file'lar ve subdirectory'ler kopyalanmaz. Deep copy (derin kopya) ise bütün içeriği ve subdirectory'leriyle birlikte ağacın tamamını kopyalar. Deep copy için genellikle bir method'un kendisini çağırdığı recursion gerekir.

```
public void copyPath(Path source, Path target) {
    try {
        Files.copy(source, target);
        if(Files.isDirectory(source))
            try (Stream<Path> s = Files.list(source)) {
                s.forEach(p -> copyPath(p,
                    target.resolve(p.getFileName())));
            }
    }
```

```
} catch(IOException e) {
    // Handle exception
}
}
```

English: The method first copies the path, whether a file or a directory. If it is a directory, only a shallow copy is performed. Next, it checks whether the path is a directory and, if it is, performs a recursive copy of each of its elements. What if the method comes across a symbolic link? Don't worry: the JVM will not follow symbolic links when using the list() method.

Türkçe: Method önce path'i, file veya directory olmasına bakmadan kopyalar. Hedef bir directory ise bu ilk işlem yalnızca shallow copy'dir. Ardından source'un directory olup olmadığını denetler; öyleyse her element'i recursive olarak kopyalar. Peki symbolic link ile karşılaşır mı? Kaynak metne göre JVM, list() kullanılırken symbolic link'leri takip etmez.

Java 17 editör notu: Files.list() link'in bulunduğu directory'nin doğrudan entry'lerini listeler; fakat örnekteki default Files.isDirectory(source) bir directory symbolic link'in target'ını izleyebilir. Bu recursive kod için explicit NOFOLLOW_LINKS/link policy olmadan "hiçbir symbolic link izlenmez" genellemesi güvenli değildir.

English: Copying and Replacing Files By default, if the target already exists, the copy() method will throw an exception. You can change this behavior by providing the StandardCopyOption enum value REPLACE_EXISTING to the method. The following method call will overwrite the movie.txt file if it already exists:

Türkçe: File Kopyalama ve Değiştirme — Hedef zaten varsa copy() varsayılan olarak exception fırlatır. Method'a StandardCopyOption.REPLACE_EXISTING enum değeri verilerek bu davranış değiştirilebilir. Aşağıdaki method call, movie.txt varsa üzerine yazar:

```
Files.copy(Paths.get("book.txt"), Paths.get("movie.txt"),
    StandardCopyOption.REPLACE_EXISTING);
```

English: For the exam, you need to know that without the `REPLACE_EXISTING` option, this method will throw an exception if the file already exists.

Türkçe: Sınav için, `REPLACE_EXISTING` option'ı verilmezse hedef file zaten mevcut olduğunda method'un exception fırlatacağını bilmelisiniz.

English: Copying Files with I/O Streams The `Files` class includes two `copy()` methods that operate with I/O streams.

Türkçe: I/O Stream'leriyle File Kopyalama — `Files` class'ı, I/O stream'leriyle çalışan iki `copy()` method'u içerir.

```
public static long copy(InputStream in, Path target,
CopyOption... options) throws IOException
public static long copy(Path source, OutputStream out)
throws IOException
```

English: The first method reads the contents of an I/O stream and writes the output to a file. The second method reads the contents of a file and writes the output to an I/O stream. These methods are quite convenient if you need to quickly read/write data from/to disk.

Türkçe: İlk method bir I/O stream'in content'ini okuyup output'u file'a yazar. İkinci method file content'ini okuyup output'u bir I/O stream'e yazar. Disk'ten hızlı data okumak veya disk'e hızlı data yazmak gerektiğinde bu method'lar kullanışlıdır.

English: The following are examples of each `copy()` method:

Türkçe: Aşağıda iki `copy()` method'una da birer örnek verilmiştir:

```
try (var is = new FileInputStream("source-data.txt")) {
// Write I/O stream data to a file
Files.copy(is, Paths.get("/mammals/wolf.txt"));
}
Files.copy(Paths.get("/fish/clown.xml"), System.out);
```

English: While we used `FileInputStream` in the first example, the I/O stream could have been any valid I/O stream including website connections, in-memory stream resources, and so forth. The second example prints the contents of a file directly to the `System.out` stream.

Türkçe: İlk örnekte `FileInputStream` kullansak da burada web connection'ı veya in-memory stream resource'u gibi herhangi bir geçerli I/O stream bulunabilirdi. İkinci örnek bir file'ın içeriğini doğrudan `System.out` stream'ine yazdırır.

English: Copying Files into a Directory For the exam, it is important that you understand how the `copy()` method operates on both files and directories. For example, let's say we have a file, `food.txt`, and a directory, `/enclosure`. Both the file and directory exist. What do you think is the result of executing the following process?

Türkçe: File'i Directory'ye Kopyalama — Sınav için `copy()` method'unun file ve directory üzerinde nasıl çalıştığını bilmek önemlidir. `food.txt` file'ının ve `/enclosure` directory'sinin var olduğunu varsayalım. Aşağıdaki operation'ın sonucu nedir?

```
var file = Paths.get("food.txt");
var directory = Paths.get("/enclosure");
Files.copy(file, directory);
```

English: If you said it would create a new file at `/enclosure/food.txt`, you're way off. It throws an exception. The command tries to create a new file named `/enclosure`. Since the path `/enclosure` already exists, an exception is thrown at runtime.

Türkçe: Operation `/enclosure/food.txt` oluşturmaz; exception fırlatır. Command, `/enclosure` adında yeni file oluşturmaya çalışır. Bu path zaten var olduğundan çalışma zamanında exception oluşur.

English: On the other hand, if the directory did not exist, the process would create a new file with the contents of `food.txt`, but the file would be called `/enclosure`. Remember, we said files may not need to have extensions, and in this example, it matters.

Türkçe: Directory var olmasaydı operation `food.txt` content'ine sahip, fakat adı `/enclosure` olan yeni file oluştururdu. File adlarının extension taşıması zorunlu değildir; bu örnekte bu ayrıntı önemlidir.

English: This behavior applies to both the `copy()` and `move()` methods, the latter of which we cover next. In case you're curious, the correct way to copy the file into the directory is to do the following:

Türkçe: Bu davranış hem `copy()` hem de birazdan ele alacağımız `move()` için geçerlidir. File'ı directory içine kopyalamanın doğru yolu hedef path'te filename'i de belirtmektir:

```
var file = Paths.get("food.txt");
var directory = Paths.get("/enclosure/food.txt");
Files.copy(file, directory);
```

Moving or Renaming Paths with `move()`

English: The `Files` class provides a useful method for moving or renaming files and directories.

Türkçe: `Files` class'ı file ve directory'leri taşımak ya da yeniden adlandırmak için kullanışlı bir method sunar.

```
public static Path move(Path source, Path target,
    CopyOption... options) throws IOException
```

English: The following sample code uses the `move()` method:

Türkçe: Aşağıdaki örnek kod `move()` method'unu kullanır:

```
Files.move(Path.of("C:\\zoo"), Path.of("C:\\zoo-new"));
Files.move(Path.of("C:\\user\\addresses.txt"),
    Path.of("C:\\zoo-new\\addresses2.txt"));
```

English: The first example renames the `zoo` directory to a `zoo-new` directory, keeping all of the original contents from the source directory. The second example moves the `addresses.txt` file from the directory `user` to the directory `zoo-new` and renames it `addresses2.txt`.

Türkçe: İlk örnek `zoo` directory'sini `zoo-new` olarak yeniden adlandırır ve source directory'nin bütün içeriğini korur. İkinci örnek `addresses.txt` file'ını `user` directory'sinden `zoo-new` directory'sine taşır ve adını `addresses2.txt` olarak değiştirir.

English: Similarities between move() and copy() Like copy(), move() requires REPLACE_EXISTING to overwrite the target if it exists; otherwise, it will throw an exception. Also like copy(), move() will not put a file in a directory if the source is a file and the target is a directory. Instead, it will create a new file with the name of the directory.

Türkçe: move() ve copy() Arasındaki Benzerlikler — copy() gibi move() da mevcut hedefin üzerine yazmak için REPLACE_EXISTING gerektirir; aksi hâlde exception fırlatır. Source bir file, target ise bir directory olduğunda move() file'ı directory'nin içine yerleştirmez; bunun yerine directory'nin adıyla yeni bir file oluşturmaya çalışır.

English: Performing an Atomic Move Another enum value that you need to know for the exam when working with the move() method is the StandardCopyOption value ATOMIC_MOVE.

Türkçe: Atomic Move Gerçekleştirme — move() ile çalışırken sınav için bilmeniz gereken bir başka enum değeri StandardCopyOption.ATOMIC_MOVE'dur.

```
Files.move(Path.of("mouse.txt"), Path.of("gerbil.txt"),
StandardCopyOption.ATOMIC_MOVE);
```

English: You may remember the atomic property from Chapter 13, “Concurrency,” and the principle of an atomic move is similar. An atomic move is one in which a file is moved within the file system as a single indivisible operation. Put another way, any process monitoring the file system never sees an incomplete or partially written file. If the file system does not support this feature, an AtomicMoveNotSupportedException will be thrown.

Türkçe: Atomic özelliğini Bölüm 13, “Concurrency” konusundan hatırlayabilirsiniz; atomic move ilkesi de benzerdir. Atomic move, bir file'ın file system içinde tek ve bölünmez bir operation olarak taşınmasıdır. Başka bir deyişle, file system'i izleyen hiçbir process eksik veya kısmen yazılmış bir file görmez. File system bu özelliği desteklemiyorsa AtomicMoveNotSupportedException fırlatılır.

English: Note that while ATOMIC_MOVE is available as a member of the StandardCopyOption type, it will likely throw an exception if passed to a copy() method.

Türkçe: ATOMIC_MOVE, StandardCopyOption type'ının bir member'ı olsa da copy() method'una verilirse büyük olasılıkla exception fırlatılır.

Deleting a File with delete() and deleteIfExists()

English: The Files class includes two methods that delete a file or empty directory within the file system.

Türkçe: Files class'ı, file system içindeki bir file'ı ya da boş directory'yi silen iki method içerir.

```
public static void delete(Path path) throws IOException
public static boolean deleteIfExists(Path path) throws IOException
```

English: To delete a directory, it must be empty. Both of these methods throw an exception if operated on a nonempty directory. In addition, if the path is a symbolic link, the symbolic link will be deleted, not the path that the symbolic link points to.

Türkçe: Bir directory'nin silinebilmesi için boş olması gerekir; iki method da nonempty directory üzerinde çağrılırsa exception fırlatır. Path bir symbolic link ise link'in gösterdiği target değil, symbolic link'in kendisi silinir.

English: The methods differ on how they handle a path that does not exist. The `delete()` method throws an exception if the path does not exist, while the `deleteIfExists()` method returns `true` if the delete was successful or `false` otherwise. Similar to `createDirectories()`, `deleteIfExists()` is useful in situations where you want to ensure that a path does not exist and delete it if it does.

Türkçe: Method'lar var olmayan path'i ele alış biçimleri bakımından farklıdır. `delete()`, path yoksa exception fırlatır. `deleteIfExists()` ise silme gerçekleşirse `true`, aksi hâlde `false` döndürür. `createDirectories()` a benzer biçimde `deleteIfExists()`, bir path'in bulunmadığından emin olmak ve varsa onu silmek istediğiniz durumlarda kullanışlıdır.

English: Here we provide sample code that performs `delete()` operations:

Türkçe: Aşağıda delete operation'ları gerçekleştiren örnek kod vardır:

```
Files.delete(Paths.get("/vulture/feathers.txt"));
Files.deleteIfExists(Paths.get("/pigeon"));
```

English: The first example deletes the `feathers.txt` file in the vulture directory, and it throws a `NoSuchFileException` if the file or directory does not exist. The second example deletes the pigeon directory, assuming it is empty. If the pigeon directory does not exist, the second line will not throw an exception.

Türkçe: İlk örnek `/vulture/feathers.txt` file'ını siler; file veya directory yoksa `NoSuchFileException` fırlatır. İkinci örnek `/pigeon` directory'sinin boş olduğunu varsayarak onu siler. `/pigeon` yoksa ikinci satır exception fırlatmaz.

Comparing Files with `isSameFile()` and `mismatch()`

English: Since a path may include path symbols and symbolic links within a file system, the `equals()` method can't be relied on to know if two `Path` instances refer to the same file. Luckily, there is the `isSameFile()` method. This method takes two `Path` objects as input,

Türkçe: Bir path, file system içinde path symbol ve symbolic link içerebildiğinden iki `Path` instance'ının aynı file'ı gösterip göstermediğini belirlemek için yalnızca `equals()` a güvenemeyiz. Bunun için `isSameFile()` vardır. Bu method iki `Path` object'ini input olarak alır,

English: resolves all path symbols, and follows symbolic links. Despite the name, the method can also be used to determine whether two `Path` objects refer to the same directory.

Türkçe: bütün path symbol'larını çözer ve symbolic link'leri takip eder. Adına rağmen method, iki `Path` object'inin aynı directory'yi gösterip göstermediğini belirlemek için de kullanılabilir.

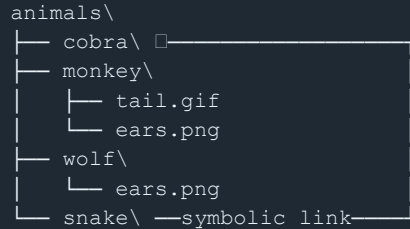
English: While most uses of `isSameFile()` will trigger an exception if the paths do not exist, there is a special case in which it does not. If the two path objects are equal in terms of `equals()`, the method will just return `true` without checking whether the file exists.

Türkçe: `isSameFile()` çoğu kullanımda path'ler yoksa exception fırlatır; fakat bir istisnası vardır. İki `Path` object'i `equals()` bakımından eşitse method, file'ın var olup olmadığını denetlemeden `true` döndürür.

English: Assume that the file system exists, as shown in FIGURE 14.4, with a symbolic link from `/animals/snake` to `/animals/cobra`.

Türkçe: FIGURE 14.4'teki file system'in var olduğunu ve `/animals/snake` path'inin `/animals/cobra`ya symbolic link olduğunu varsayın.

English: FIGURE 14.4 — Comparing file uniqueness



Türkçe: ŞEKİL 14.4 — File uniqueness karşılaştırması. `/animals/snake`, `/animals/cobra` directory'sine symbolic link'tir. `monkey/ears.png` ile `wolf/ears.png` aynı filename'i taşısa da farklı file'lardır.

English: Given the structure defined in FIGURE 14.4, what does the following output?

Türkçe: FIGURE 14.4'teki yapıya göre aşağıdaki kod ne yazdırır?

```

System.out.println(Files.isSameFile(
    Path.of("/animals/cobra"),
    Path.of("/animals/snake")));
System.out.println(Files.isSameFile(
    Path.of("/animals/monkey/ears.png"),
    Path.of("/animals/wolf/ears.png")));
  
```

English: Since `snake` is a symbolic link to `cobra`, the first example outputs `true`. In the second example, the paths refer to different files, so `false` is printed.

Türkçe: `snake`, `cobra`ya symbolic link olduğu için ilk örnek `true` yazdırır. İkinci örnekte path'ler farklı file'ları gösterdiğinden `false` yazdırılır.

English: Sometimes you want to compare the contents of the file rather than whether it is physically the same file. For example, we could have two files with text `hello`. The `mismatch()` method was introduced in Java 12 to help us out here. It takes two `Path` objects as input. The method returns `-1` if the files are the same; otherwise, it returns the index of the first position in the file that differs.

Türkçe: Bazen file'ların fiziksel olarak aynı olup olmadığını değil içeriklerini karşılaştırmak isteriz. Örneğin iki ayrı file'ın ikisi de `hello` metnini içerebilir. Java 12'de eklenen `mismatch()` bunun için kullanılır ve input olarak iki `Path` object'i alır. File içerikleri aynıysa `-1`; farklıysa ilk farklı byte konumunun index'ini döndürür.

```

System.out.println(Files.mismatch(
    Path.of("/animals/monkey.txt"),
    Path.of("/animals/wolf.txt")));
  
```

English: Suppose `monkey.txt` contains the name Harold and `wolf.txt` contains the name Howler. The previous code prints 1 in that case because the second position is different, and we use zero-based indexing in Java. Given those values, what do you think this code prints?

Türkçe: `monkey.txt`in Harold, `wolf.txt`in Howler metnini içerdiğini varsayalım. İkinci konum farklı ve Java zero-based indexing kullandığı için önceki kod 1 yazdırır. Aynı değerlerle aşağıdaki kod sizce ne yazdırır?

```
System.out.println(Files.mismatch(  
    Path.of("/animals/wolf.txt"),  
    Path.of("/animals/monkey.txt")));
```

English: The answer is the same as the previous example. The code prints 1 again. The `mismatch()` method is symmetric and returns the same result regardless of the order of the parameters.

Türkçe: Cevap önceki örnekle aynıdır: Kod yine 1 yazdırır. `mismatch()` symmetric'tir; parameter sırası değişse de aynı sonucu döndürür.

Introducing I/O Streams

English: Now that we have the basics out of the way, let's move on to I/O streams, which are far more interesting. In this section, we show you how to use I/O streams to read and write data. The "I/O" refers to the nature of how data is accessed, either by reading the data from a resource (input) or by writing the data to a resource (output).

Türkçe: Artık temelleri ortadan kaldırdığımıza göre, çok daha ilginç olan I/O streams kısmına geçelim. Bu bölümde, verileri okumak ve yazmak için I/O streams nasıl kullanılacağını gösteriyoruz. "I/O", bir kaynaktan (giriş) verileri okuyarak veya verileri bir kaynağa (çıkış) yazarak verilere nasıl erişildiğini ifade eder.

English: When we refer to I/O streams in this chapter, we are referring to the ones found in the `java.io` API. If we just say streams, it means the ones from Chapter 10. We agree that the naming can be a bit confusing!

Türkçe: Bu bölümde I/O streams'e değindiğimizde, `java.io` API 'de bulunanları kastediyoruz. Sadece streams dersek, Bölüm 10'dan olanlar anlamına gelir. İsimlendirmenin biraz kafa karıştırıcı olabileceği konusunda hemfikiriz!

Understanding I/O Stream Fundamentals

English: The contents of a file may be accessed or written via an I/O stream, which is a list of data elements presented sequentially. An I/O stream can be conceptually thought of as a long, nearly never-ending stream of water with data presented one wave at a time.

Türkçe: Bir dosyanın içeriğine, sıralı olarak sunulan veri öğelerinin bir listesi olan I/O stream üzerinden erişilebilir veya yazılabilir. Bir I/O stream kavramsal olarak bir long, neredeyse hiç bitmeyen stream su olarak düşünülebilir ve veriler her seferinde bir dalga sunulur.

English: We demonstrate this principle in FIGURE 14.5. The I/O stream is so large that once we start reading it, we have no idea where the beginning or the end is. We just have a pointer to our current position in the I/O stream and read data one block at a time.

Türkçe: Bu ilkeyi FIGURE 14.5'te gösteriyoruz. I/O stream o kadar büyüktür ki, bir kez okumaya başladığımızda, başlangıcın veya sonun nerede olduğu hakkında hiçbir fikrimiz yoktur. Sadece I/O stream içindeki mevcut konumumuzu gösteren bir işaretçimiz var ve verileri tek seferde bir blok halinde okuyoruz.

English: FIGURE 14.5 — Visual representation of an I/O stream

Türkçe: ŞEKİL 14.5 — I/O stream'in görsel temsili. Pointer current position'ı gösterir; data stream head'den tail'e doğru block'lar hâlinde okunur.

English: Each type of I/O stream segments data into a wave or block in a particular way. For example, some I/O stream classes read or write data as individual bytes. Other I/O stream classes read or write individual characters or strings of characters. On top of that, some I/O stream classes read or write larger groups of bytes or characters at a time, specifically those with the word `Buffered` in their name.

Türkçe: Her I/O stream type'ı data'yı kendine özgü biçimde parçalara veya block'lara ayırır. Bazı I/O stream class'ları tek tek byte; bazıları tek tek character veya character sequence okur/yazar. Adında **Buffered** bulunanlar ise bir defada daha büyük byte veya character grupları işleyebilir.

English: Although the java.io API is full of I/O streams that handle characters, strings, groups of bytes, and so on, nearly all are built on top of reading or writing an individual byte or an array of bytes at a time. Higher-level I/O streams exist for convenience as well as performance.

Türkçe: java.io API karakterleri, dizeleri, bytes gruplarını ve benzerlerini işleyen I/O streams ile dolu olsa da, neredeyse hepsi bir seferde bir byte veya bir bytes dizisi okumak veya yazmak üzerine inşa edilmiştir. Daha yüksek seviye I/O streams kolaylık ve performans için mevcuttur.

English: Although I/O streams are commonly used with file I/O, they are more generally used to handle the reading/writing of any sequential data source. For example, you might construct a Java application that submits data to a website using an output stream and reads the result via an input stream.

Türkçe: I/O streams genellikle I/O dosyasıyla birlikte kullanılsa da, daha genel olarak herhangi bir ardışık veri kaynağının reading/writing dosyasını işlemek için kullanılırlar. Örneğin, bir output stream kullanarak bir web sitesine veri gönderen ve sonucu bir input stream aracılığıyla okuyan bir Java uygulaması oluşturabilirsiniz.

English: I/O Streams Can Be Big When writing code where you don't know what the I/O stream size will be at runtime, it may be helpful to visualize an I/O stream as being so large that all of the data contained in it could not possibly fit into memory. For example, a 1 TB file could not be stored entirely in memory by most computer systems(at the time this book is being written). The file can still be read and written by a program with very little memory, since the I/O stream allows the application to focus on only a small portion of the overall I/O stream at any given time.

Türkçe: I/O Stream'leri Büyük Olabilir — Runtime'da I/O stream size'ı bilinmiyorsa stream'i, bütün data'sı memory'ye sığmayacak kadar büyük düşünmek yararlıdır. Örneğin kitap yazılırken çoğu computer system, 1 TB'lık file'ı bütünüyle memory'de tutamazdı. Yine de I/O stream application'ın her anda yalnız küçük bir bölüme odaklanmasını sağladığı için file, az memory kullanan bir programla okunup yazılabilir.

Learning I/O Stream Nomenclature

English: The java.io API provides numerous classes for creating, accessing, and manipulating I/O streams— so many that it tends to overwhelm many new Java developers. Stay calm! We review the major differences between each I/O stream class and show you how to distinguish between them.

Türkçe: `java.io` API, I/O stream oluşturmak, bunlara erişmek ve bunları yönetmek için çok sayıda class sunar; bu çeşitlilik yeni Java geliştiricilerine ilk bakışta bunaltıcı gelebilir. Endişelenmeyin: I/O stream class'ları arasındaki temel farkları inceleyip onları nasıl ayırt edeceğinizi göstereceğiz.

English: Even if you come across a particular I/O stream on the exam that you do not recognize, the name of the I/O stream often gives you enough information to understand exactly what it does.

Türkçe: Tanımadığınız sınavda belirli bir I/O stream ile karşılaştığınız bile, I/O stream adı size ne yaptığını tam olarak anlamak için yeterli bilgi verir.

English: The goal of this section is to familiarize you with common terminology and naming conventions used with I/O streams. Don't worry if you don't recognize the particular stream class names used in this section or their function; we cover how to use them in detail in this chapter.

Türkçe: Bu bölümün amacı, I/O streams ile kullanılan ortak terminoloji ve adlandırma sözleşmelerini size tanıtmaktır. Bu bölümde veya işlevlerinde kullanılan belirli stream sınıf adlarını tanıımıyorsanız endişelenmeyin; bu bölümde bunları nasıl ayrıntılı olarak kullanacağımızı ele alıyoruz.

Storing Data as Bytes

English: Data is stored in a file system (and memory) as a 0 or 1, called a bit. Since it's really hard for humans to read/write data that is just 0 s and 1 s, they are grouped into a set of 8 bits, called a byte.

Türkçe: Veriler file system (ve bellek) olarak 0 veya 1 olarak saklanır. İnsanların sadece 0 s ve 1 s olan read/write verilerini byte olarak adlandırılan 8 bitlik bir kümeye gruplandırmaları gerçekten zor olduğu için.

English: What about the Java byte primitive type? As you learn later, when we use I/O streams, values are often read or written using byte values and arrays.

Türkçe: Java byte ilkel tipi hakkında ne düşünüyorsunuz? Daha sonra öğrendiğiniz gibi, I/O streams kullandığımızda, değerler genellikle byte değerleri ve dizileri kullanılarak okunur veya yazılır.

Byte Streams vs. Character Streams

English: The java.io API defines two sets of I/O stream classes for reading and writing I/O streams: byte I/O streams and character I/O streams. We use both types of I/O streams throughout this chapter. Differences between Byte and Character I/O Streams Byte I/O streams read/write binary data (0 s and 1 s) and have class names that end in InputStream or OutputStream.

Türkçe: java.io API, I/O stream okumak ve yazmak için iki class grubu tanımlar: byte I/O stream'leri ve character I/O stream'leri. Bölüm boyunca iki türü de kullanacağız. Byte ve Character I/O Stream'leri Arasındaki Farklar — Byte I/O stream'leri binary data'yı (0 ve 1ler) okur/yazar; class adları InputStream veya OutputStream ile biter.

English: Character I/O streams read/write text data and have class names that end in Reader or Writer.

Türkçe: Karakter I/O streams read/write metin verisi ve Reader veya Writer ile biten sınıf adlarına sahiptir.

English: The API frequently includes similar classes for both byte and character I/O streams, such as FileInputStream and FileReader. The difference between the two classes is based on how the bytes are read or written.

Türkçe: API'de byte ve character I/O stream'leri için FileInputStream ile FileReader gibi benzer class'lar bulunur. İki class arasındaki fark byte'ların nasıl okunup yazıldığıdır.

English: It is important to remember that even though character I/O streams do not contain the word Stream in their class name, they are still I/O streams. The use of Reader /Writer in the name is just to distinguish them from byte streams.

Türkçe: Character I/O stream class'larının adında Stream geçmese de bunlar hâlâ I/O stream'idir. Addaki Reader/Writer ifadeleri yalnız bunları byte stream'lerinden ayırır.

English: Throughout the chapter, we refer to both InputStream and Reader as input streams, and we refer to both OutputStream and Writer as output streams.

Türkçe: Bölüm boyunca InputStream ile Reader input stream; OutputStream ile Writer output stream olarak adlandırılır.

English: The byte I/O streams are primarily used to work with binary data, such as an image or executable file, while character I/O streams are used to work with text files. For example, you can use a Writer class to output a String value to a file without necessarily having to worry about the underlying character encoding of the file.

Türkçe: Byte I/O stream'leri öncelikle image veya executable file gibi binary data'yla, character I/O stream'leri ise text file'larla çalışır. Örneğin alttaki character encoding ayrıntısıyla doğrudan uğraşmadan bir String değerini file'a yazmak için bir Writer class'ı kullanabilirsiniz.

English: The character encoding determines how characters are encoded and stored in bytes in an I/O stream and later read back or decoded as characters. Although this may sound simple, Java supports a wide variety of character encodings, ranging from ones that may use one byte for Latin characters, UTF-8 and ASCII for example, to using two or more bytes per character, such as UTF-16. For the exam, you don't need to memorize the character encodings, but you should be familiar with the names.

Türkçe: Karakter kodlaması, karakterlerin bytes içinde bir I/O stream içinde nasıl kodlandığını ve saklandığını belirler ve daha sonra karakter olarak geri okunur veya deşifre edilir. Bu basit gibi görünse de, Java, Latin karakterleri için bir byte, UTF-8 ve ASCII, örneğin UTF-16 gibi karakter başına iki veya daha fazla bytes kullanmaya kadar değişen çok çeşitli karakter kodlamalarını destekler. Sınav için karakter kodlamalarını ezberlemenize gerek yoktur, ancak isimlere aşina olmalısınız.

English: Character Encoding in Java In Java, the character encoding can be specified using the Charset class by passing a name value to the static Charset.forName() method, such as in the following examples:

Türkçe: Java'da Karakter Kodlaması Java'da karakter kodlaması, aşağıdaki örneklerde olduğu gibi, bir isim değerini statik Charset.forName() yöntemine geçirerek Charset sınıfı kullanılarak belirtilebilir:

```
Charset usAsciiCharset = Charset.forName("US-ASCII");  
Charset utf8Charset = Charset.forName("UTF-8");  
Charset utf16Charset = Charset.forName("UTF-16");
```

English: Java supports numerous character encodings, each specified by a different standard name value.

Türkçe: Java, her biri farklı bir standart ad değeri ile belirtilen çok sayıda karakter kodlamasını destekler.

Input vs. Output Streams

English: Most InputStream classes have a corresponding OutputStream class, and vice versa. For example, the FileOutputStream class writes data that can be read by a FileInputStream. If you understand the features of a particular Input or Output stream class, you should naturally know what its complementary class does.

Türkçe: Çoğu `InputStream` class'ının karşılık gelen bir `OutputStream` class'ı vardır; tersi de geçerlidir. Örneğin `FileOutputStream`, `FileInputStream` tarafından okunabilen data'yı yazar. Belirli bir input veya output stream class'ının özelliklerini bilerseniz tamamlayıcı class'ın ne yaptığını da çıkarabilirsiniz.

English: It follows, then, that most Reader classes have a corresponding Writer class. For example, the FileWriter class writes data that can be read by a FileReader.

Türkçe: Benzer biçimde çoğu `Reader` class'ının karşılık gelen bir `Writer` class'ı vardır. Örneğin `FileWriter`, `FileReader` tarafından okunabilen data'yı yazar.

English: There are exceptions to this rule. For the exam, you should know that PrintWriter has no accompanying PrintReader class. Likewise, the PrintStream is an OutputStream that has no corresponding InputStream class. It also does not have Output in its name. We discuss these classes later in this chapter.

Türkçe: Bu kuralın istisnaları vardır. Sınav için, `PrintWriter` ile birlikte `PrintReader` sınıfının olmadığını bilmelisiniz. Aynı şekilde, `PrintStream` karşılık gelen `InputStream` sınıfı olmayan bir `OutputStream`'dir. Ayrıca kendi adına `Output` yoktur. Bu dersleri daha sonra bu bölümde tartışacağız.

Low-Level vs. High-Level Streams

English: Another way that you can familiarize yourself with the java.io API is by segmenting I/O streams into low-level and high-level streams.

Türkçe: Kendinizi java.io API ile tanıştırmamanın bir başka yolu da I/O streams alt seviye ve high-level streams olarak bölümlere ayırmaktır.

English: A low-level stream connects directly with the source of the data, such as a file, an array, or a String. Low-level I/O streams process the raw data or resource and are accessed in a direct and unfiltered manner. For example, a `FileInputStream` is a class that reads file data one byte at a time.

Türkçe: Low-level stream, file, array veya `String` gibi veri kaynağına doğrudan bağlanır. Ham data/resource üzerinde doğrudan ve filtre uygulanmadan çalışır. Örneğin `FileInputStream`, file data'sını her seferinde bir byte okuyabilen bir class'tır.

English: Alternatively, a high-level stream is built on top of another I/O stream using wrapping. Wrapping is the process by which an instance is passed to the constructor of another class, and operations on the resulting instance are filtered and applied to the original instance. For example, take a look at the `FileReader` and `BufferedReader` objects in the following sample code:

Türkçe: Alternatif olarak, bir high-level stream, sarma kullanarak başka bir I/O stream üzerine inşa edilir. Sarma işlemi, bir örneğin başka bir sınıfın yapısına aktarıldığı ve sonuçta ortaya çıkan örnekteki işlemlerin filtrelendiği ve orijinal örneğe uygulandığı süreçtir. Örneğin, aşağıdaki örnek koddaki `FileReader` ve `BufferedReader` nesnelerine bir göz atın:

```
try (var br = new BufferedReader(new FileReader("zoo-data.txt"))) {  
    System.out.println(br.readLine());  
}
```

English: In this example, `FileReader` is the low-level I/O stream, whereas `BufferedReader` is the high-level I/O stream that takes a `FileReader` as input. Many operations on the high-level I/O stream pass through as operations to the underlying low-level I/O stream, such as `read()` or `close()`. Other operations override or add new functionality to the low-level I/O stream methods. The high-level I/O stream may add new methods, such as `readLine()`, as well as performance enhancements for reading and filtering the low-level data.

Türkçe: Bu örnekte, `FileReader` düşük seviyeli I/O stream iken, `BufferedReader` giriş olarak `FileReader` alan high-level bir I/O stream'dir. High-level I/O stream üzerindeki birçok işlem, `read()` veya `close()` gibi altta yatan düşük seviye I/O stream işlemleri olarak geçer. Diğer işlemler, düşük seviyeli I/O stream yöntemlerine yeni işlevler geçersiz kılar veya ekler. Üst düzey I/O stream, düşük seviyeli verileri okumak ve filtrelemek için `readLine()` gibi yeni yöntemlerin yanı sıra performans geliştirmeleri de ekleyebilir.

English: High-level I/O streams can also take other high-level I/O streams as input. For example, although the following code might seem a little odd at first, the style of wrapping an I/O stream is quite common in practice:

Türkçe: Üst düzey I/O streams diğer üst düzey I/O streams girişini de alabilir. Örneğin, ilk başta aşağıdaki kod biraz garip görünse de, bir I/O stream sarma stili pratikte oldukça yaygındır:

```
try (var ois = new ObjectInputStream(  
    new BufferedInputStream(  
        new FileInputStream("zoo-data.txt"))) ) {  
    System.out.print(ois.readObject());  
}
```

English: In this example, the low-level `FileInputStream` interacts directly with the file, which is wrapped by a high-level `BufferedInputStream` to improve performance. Finally, the entire object is wrapped by another high-level `ObjectInputStream`, which allows us to interpret the data as a Java object.

Türkçe: Bu örnekte low-level `FileInputStream` file'la doğrudan etkileşir ve performansı artırmak için high-level `BufferedInputStream` tarafından wrap edilir. Son olarak bütün object, data'yı bir Java object'i olarak yorumlamamızı sağlayan başka bir high-level stream olan `ObjectInputStream` ile wrap edilir.

English: For the exam, the only low-level stream classes you need to be familiar with are the ones that operate on files. The rest of the nonabstract stream classes are all high-level streams.

Türkçe: Sınav için, aşına olmanız gereken tek low-level stream sınıfları dosyalarda çalışan sınıflardır. stream sınıflarının geri kalanı high-level streams şeklindedir.

Stream Base Classes

English: The `java.io` library defines four abstract classes that are the parents of all I/O stream classes defined within the API: `InputStream`, `OutputStream`, `Reader`, and `Writer`.

Türkçe: `java.io` kütüphanesi, API: `InputStream`, `OutputStream`, `Reader` ve `Writer` içinde tanımlanan tüm I/O stream sınıflarının ebeveyni olan dört abstract sınıfını tanımlar.

English: The constructors of high-level I/O streams often take a reference to the abstract class. For example, `BufferedWriter` takes a `Writer` object as input, which allows it to take any subclass of `Writer`.

Türkçe: Üst düzey I/O streams oluşturucuları genellikle abstract sınıfına referans alırlar. Örneğin, `BufferedWriter` bir `Writer` nesnesini girdi olarak alır, bu da `Writer` alt sınıfının herhangi birini almasını sağlar.

English: One common area where the exam likes to play tricks on you is mixing and matching I/O stream classes that are not compatible with each other. For example, take a look at each of the following examples and see whether you can determine why they do not compile:

Türkçe: Sınavın size oyunlar oynamayı sevdiği ortak bir alan, birbirleriyle uyumlu olmayan I/O stream sınıflarını karıştırmak ve eşleştirmektir. Örneğin, aşağıdaki örneklerin her birine bir göz atın ve neden derlemediklerini belirleyip belirleyemeyeceğinizi görün:

```
new BufferedInputStream(new FileReader("z.txt")); // DOES NOT COMPILE  
new BufferedWriter(new FileOutputStream("z.txt")); // DOES NOT COMPILE  
new ObjectInputStream(  
    new FileOutputStream("z.txt")); // DOES NOT COMPILE  
new BufferedInputStream(new InputStream()); // DOES NOT COMPILE
```

English: The first two examples do not compile because they mix Reader /Writer classes with InputStream /OutputStream classes, respectively. The third example does not compile because we are mixing an OutputStream with an InputStream. Although it is possible to read data from an InputStream and write it to an OutputStream, wrapping the I/O

Türkçe: İlk iki örnek sırasıyla Reader/Writer class'larını InputStream/OutputStream class'larıyla karıştırdığı için derlenmez. Üçüncü örnek OutputStream ile InputStreami karıştırdığı için derlenmez. InputStreamden data okuyup OutputStreame yazmak mümkün olsa da I/O stream'i bu biçimde wrap etmek

English: stream is not the way to do so. As you see later in this chapter, the data must be copied over. Finally, the last example does not compile because InputStream is an abstract class, and therefore you cannot create an instance of it.

Türkçe: stream bunu yapmanın yolu değil. Bu bölümde daha sonra da gördüğünüz gibi, verilerin kopyalanması gerekir. Son olarak, son örnek derlenmez çünkü InputStream bir abstract sınıftır ve bu nedenle bunun bir örneğini oluşturamazsınız.

Decoding I/O Class Names

English: Pay close attention to the name of the I/O class on the exam, as decoding it often gives you context clues as to what the class does. For example, without needing to look it up, it should be clear that FileReader is a class that reads data from a file as characters or strings. Furthermore, ObjectOutputStream sounds like a class that writes object data to a byte stream.

Türkçe: Sınavda I/O class'ının adına dikkat edin; adı parçalarına ayırmak çoğu zaman class'ın ne yaptığına ilişkin ipuçları verir. Örneğin FileReaderın file'dan character veya string okuduğu, ObjectOutputStreamın ise object data'sını bir byte stream'e yazdığı adına bakılarak anlaşılabilir.

English: TABLE 14.7 lists the abstract base classes that all I/O streams inherit from.

Türkçe: TABLE 14.7, tüm I/O streams miras aldığı abstract taban sınıflarını listeler.

English: TABLE 14.7 — The `java.io` abstract stream base classes

Class name	Description
<code>InputStream</code>	Abstract class for all input byte streams
<code>OutputStream</code>	Abstract class for all output byte streams
<code>Reader</code>	Abstract class for all input character streams
<code>Writer</code>	Abstract class for all output character streams

Türkçe: TABLO 14.7 — `java.io` abstract stream base class'ları. `InputStream/OutputStream` byte; `Reader/Writer` character data işler.

English: TABLE 14.8 lists the concrete I/O streams that you should be familiar with for the exam. Most information about direction, byte/character access and level can be decoded from the class name.

Türkçe: TABLE 14.8, sınav için aşina olmanız gereken beton I/O streams listeler. Yön, byte/character erişim ve seviye ile ilgili çoğu bilgi sınıf adından çözülebilir.

English: TABLE 14.8 — The `java.io` concrete I/O stream classes

Class name	Level	Description
<code>FileInputStream</code>	Low	Reads file data as bytes
<code>FileOutputStream</code>	Low	Writes file data as bytes
<code>FileReader</code>	Low	Reads file data as characters
<code>FileWriter</code>	Low	Writes file data as characters
<code>BufferedInputStream</code>	High	Reads byte data from an existing <code>InputStream</code> in a buffered manner

Türkçe: TABLO 14.8 — Concrete `java.io` stream class'ları

Class adı	Level	Açıklama
<code>FileInputStream</code>	Low	File data'yı byte olarak okur
<code>FileOutputStream</code>	Low	File data'yı byte olarak yazar
<code>FileReader</code>	Low	File data'yı character olarak okur
<code>FileWriter</code>	Low	File data'yı character olarak yazar
<code>BufferedInputStream</code>	High	Mevcut <code>InputStream</code> den buffered byte okur

English: TABLE 14.8 — Concrete I/O stream classes (continued)

Class name	Level	Description
<code>BufferedOutputStream</code>	High	Writes byte data to an existing <code>OutputStream</code> in a buffered manner
<code>BufferedReader</code>	High	Reads character data from an existing <code>Reader</code> in a buffered manner
<code>BufferedWriter</code>	High	Writes character data to an existing <code>Writer</code> in a buffered manner
<code>ObjectInputStream</code>	High	Deserializes primitive data and object graphs from an existing <code>InputStream</code>
<code>ObjectOutputStream</code>	High	Serializes primitive data and object graphs to an existing <code>OutputStream</code>
<code>PrintStream</code>	High	Writes formatted Java object representations to a binary stream
<code>PrintWriter</code>	High	Writes formatted Java object representations to a character stream

Türkçe: TABLO 14.8 — Concrete I/O stream class'ları (devam). Buffered class'lar efficiency; object stream'ler serialization; `PrintStream`/`PrintWriter` formatted output sağlar.

English: Keep TABLE 14.7 and TABLE 14.8 handy as you learn more about I/O streams in this chapter. We discuss them in more detail, including examples of each.

Türkçe: TABLE 14.7 ve TABLE 14.8'i bu bölümde I/O streams hakkında daha fazla bilgi edinirken kullanışlı tutun. Bunları her birinden örnekler de dahil olmak üzere daha ayrıntılı olarak tartışıyoruz.

Reading and Writing Files

English: There are a number of ways to read and write from a file. We show them in this section by copying one file to another.

Türkçe: Bir file'ı okuyup yazmanın çeşitli yolları vardır. Bu bölümde bunları bir file'ı diğerine kopyalayarak göstereceğiz.

Using I/O Streams

English: I/O streams are all about reading/writing data, so it shouldn't be a surprise that the most important methods are `read()` and `write()`. Both `InputStream` and `Reader` declare a `read()` method to read byte data from an I/O stream. Likewise, `OutputStream` and `Writer` both define a `write()` method to write a byte to the stream:

Türkçe: I/O streams tamamen reading/writing verileriyle ilgilidir, bu nedenle en önemli yöntemlerin `read()` ve `write()` olması sürpriz olmamalıdır. Hem `InputStream` hem de `Reader` bir I/O stream verisinden byte verilerini okumak için bir `read()` yöntemi beyan eder. Aynı şekilde, `OutputStream` ve `Writer` her ikisi de stream'ye byte yazmak için bir `write()` yöntemi tanımlar:

English: The following `copyStream()` methods show an example of reading all of the values of an `InputStream` and `Reader` and writing them to an `OutputStream` and `Writer`, respectively. In both examples, `-1` is used to indicate the end of the stream.

Türkçe: Aşağıdaki `copyStream()` yöntemleri, bir `InputStream` ve `Reader` değerlerinin tümünü okumanın ve bunları sırasıyla bir `OutputStream` ve `Writer` yazmanın bir örneğini gösterir. Her iki örnekte de, stream sonunu belirtmek için `-1` kullanılır.

```
void copyStream(InputStream in, OutputStream out) throws IOException {
    int b;
    while ((b = in.read()) != -1) {
        out.write(b);
    }
}

void copyStream(Reader in, Writer out) throws IOException {
    int b;
    while ((b = in.read()) != -1) {
        out.write(b);
    }
}
```

English: Hold on. We said we are reading and writing bytes, so why do the methods use int instead of byte? Remember, the byte data type has a range of 256 characters. They needed an extra value to indicate the end of an I/O stream. The authors of Java decided to use a larger data type, int, so that special values like -1 would indicate the end of an I/O stream. The output stream classes use int as well, to be consistent with the input stream classes.

Türkçe: Bir dakika. bytes okuduğumuzu ve yazdığımızı söyledik, bu yüzden yöntemler neden byte yerine int kullanıyor? byte veri türünün 256 karakterlik bir aralığı olduğunu unutmayın. Bir I/O stream sonunu belirtmek için fazladan bir değere ihtiyaçları vardı. Java yazarları daha büyük bir veri türü olan int kullanmaya karar verdiler, böylece -1 gibi özel değerler bir I/O stream sonunu gösterecekti. output stream sınıfları, input stream sınıflarıyla tutarlı olmak için int sınıflarını da kullanır.

English: Reading and writing one byte at a time isn't a particularly efficient way of doing this. Luckily, there are overloaded methods for reading and writing multiple bytes at a time. The offset and length values are applied to the array itself. For example, an offset of 3 and length of 5 indicates that the stream should read up to five bytes/characters of data and put them into the array starting with position 3. Let's look at an example:

Türkçe: Bir seferde bir byte okumak ve yazmak, bunu yapmanın özellikle etkili bir yolu değildir. Neyse ki, bir seferde birden fazla bytes okumak ve yazmak için aşırı yüklenmiş yöntemler vardır. Ofset ve uzunluk değerleri dizinin kendisine uygulanır. Örneğin, 3'lük bir ofset ve 5'lik bir uzunluk, stream'in 5 bytes/characters'a kadar veri okuması ve bunları 3 konumundan başlayarak diziye koymasına gerektiğini gösterir. Bir örneğe bakalım:

```
void copyStream(InputStream in, OutputStream out) throws IOException {
    int batchSize = 1024;
    var buffer = new byte[batchSize];
    int lengthRead;
    while ((lengthRead = in.read(buffer, 0, batchSize)) > 0) {
        out.write(buffer, 0, lengthRead);
        out.flush();
    }
}
```

English: Instead of reading the data one byte at a time, we read and write up to 1024 bytes at a time on line 14. The return value lengthRead is critical for determining whether we are at the end of the stream and knowing how many bytes we should write into our output stream.

Türkçe: Her seferinde tek byte okumak yerine 14. satırda bir kerede 1.024 byte'a kadar okuyup yazarız. `lengthRead` return value'su, stream'in sonuna ulaşıp ulaşılmadığını ve output stream'e kaç byte yazılması gerektiğini belirlemek için kritik önemlidir.

English: Unless our file happens to be a multiple of 1024 bytes, the last iteration of the while loop will write some value less than 1024 bytes. For example, if the buffer size is 1,024 bytes

Türkçe: Dosyamız 1024 bytes'un bir katı olmadıkça, while döngüsündeki son yineleme 1024 bytes'den daha az bir değer yazacaktır. Örneğin, tampon boyutu 1,024 bytes

English: and the file size is 1,054 bytes, the last read will be only 30 bytes. If we ignored this return value and instead wrote 1,024 bytes, 994 bytes from the previous loop would be written to the end of the file.

Türkçe: ve file boyutu 1.054 byte ise son okuma yalnızca 30 byte alır. Bu return value'yu göz ardı edip bunun yerine 1.024 byte yazsaydık önceki loop'tan kalan 994 byte da file'in sonuna eklenirdi.

English: We also added a `flush()` method on line 16 to reduce the amount of data lost if the application terminates unexpectedly. When data is written to an output stream, the underlying operating system does not guarantee that the data will make it to the file system immediately. The `flush()` method requests that all accumulated data be written immediately to disk. It is not without cost, though. Each time it is used, it may cause a noticeable delay in the application, especially for large files. Unless the data that you are writing is extremely critical, the `flush()` method should be used only intermittently. For example, it should not necessarily be called after every write, as it is in this example.

Türkçe: Ayrıca, uygulama beklenmedik bir şekilde sona ererse kaybedilen veri miktarını azaltmak için 16. satırda bir `flush()` yöntemi ekledik. Veriler bir output stream'e yazıldığında, altta yatan işletim sistemi, verilerin file system'ye hemen ulaşacağını garanti etmez. `flush()` yöntemi, birikmiş tüm verilerin derhal diske yazılmasını ister. Yine de maliyetsiz değil. Her kullanıldığında, özellikle büyük dosyalar için uygulamada gözle görülür bir gecikmeye neden olabilir. Yazdığınız veriler son derece kritik olmadığı sürece, `flush()` yöntemi yalnızca aralıklı olarak kullanılmalıdır. Örneğin, bu örnekte olduğu gibi, her yazıdan sonra mutlaka çağırılmamalıdır.

English: Equivalent methods exist on Reader and Writer, but they use char rather than byte, making the equivalent `copyStream()` method very similar.

Türkçe: Eşdeğer yöntemler Reader ve Writer üzerinde mevcuttur, ancak byte yerine char kullanırlar ve eşdeğer `copyStream()` yöntemini çok benzer hale getirirler.

English: The previous example makes reading and writing a file look like a lot to think about. That's because it only uses low-level I/O streams. Let's try again using high-level streams.

Türkçe: Önceki örnek, bir dosyayı okuma ve yazmanın düşünülmesi gereken çok şey gibi görünmesini sağlar. Bunun nedeni yalnızca low-level I/O stream'leri kullanmasıdır. High-level streams kullanarak tekrar deneyelim.

```
void copyTextFile(File src, File dest) throws IOException {
    try (var reader = new BufferedReader(new FileReader(src));
        var writer = new BufferedWriter(new FileWriter(dest))) {
        String line = null;
        while ((line = reader.readLine()) != null) {
            writer.write(line);
            writer.newLine();
        }
    }
}
```

English: The key is to choose the most useful high-level classes. In this case, we are dealing with a File, so we want to use a FileReader and FileWriter. Both classes have constructors that can take either a String representing the location or a File directly.

Türkçe: Önemli olan, en kullanışlı high-level class'ları seçmektir. Burada bir File ile çalıştığımız için FileReader ve FileWriter kullanırız. İki class'ın da location gösteren String veya doğrudan File alan constructor'ları vardır.

English: If the source file does not exist, a FileNotFoundException, which inherits IOException, will be thrown. If the destination file already exists, this implementation will overwrite it. We can pass an optional boolean second parameter to FileWriter for an append flag if we want to change this behavior.

Türkçe: Source file yoksa IOException'dan türeyen FileNotFoundException fırlatılır. Target file zaten varsa bu implementation onun üzerine yazar. Davranışı append moduna çevirmek için FileWriter constructor'ına isteğe bağlı ikinci bir boolean parameter verilebilir.

English: We also chose to use a `BufferedReader` and `BufferedWriter` so we can read a whole line at a time. This gives us the benefits of reading batches of characters on line 30 without having to write custom logic. Line 31 writes out the whole line of data at once. Since reading a line strips the line breaks, we add those back on line 32. Lines 27 and 28 demonstrate chaining constructors. The try-with-resources constructor takes care of closing all the objects in the chain.

Türkçe: Bir defada bütün line'ı okuyabilmek için `BufferedReader` ve `BufferedWriter` da kullanılır. Böylece 30. satırda özel logic yazmadan character batch'leri okunur. 31. satır bütün data line'ını bir defada yazar. `readLine()` line break'i kaldırdığı için 32. satırda geri eklenir. 27 ve 28. satırlar constructor chaining'i gösterir. try-with-resources, chain'deki bütün object'leri kapatır.

English: Now imagine that we wanted byte data instead of characters. We would need to choose different high-level classes: `BufferedInputStream`, `BufferedOutputStream`, `FileInputStream`, and `FileOutputStream`. We would call `readAllBytes()` instead of `readLine()` and store the result in a `byte[]` instead of a `String`. Finally, we wouldn't need to handle new lines since the data is binary.

Türkçe: Şimdi karakterler yerine byte verilerini istediğimizi hayal edin. Farklı üst düzey sınıfları seçmemiz gerekir: `BufferedInputStream`, `BufferedOutputStream`, `FileInputStream` ve `FileOutputStream`. `readLine()` yerine `readAllBytes()` çağırır ve sonucu `String` yerine `byte[]` olarak sakladık. Son olarak, veriler ikili olduğundan yeni satırları ele almamıza gerek kalmaz.

English: We can do a little better than `BufferedOutputStream` and `BufferedWriter` by using a `PrintStream` and `PrintWriter`. These classes contain four key methods. The `print()` and `println()` methods print data with and without a new line, respectively. There are also the `format()` and `printf()` methods, which we describe in the section on user interactions.

Türkçe: `BufferedOutputStream` ve `BufferedWriter`'den biraz daha iyi bir `PrintStream` ve `PrintWriter` kullanarak yapabiliriz. Bu sınıflar dört temel yöntem içerir. `print()` ve `println()` yöntemleri verileri sırasıyla yeni bir satırla ve olmadan yazdırır. Ayrıca, kullanıcı etkileşimleri bölümünde tanımladığımız `format()` ve `printf()` yöntemleri de vardır.

```
void copyTextFile(File src, File dest) throws IOException {
    try (var reader = new BufferedReader(new FileReader(src));
        var writer = new PrintWriter(new FileWriter(dest))) {
        String line = null;
        while ((line = reader.readLine()) != null)
            writer.println(line);
    }
}
```

English: While we used a `String`, there are numerous overloaded versions of `println()`, which take everything from primitives and `String` values to objects. Under the covers, these methods often just perform `String.valueOf()`.

Türkçe: Bir `String` kullanırken, ilkellerden ve `String` değerlerinden nesnelere kadar her şeyi alan çok sayıda aşırı yüklenmiş `println()` sürümü vardır. Kapakların altında, bu yöntemler genellikle sadece `String.valueOf()` gerçekleştirir.

English: The print stream classes have the distinction of being the only I/O stream classes we cover that do not have corresponding input stream classes. And unlike other OutputStream classes, PrintStream does not have Output in its name.

Türkçe: Baskı stream sınıfları, ilgili input stream sınıflarına sahip olmayan tek I/O stream sınıfları olma ayrımlına sahiptir. Ve diğer OutputStream sınıflarının aksine, PrintStream kendi adına Çıktıya sahip değildir.

English: It may surprise you that you've been regularly using a PrintStream throughout this book. Both System.out and System.err are PrintStream objects. Likewise, System.in, often useful for reading user input, is an InputStream.

Türkçe: Bu kitap boyunca düzenli olarak bir PrintStream kullanmanız sizi şaşırtabilir. System.out ve System.err, PrintStream nesneleridir. Aynı şekilde, kullanıcı girdisini okumak için genellikle yararlı olan System.in bir InputStream dir.

English: Unlike the majority of the other I/O streams we've covered, the methods in the print stream classes do not throw any checked exceptions. If they did, you would be required to catch a checked exception any time you called System.out.print()!

Türkçe: Diğer I/O streams sınıflarının çoğundan farklı olarak, stream sınıflarındaki yöntemler herhangi bir checked exceptions atmaz. Eğer yaptılarsa, System.out.print() diye çağırdığınız her zaman bir checked exception yakalamanız gerekir!

English: The line separator is \n or \r\n, depending on your operating system. The println() method takes care of this for you. If you need to get the character directly, either of the following will return it for you:

Türkçe: Hat ayırıcı, işletim sisteminize bağlı olarak n veya rn'dir. println() yöntemi bunu sizin için halleder. Karakteri doğrudan almanız gerekiyorsa, aşağıdakilerden biri sizin için iade edecektir:

```
System.getProperty(" line.separator ");
System.lineSeparator();
```

Enhancing with Files

English: The NIO.2 APIs provide even easier ways to read and write a file using the Files class. Let's start by looking at three ways of copying a file by reading in the data and writing it back:

Türkçe: NIO.2 API'leri, Files sınıfını kullanarak bir dosyayı okumanın ve yazmanın daha kolay yollarını sağlar. Veriyi okuyarak ve geri yazarak bir dosyayı kopyalamanın üç yoluna bakarak başlayalım:

```
private void copyPathAsString(Path input, Path output) throws IOException {
    String string = Files.readString(input);
```

```
    Files.writeString(output, string);
}
private void copyPathAsBytes(Path input, Path output) throws IOException {
    byte[] bytes = Files.readAllBytes(input);
    Files.write(output, bytes);
}
private void copyPathAsLines(Path input, Path output) throws IOException {
    List<String> lines = Files.readAllLines(input);
    Files.write(output, lines);
}
```

English: That's pretty concise! You can read a Path as a String, a byte array, or a List. Be aware that the entire file is read at once for all three of these, thereby storing all of the contents of the file in memory at the same time. If the file is significantly large, you may trigger an `OutOfMemoryError` when trying to load all of it into memory. Luckily, there is an alternative. This time, we print out the file as we read it.

Türkçe: Oldukça kısa! Bir `Path` ile gösterilen file'ı `String`, byte array veya `List` olarak okuyabilirsiniz. Ancak bu üç yaklaşım da file'ın tamamını bir kerede okuyup bütün içeriği aynı anda memory'de tutar. File çok büyükse tamamını yüklemeye çalışmak `OutOfMemoryError`a yol açabilir. Neyse ki file'ı okurken eşzamanlı yazdıran bir alternatif vardır:

```
private void readLazily(Path path) throws IOException {
    try (Stream<String> s = Files.lines(path)) {
        s.forEach(System.out::println);
    }
}
```

English: Now the contents of the file are read and processed lazily, which means that only a small portion of the file is stored in memory at any given time. Taking things one step further, we can leverage other stream methods for a more powerful example.

Türkçe: Şimdi dosyanın içeriği okunur ve tembelce işlenir, bu da dosyanın yalnızca küçük bir kısmının herhangi bir zamanda bellekte depolandığı anlamına gelir. İşleri bir adım daha ileri götürerek, daha güçlü bir örnek için diğer stream yöntemlerinden yararlanabiliriz.

```
try (var s = Files.lines(path)) {
    s.filter(f -> f.startsWith("WARN:"))
      .map(f -> f.substring(5))
      .forEach(System.out::println);
}
```

English: This sample code searches a log for lines that start with `WARN:`, outputting the text that follows. Assuming that the input file `sharks.log` is as follows:

Türkçe: Bu örnek kod `WARN` ile başlayan satırlar için bir günlük arar: aşağıdaki metni çıkarır. `sharks.log` giriş dosyasının aşağıdaki gibi olduğunu varsayarsak:

```
INFO:Server starting
DEBUG:Processes available = 10
WARN:No database could be detected
DEBUG:Processes available reset to 0
WARN:Performing manual recovery
INFO:Server successfully started
```

English: Then the sample output would be the following:

Türkçe: Daha sonra örnek çıktı aşağıdaki olacaktır:

```
No database could be detected
Performing manual recovery
```

English: As you can see, we have the ability to manipulate files in complex ways, often with only a few short expressions.

Türkçe: Gördüğünüz gibi, dosyaları karmaşık şekillerde, genellikle sadece birkaç short ifadeyle manipüle etme yeteneğine sahibiz.

English: `Files.readAllLines()` vs. `Files.lines()`

Türkçe: `Files.readAllLines()` vs. `Files.lines()`

English: For the exam, you need to know the difference between `readAllLines()` and `lines()`. Both of these examples compile and run:

Türkçe: Sınav için `readAllLines()` ve `lines()` arasındaki farkı bilmeniz gerekir. Bu örneklerin her ikisi de derleyip çalıştırır:

```
Files.readAllLines(Paths.get("birds.txt")).forEach(System.out::println);  
Files.lines(Paths.get("birds.txt")).forEach(System.out::println);
```

English: The first line reads the entire file into memory and performs a print operation on the result, while the second line lazily processes each line and prints it as it is read. The advantage of the second code snippet is that it does not require the entire file to be stored in memory at any time.

Türkçe: İlk satır file'ın tamamını memory'ye alıp sonucu yazdırır. İkinci satır ise her line'ı lazy biçimde işler ve okunduğu anda yazdırır. İkinci snippet'in avantajı, hiçbir anda file'ın tamamını memory'de tutmayı gerektirmemesidir.

English: You should also be aware of when they are mixing incompatible types on the exam. Do you see why the following does not compile?

Türkçe: Sınavda uyumsuz tipleri ne zaman karıştırdıklarının da farkında olmalısınız. Aşağıdakilerin neden derlenmediğini görüyor musunuz?

```
Files.readAllLines(Paths.get("birds.txt"))  
.filter(s -> s.length() > 2)  
.forEach(System.out::println);
```

English: The `readAllLines()` method returns a `List`, not a `Stream`, so the `filter()` method is not available.

Türkçe: `readAllLines()` yöntemi `Stream` değil, `List` döndürür, bu nedenle `filter()` yöntemi kullanılamaz.

Combining with `newBufferedReader()` and `newBufferedWriter()`

English: Sometimes you need to mix I/O streams and NIO.2. Conveniently, `Files` includes two convenience methods for getting I/O streams.

Türkçe: Bazen I/O streams ve NIO.2'yi karıştırmamız gerekir. Uygun bir şekilde, `Files` I/O streams elde etmek için iki kolaylık yöntemi içerir.

```
private void copyPath(Path input, Path output) throws IOException {  
    try (var reader = Files.newBufferedReader(input);  
        var writer = Files.newBufferedWriter(output)) {  
        String line = null;
```

```
while ((line = reader.readLine()) != null)
writer.write(line);
writer.newLine();
} } }
```

English: You can wrap I/O stream constructors to produce the same effect, although it's a lot easier to use the factory method. The first method, `newBufferedReader()`, reads the file specified at the `Path` location using a `BufferedReader` object.

Türkçe: Aynı etkiyi I/O stream constructor'larını wrap ederek de elde edebilirsiniz; fakat factory method kullanmak çok daha kolaydır. İlk method olan `newBufferedReader()`, `Path` ile belirtilen file'ı bir `BufferedReader` object'i aracılığıyla okur.

Reviewing Common Read and Write Methods

English: TABLE 14.9 reviews the public common I/O stream methods you should know for reading and writing. We also include `close()` and `flush()` since they are used when performing these actions. TABLE 14.10 does the same for common public NIO.2 read and write methods.

Türkçe: TABLE 14.9, okuma ve yazma için bilmeniz gereken public ortak I/O stream yöntemlerini gözden geçirir. Bu eylemleri gerçekleştirirken kullanıldıkları için `close()` ve `flush()`'ü de dahil ediyoruz. TABLE 14.10 ortak public NIO.2 okuma ve yazma yöntemleri için de aynı şeyi yapar.

English: TABLE 14.9 — Common I/O read and write methods

Class	Method	Description
All input streams	<code>public int read()</code>	Reads one value; returns <code>-1</code> if no bytes are available
<code>InputStream</code>	<code>public int read(byte[] b)</code>	Reads into a buffer; returns byte count
<code>Reader</code>	<code>public int read(char[] c)</code>	Reads into a buffer; returns character count
<code>InputStream</code>	<code>public int read(byte[] b, int offset, int length)</code>	Reads up to <code>length</code> bytes starting at <code>offset</code>
<code>Reader</code>	<code>public int read(char[] c, int offset, int length)</code>	Reads up to <code>length</code> characters starting at <code>offset</code>
All output streams	<code>public void write(int b)</code>	Writes one value
<code>OutputStream</code>	<code>public void write(byte[] b)</code>	Writes a byte array
<code>Writer</code>	<code>public void write(char[] c)</code>	Writes a character array
<code>OutputStream</code>	<code>public void write(byte[] b, int offset, int length)</code>	Writes <code>length</code> bytes starting at <code>offset</code>
<code>Writer</code>	<code>public void write(char[] c, int offset, int length)</code>	Writes <code>length</code> characters starting at <code>offset</code>
<code>BufferedInputStream</code>	<code>public byte[] readAllBytes()</code>	Reads data in bytes

Türkçe: TABLO 14.9 — Yaygın I/O read/write method'ları. Input method'ları EOF için `-1`, buffer overload'larında actual count döndürür. Output method'larında `offset` başlangıç index'idir.

Java 17 editör notu: Kaynak tablonun “returns `-1` if no bytes are available” ifadesi geçici availability olarak okunmamalıdır. `read()` için `-1`, **end-of-stream (EOF)** demektir. Ayrıca `readAllBytes()` yalnız `BufferedInputStream`a özgü değildir; Java 17’de `InputStream` instance method’ıdır ve subclass’lar tarafından miras alınır.

English: TABLE 14.9 — Common I/O read and write methods (continued)

Class	Method	Description
<code>BufferedReader</code>	<code>public String readLine()</code>	Reads a line
<code>BufferedWriter</code>	<code>public void write(String line)</code>	Writes a line
<code>BufferedWriter</code>	<code>public void newLine()</code>	Writes a new line
All output streams	<code>public void flush()</code>	Flushes buffered data through the stream
All streams	<code>public void close()</code>	Closes stream and releases resources

Türkçe: TABLO 14.9 — Yaygın I/O read/write method'ları (devam). `readLine()` line break'i result'a katmaz; `newLine()` platform line separator'ını yazar. `flush()` pending output'u aktarır; `close()` resource'u serbest bırakır.

English: TABLE 14.10 — Common Files NIO.2 read and write methods

Method	Description
<code>public static byte[] readAllBytes()</code>	Reads all data as bytes
<code>public static String readString()</code>	Reads all data into a <code>String</code>
<code>public static List<String> readAllLines()</code>	Reads all data into a <code>List</code>
<code>public static Stream<String> lines()</code>	Lazily reads data
<code>public static void write(Path path, byte[] bytes)</code>	Writes a byte array
<code>public static void writeString(Path path, String string)</code>	Writes a <code>String</code>
<code>public static void write(Path path, List<String> list)</code>	Writes a list of lines

Türkçe: TABLO 14.10 — Yaygın Files NIO.2 read/write method'ları. `readAll*` method'ları eager; `lines()` lazy'dir. Write method'ları written `Path` value'sunu döndürür.

Java 17 editör notu: Tablo kaynak gösterimini korur; bunlar tam formal signature değildir. Read method'ları `Path` alır. `Files.write(...)` ve `Files.writeString(...)` Java 17'de `void` değil `written Path` value'sunu döndürür; line-writing overload'u `Iterable<? extends CharSequence>` kabul eder.

Serializing Data

English: Throughout this book, we have been managing our data model using classes, so it makes sense that we would want to save these objects between program executions. Data about our zoo animals' health wouldn't be particularly useful if it had to be entered every time the program runs!

Türkçe: Bu kitap boyunca, veri modelimizi sınıfları kullanarak yönetiyoruz, bu yüzden bu nesneleri program yürütmeleri arasında kaydetmek istememiz mantıklı. Hayvanat bahçesi hayvanlarımızın sağlığıyla ilgili veriler, program her çalıştığında girilmesi gerekseydi özellikle yararlı olmazdı!

English: You can certainly use the I/O stream classes you've learned about so far to store text and binary data, but you still have to figure out how to put the data in the I/O stream and then decode it later. There are various file formats like XML and CSV you can standardize to, but you often have to build the translation yourself.

Türkçe: Metin ve binary data'yı saklamak için öğrendiğiniz I/O stream class'larını kullanabilirsiniz; ancak data'yı stream'e hangi formatta yazacağınızı ve daha sonra nasıl decode edeceğinizi kendiniz belirlemelisiniz. XML ve CSV gibi standardize edilebilen çeşitli file format'ları vardır, fakat object ile bu format arasındaki dönüşümü çoğunlukla sizin yazmanız gerekir.

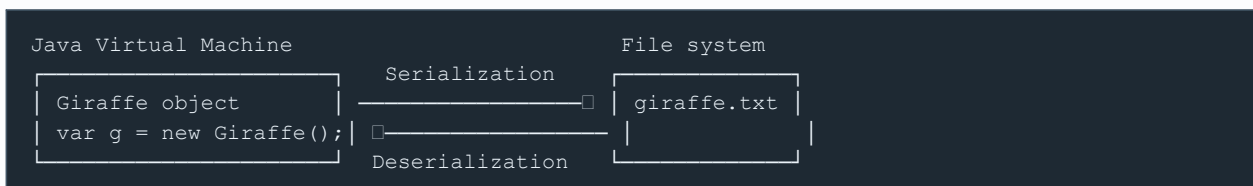
English: Alternatively, we can use serialization to solve the problem of how to convert objects to/from an I/O stream. Serialization is the process of converting an in-memory object to a byte stream. Likewise, deserialization is the process of converting from a byte stream into an object. Serialization often involves writing an object to a stored or transmittable format, while deserialization is the reciprocal process.

Türkçe: Alternatif olarak object'leri bir I/O stream'e dönüştürme ve stream'den geri oluşturma sorununu serialization ile çözebiliriz. Serialization, memory'deki bir object'i byte stream'e dönüştürme işlemidir. Deserialization ise byte stream'i yeniden object'e dönüştüren ters işlemdir. Serialization object'i saklanabilir veya iletilabilir bir formata yazar; deserialization bu dönüşümü geri alır.

English: FIGURE 14.6 shows a visual representation of serializing and deserializing a Giraffe object to and from a giraffe.txt file.

Türkçe: FIGURE 14.6, bir `Giraffe` object'inin `giraffe.txt` file'ına serialization'ını ve file'dan deserialization'ını görsel olarak gösterir.

English: FIGURE 14.6 — Serialization process



Türkçe: ŞEKİL 14.6 — Serialization süreci. JVM'deki `Giraffe` object'i serialization ile `giraffe.txt` file'ına yazılır; deserialization ters yönde byte representation'dan object oluşturur.

English: In this section, we show you how Java provides built-in mechanisms for serializing and deserializing I/O streams of objects directly to and from disk, respectively.

Türkçe: Bu bölümde Java'nın object'leri doğrudan disk'e serialize etmek ve disk'ten deserialize etmek için sunduğu yerleşik I/O stream mekanizmalarını göstereceğiz.

Applying the Serializable Interface

English: To serialize an object using the I/O API, the object must implement the `java.io.Serializable` interface. The `Serializable` interface is a marker interface, which means it does not have any methods. Any class can implement the `Serializable` interface since there are no required methods to implement.

Türkçe: I/O API ile bir object'i serialize etmek için object'in class'ı `java.io.Serializable` interface'ini implement etmelidir. `Serializable`, method içermeyen bir marker interface'tir. Implement edilmesi zorunlu method olmadığı için herhangi bir class bu interface'i implement edebilir.

English: Since `Serializable` is a marker interface with no abstract members, why not just apply it to every class? Generally speaking, you should only mark data-oriented classes serializable. Process-oriented classes, such as the I/O streams discussed in this chapter or the `Thread` instances you learned about in Chapter 13, are often poor candidates for serialization, as the internal state of those classes tends to be ephemeral or short-lived.

Türkçe: `Serializable` abstract member içermeyen bir marker interface ise neden her class'a uygulanmasın? Genel olarak yalnızca data-oriented class'lar serializable olarak işaretlenmelidir. Bu bölümdeki I/O stream'ler veya Bölüm 13'teki `Thread` instance'ları gibi process-oriented class'lar serialization için çoğunlukla uygun değildir; çünkü internal state'leri geçici ve kısa ömürlüdür.

English: The purpose of using the `Serializable` interface is to inform any process attempting to serialize the object that you have taken the proper steps to make the object serializable. All Java primitives and many of the built-in Java classes that you have worked with throughout this book are `Serializable`. For example, this class can be serialized:

Türkçe: `Serializable` interface'ini kullanmanın amacı, object'i serialize etmeye çalışan process'e class'ı serializable hâle getirmek için gerekli adımların atıldığını bildirmektir. Bütün Java primitive'leri ve kitap boyunca kullandığımız built-in Java class'larının çoğu `Serializable`dir. Örneğin aşağıdaki class serialize edilebilir:

```
import java.io.Serializable;
public class Gorilla implements Serializable {
    private static final long serialVersionUID = 1L;
    private String name;
    private int age;
    private Boolean friendly;
    private transient String favoriteFood;
    // Constructors/Getters/Setters/toString() omitted
}
```

English: In this example, the Gorilla class contains three instance members (name, age, friendly) that will be saved to an I/O stream if the class is serialized. Note that since Serializable is not part of the java.lang package, it must be imported or referenced with the package name.

Türkçe: Bu örnekte Gorilla class'ı, class serialize edildiğinde I/O stream'e kaydedilecek üç instance member (name, age, friendly) içerir. Serializable, java.lang package'ında olmadığından import edilmeli ya da fully qualified name ile kullanılmalıdır.

English: What about the favoriteFood field that is marked transient? Any field that is marked transient will not be saved to an I/O stream when the class is serialized. We discuss that in more detail next.

Türkçe: Peki transient işaretli favoriteFood field'ı ne olur? transient olan hiçbir field, class serialize edildiğinde I/O stream'e kaydedilmez. Bunu birazdan ayrıntılı ele alacağız.

English: Maintaining a serialVersionUID It's a good practice to declare a static serialVersionUID variable in every class that implements Serializable. The version is stored with each object as part of serialization. Then, every time the class structure changes, this value is updated or incremented.

Türkçe: serialVersionUID'i Korumak — Serializable implement eden her class'ta static serialVersionUID variable'ı declare etmek iyi bir uygulamadır. Bu version bilgisi serialization sırasında her object ile birlikte saklanır. Kaynak metin, class yapısı değiştiğinde değerin güncellendiğini veya artırıldığını belirtir.

Java 17 editör notu: serialVersionUID her class structure değişiminde mekanik olarak artırılan bir counter değildir. Serialized compatibility bilinçli olarak bozulacaksa değiştirilir; compatible evolution sırasında aynı tutulabilir.

English: Perhaps our Gorilla class receives a new instance member Double banana, or maybe the age field is renamed. The idea is a class could have been serialized with an older version of the class and deserialized with a newer version of the class.

Türkçe: Örneğin Gorilla class'ına Double banana adlı yeni bir instance member eklenebilir veya age field'ı yeniden adlandırılabilir. Bir object class'ın eski sürümüyle serialize edilip daha yeni sürümüyle deserialize edilmeye çalışılabilir.

English: The serialVersionUID helps inform the JVM that the stored data may not match the new class definition. If an older version of the class is encountered during deserialization, a java.io.InvalidClassException may be thrown. Alternatively, some APIs support converting data between versions.

Türkçe: serialVersionUID, saklanan data'nın yeni class definition'ıyla eşleşmeyebileceğini JVM'in anlamasına yardımcı olur. Deserialization sırasında class'ın eski bir sürümüyle karşılaşırsa java.io.InvalidClassException fırlatılabilir. Bazı API'ler ise sürümler arasında data dönüşümünü destekler.

Marking Data transient

English: The transient modifier can be used for sensitive data of the class, like a password. There are other objects it does not make sense to serialize, like the state of an in-memory Thread. If the object is part of a serializable object, we just mark it transient to ignore these select instance members.

Türkçe: `transient` modifier, password gibi hassas class data'sı için kullanılabilir. Memory'deki bir `Thread` state'i gibi serialize edilmesi anlamlı olmayan başka object'ler de vardır. Böyle bir object serializable bir object'in parçasıysa ilgili instance member'ı serialization dışında bırakmak için `transient` işaretleriz.

English: What happens to data marked transient on deserialization? It reverts to its default Java values, such as 0.0 for double, or null for an object. You see examples of this shortly when we present the object stream classes.

Türkçe: Deserialization sırasında `transient` işaretli data'ya ne olur? Değer, `double` için `0.0`, object reference için `null` gibi ilgili default Java value'ya döner. Object stream class'larını tanıtırken bunun örneklerini göreceğiz.

English: Marking static fields transient has little effect on serialization. Other than the `serialVersionUID`, only the instance members of a class are serialized.

Türkçe: Static field'ları `transient` işaretlemek serialization üzerinde etkisiz sayılır. `serialVersionUID` dışında yalnızca class'ın instance member'ları serialize edilir.

Ensuring That a Class Is Serializable

English: Since Serializable is a marker interface, you might think there are no rules to using it. Not quite! Any process attempting to serialize an object will throw a `NotSerializableException` if the class does not implement the Serializable interface properly.

Türkçe: `Serializable` bir marker interface olduğu için kullanımının hiçbir kuralı olmadığını düşünebilirsiniz. Oysa bir object'i serialize etmeye çalışan process, class `Serializable` kurallarını karşılamıyorsa `NotSerializableException` fırlatır.

English: How to Make a Class Serializable The class must be marked Serializable.

Türkçe: Bir Class Nasıl Serializable Yapılır? — Class, `Serializable` olarak işaretlenmelidir.

English: Every instance member of the class must be serializable, marked transient, or have a null value at the time of serialization.

Türkçe: Class'ın her instance member'ı serializable olmalı, `transient` işaretlenmeli ya da serialization anında `null` değerini taşımalıdır.

English: Be careful with the second rule. For a class to be serializable, we must apply the second rule recursively. Do you see why the following Cat class is not serializable?

Türkçe: İkinci kurala dikkat edin: Bir class'ın serializable olması için bu kural recursive olarak uygulanmalıdır. Aşağıdaki `Cat` class'ının neden serialize edilemediğini görüyor musunuz?

```
public class Cat implements Serializable {
    private Tail tail = new Tail();
}
public class Tail implements Serializable {
    private Fur fur = new Fur();
}
public class Fur {}
```

English: Cat contains an instance of Tail, and both of those classes are marked Serializable, so no problems there. Unfortunately, Tail contains an instance of Fur that is not marked Serializable.

Türkçe: `Cat`, bir `Tail` instance'ı içerir ve iki class da `Serializable` işaretlidir; buraya kadar sorun yoktur. Ne var ki `Tail`, `Serializable` olmayan bir `Fur` instance'ı içerir.

English: Either of the following changes fixes the problem and allows Cat to be serialized:

Türkçe: Aşağıdaki değişikliklerden herhangi biri sorunu giderir ve Cat'in seri hale getirilmesine izin verir:

```
public class Tail implements Serializable {
    private transient Fur fur = new Fur();
}
public class Fur implements Serializable {}
```

English: We could also make our tail or fur instance members null, although this would make Cat serializable only for particular instances, rather than all instances.

Türkçe: `tail` veya `fur` instance member'larını `null` da yapabiliriz; fakat bu durumda `Cat`, bütün instance'lar için değil yalnızca bu değerin `null` olduğu belirli instance'lar için serialize edilebilirdi.

English: Serializing Records Do you think this record is serializable?

Türkçe: Record'ları Serialize Etme — Sizce bu record serializable mıdır?

```
record Record(String name) {}
```

English: It is not serializable because it does not implement Serializable. A record follows the same rules as other types of classes with respect to whether it can be serialized. Therefore, this one can be:

Türkçe: `Serializable` uygulamadığı için serileştirilebilir değildir. Bir kayıt, serileştirilip serileştirilemeyeceği konusunda diğer sınıf türleri ile aynı kuralları izler. Bu nedenle, bu olabilir:

```
record Record(String name) implements Serializable {}
```

Java 17 editör notu: `Serializable` record'lar ordinary serializable class'ların construction kuralından ayrılır. Deserialization sırasında record'un canonical constructor'ı çağrılır.

Storing Data with `ObjectOutputStream` and `ObjectInputStream`

English: The `ObjectInputStream` class is used to deserialize an object, while the `ObjectOutputStream` is used to serialize an object. They are high-level streams that operate on existing I/O streams. While both of these classes contain a number of methods for built-in data types like primitives, the two methods you need to know for the exam are the ones related to working with objects.

Türkçe: `ObjectInputStream` bir object'i deserialize, `ObjectOutputStream` ise serialize etmek için kullanılır. İkisi de mevcut I/O stream'ler üzerinde çalışan high-level stream'lerdir. Bu class'lar primitive gibi built-in data type'lar için birçok method içerir; sınav açısından object'lerle çalışan aşağıdaki iki method'u bilmeniz gerekir.

```
// ObjectInputStream
public Object readObject() throws IOException, ClassNotFoundException
// ObjectOutputStream
public void writeObject(Object obj) throws IOException
```

English: Note the parameters, return types, and exceptions thrown. We now provide a sample method that serializes a List of Gorilla objects to a file:

Türkçe: Parameter'lara, return type'lara ve fırlatılan exception'lara dikkat edin. Şimdi `Gorilla` object'lerinden oluşan bir `List`i file'a serialize eden örnek bir method görelim:

```
void saveToFile(List<Gorilla> gorillas, File dataFile)
throws IOException {
```

```
try (var out = new ObjectOutputStream(
    new BufferedOutputStream(
        new FileOutputStream(dataFile)))) {
    for (Gorilla gorilla: gorillas)
        out.writeObject(gorilla);
}
}
```

English: Pretty easy, right? Notice that we start with a file stream, wrap it in a buffered I/O stream to improve performance, and then wrap that with an object stream. Serializing the data is as simple as passing it to `writeObject()`.

Türkçe: Oldukça kolaydır: Önce file stream oluşturulur, performance için buffered I/O stream ile, ardından object stream ile wrap edilir. Data'yı serialize etmek `writeObject()` a vermek kadar basittir.

English: Once the data is stored in a file, we can deserialize it by using the following method:

Türkçe: Data file'a kaydedildikten sonra aşağıdaki method ile deserialize edilebilir:

```

List<Gorilla> readFromFile(File dataFile) throws IOException,
ClassNotFoundException {
    var gorillas = new ArrayList<Gorilla>();
    try (var in = new ObjectInputStream(
        new BufferedInputStream(
            new FileInputStream(dataFile)))) {
        while (true) {
            var object = in.readObject();
            if (object instanceof Gorilla g)
                gorillas.add(g);
        }
    } catch (EOFException e) {
        // File end reached
    }
    return gorillas;
}

```

English: Ah, not as simple as our save method, was it? When calling `readObject()`, `null` and `-1` do not have any special meaning, as someone might have serialized objects with those values. Unlike our earlier techniques for reading methods from an input stream, we need to use an infinite loop to process the data, which throws an `EOFException` when the end of the I/O stream is reached.

Türkçe: Save method'umuz kadar basit değil. `readObject()` çağrısında `null` ve `-1` özel bir anlam taşımaz; çünkü bu değerlere sahip object'ler de serialize edilmiş olabilir. Önceki input stream okuma tekniklerinden farklı olarak data'yı infinite loop içinde işleriz; I/O stream'in sonuna gelindiğinde `EOFException` fırlatılır.

English: If your program happens to know the number of objects in the I/O stream, you can call `readObject()` a fixed number of times, rather than using an infinite loop.

Türkçe: Program I/O stream'deki object sayısını biliyorsa infinite loop yerine `readObject()` belirli sayıda çağrılabilir.

English: Since the return type of `readObject()` is `Object`, we need to check the type before obtaining access to our Gorilla properties. Notice that `readObject()` declares a checked `ClassNotFoundException` since the class might not be available on deserialization.

Türkçe: `readObject()` un return type'ı `Object` olduğundan `Gorilla` property'lerine erişmeden önce type'ı kontrol etmeliyiz. Ayrıca class deserialization sırasında classpath'te bulunmayabileceği için `readObject()` checked `ClassNotFoundException` declare eder.

English: The following code snippet shows how to call the serialization methods:

Türkçe: Aşağıdaki code snippet, serialization method'larının nasıl çağrıldığını gösterir:

```

var gorillas = new ArrayList<Gorilla>();
gorillas.add(new Gorilla("Grodd", 5, false));
gorillas.add(new Gorilla("Ishmael", 8, true));
File dataFile = new File("gorilla.data");
saveToFile(gorillas, dataFile);
var gorillasFromDisk = readFromFile(dataFile);
System.out.print(gorillasFromDisk);

```

English: Assuming that the `toString()` method was properly overridden in the `Gorilla` class, this prints the following at runtime:

Türkçe: `Gorilla` class'ında `toString()` un doğru biçimde override edildiğini varsayarsak runtime çıktısı şöyledir:

```
[[name=Grodd, age=5, friendly=false],
[name=Ishmael, age=8, friendly=true]]
```

English: `ObjectInputStream` inherits an `available()` method from `InputStream` that you might think can be used to check for the end of the I/O stream rather than throwing an `EOFException`. Unfortunately, this only tells you the number of blocks that can be read without blocking another thread. In other words, it can return 0 even if there are more bytes to be read.

Türkçe: `ObjectInputStream`, `InputStream`den `available()` method'unu inherit eder. Bunun `EOFException` yerine I/O stream'in sonunu denetlemek için kullanılabileceği düşünülebilir; ancak method yalnızca başka bir thread'i block etmeden okunabilecek tahmini byte sayısını bildirir. Dolayısıyla okunacak byte'lar kalsa bile 0 dönebilir.

Java 17 editör notu: `available()` “block sayısı” veya total remaining length değil, blocking olmadan okunabileceği tahmin edilen byte sayısını döndürür. 0 EOF garantisi değildir.

Understanding the Deserialization Creation Process

English: For the exam, you need to understand how a deserialized object is created. When you deserialize an object, the constructor of the serialized class, along with any instance initializers, is not called when the object is created. Java will call the no-arg constructor of the first nonserializable parent class it can find in the class hierarchy. In our `Gorilla` example, this would just be the no-arg constructor of `Object`.

Türkçe: Sınav için deserialized bir object'in nasıl oluşturulduğunu anlamalısınız. Object deserialize edilirken serialized class'ın constructor'ı ve instance initializer'ları çağrılmaz. Java, class hierarchy'de bulunduğu ilk nonserializable parent class'ın no-arg constructor'ını çağırır. `Gorilla` örneğinde bu, yalnızca `Object`in no-arg constructor'ıdır.

Java 17 editör notu: Bu construction kuralı ordinary serializable class'lar içindir. Serializable record deserialization'ında canonical constructor çağrılır.

English: As we stated earlier, any static or transient fields are ignored. Values that are not provided will be given their default Java value, such as null for String, or 0 for int values.

Türkçe: Daha önce belirttiğimiz gibi static ve `transient` field'lar göz ardı edilir. Kaydedilmemiş değerler `String` için null, `int` için 0 gibi default Java value'larını alır.

English: Let's take a look at a new `Chimpanzee` class. This time we do list the constructors to illustrate that none of them is used on deserialization.

Türkçe: Yeni bir `Chimpanzee` class'ına bakalım. Bu kez constructor'ların hiçbirinin deserialization sırasında kullanılmadığını göstermek için onları açıkça yazıyoruz.

```
import java.io.Serializable;
public class Chimpanzee implements Serializable {
    private static final long serialVersionUID = 2L;
    private transient String name;
    private transient int age = 10;
    private static char type = 'C';
    { this.age = 14; }
```

```
public Chimpanzee() {
    this.name = "Unknown";
    this.age = 12;
    this.type = 'Q';
}
public Chimpanzee(String name, int age, char type) {
    this.name = name;
    this.age = age;
    this.type = type;
}
// Getters/Setters/toString() omitted
}
```

English: Assuming we rewrite our previous serialization and deserialization methods to process a Chimpanzee object instead of a Gorilla object, what do you think the following prints?

Türkçe: Önceki serialization ve deserialization method'larımızı *Gorilla* yerine *Chimpanzee* object'i işleyecek biçimde yeniden yazdığımızı varsayalım. Aşağıdaki kod ne yazdırır?

```
var chimpanzees = new ArrayList<Chimpanzee>();
chimpanzees.add(new Chimpanzee("Ham", 2, 'A'));
chimpanzees.add(new Chimpanzee("Enos", 4, 'B'));
File dataFile = new File("chimpanzee.data");
saveToFile(chimpanzees, dataFile);
var chimpanzeesFromDisk = readFromFile(dataFile);
System.out.println(chimpanzeesFromDisk);
```

English: Think about it. Go on, we'll wait.

Türkçe: Bir düşün. Devam et, bekleyeceğiz.

English: Ready for the answer? Well, for starters, none of the instance members are serialized to a file. The name and age variables are both marked transient, while the type variable is static. We purposely accessed the type variable using this to see whether you were paying attention.

Türkçe: Öncelikle instance member'ların hiçbirisi file'a serialize edilmez. *name* ve *age* variable'ları *transient*, *type* ise *static*'tir. Dikkat edip etmediğinizi ölçmek için *type* variable'ına bilerek *this* üzerinden eriştik.

English: Upon deserialization, none of the constructors in Chimpanzee is called. Even the no-arg constructor that sets the values [name=Unknown,age=12,type=Q] is ignored. The instance initializer that sets age to 14 is also not executed.

Türkçe: Deserialization sırasında *Chimpanzee*deki constructor'ların hiçbirisi çağrılmaz.

[name=Unknown, age=12, type=Q] değerlerini atayan no-arg constructor da göz ardı edilir. *age* 14 yapan instance initializer da çalıştırılmaz.

English: In this case, the name variable is initialized to null since that's the default value for String in Java. Likewise, the age variable is initialized to 0. The program prints the following, assuming the toString() method is implemented:

Türkçe: Bu durumda name, Java'da String'in default value'su olan null; age ise 0 olarak initialize edilir. toString() un implement edildiğini varsayarsak program şu çıktıyı verir:

```
[name=null,age=0,type=B],
[name=null,age=0,type=B]]
```

English: What about the type variable? Since it's static, it will display whatever value was set last. If the data is serialized and deserialized within the same execution, it will display B, since that was the last Chimpanzee we created. On the other hand, if the program performs the deserialization and print on startup, it will print C, since that is the value the class is initialized with.

Türkçe: Peki type variable'ı? Static olduğu için en son atanan değeri gösterir. Data aynı execution içinde serialize ve deserialize edilirse, en son oluşturduğumuz Chimpanzeenin değeri olduğu için B görünür. Program yalnızca startup sırasında deserialize edip yazdırırsa class'ın initial value'su olan C görünür.

English: For the exam, make sure you understand that the constructor and any instance initializations defined in the serialized class are ignored during the deserialization process. Java only calls the constructor of the first non-serializable parent class in the class hierarchy.

Türkçe: Sınav için, deserialization sırasında serialized class'ın constructor'ı ile bütün instance initialization'larının göz ardı edildiğini iyi anlayın. Java yalnızca class hierarchy'deki ilk nonserializable parent class'ın constructor'ını çağırır.

English: Finally, let's add a subclass:

Türkçe: Son olarak, bir alt sınıf ekleyelim:

```
public class BabyChimpanzee extends Chimpanzee {
    private static final long serialVersionUID = 3L;
    private String mother = "Mom";
    public BabyChimpanzee() { super(); }
    public BabyChimpanzee(String name, char type) {
        super(name, 0, type);
    }
    // Getters/Setters/toString() omitted
}
```

English: Notice that this subclass is serializable because the superclass has implemented Serializable. We now have an additional instance variable. The code to serialize and deserialize remains the same. We can even still cast to Chimpanzee because this is a subclass.

Türkçe: Superclass Serializable implement ettiği için bu subclass da serializable'dır. Artık ek bir instance variable'ımız vardır. Serialization ve deserialization kodu aynı kalır. Object subclass instance'ı olduğundan yine Chimpanzeeye cast edilebilir.

Interacting with Users

English: Java includes numerous classes for interacting with the user. For example, you might want to write an application that asks a user to log in and then prints a success message. This section contains numerous techniques for handling and responding to user input.

Türkçe: Java kullanıcı ile etkileşim için çok sayıda sınıf içerir. Örneğin, bir kullanıcının oturum açmasını isteyen ve daha sonra bir başarı mesajı yazdıran bir uygulama yazmak isteyebilirsiniz. Bu bölüm, kullanıcı girdisini işlemek ve yanıtlamak için çok sayıda teknik içerir.

Printing Data to the User

English: Java includes two `PrintStream` instances for providing information to the user: `System.out` and `System.err`. While `System.out` should be old hat to you, `System.err` might be new to you. The syntax for calling and using `System.err` is the same as `System.out` but is used to report errors to the user in a separate I/O stream from the regular output information.

Türkçe: Java kullanıcıya bilgi sağlamak için iki `PrintStream` örneği içerir: `System.out` ve `System.err`. `System.out` sizin için eski bir şapka olsa da, `System.err` sizin için yeni olabilir. `System.err` çağırma ve kullanma sözdizimi `System.out` ile aynıdır, ancak hataları kullanıcıya normal çıktı bilgisinden ayrı bir I/O stream içinde bildirmek için kullanılır.

```
try (var in = new FileInputStream("zoo.txt")) {  
    System.out.println("Found file!");  
} catch (FileNotFoundException e) {  
    System.err.println("File not found!");  
}
```

English: How do they differ in practice? In part, that depends on what is executing the program. For example, if you are running from a command prompt, they will likely print text in the same format. On the other hand, if you are working in an integrated development environment (IDE), they might print the `System.err` text in a different color. Finally, if the code is being run on a server, the `System.err` stream might write to a different log file.

Türkçe: Uygulamada nasıl farklılıklar gösterirler? Kısım, bu programın ne yaptığına bağlıdır. Örneğin, bir komut isteminden çalıştırıyorsanız, muhtemelen metni aynı formatta yazdırırlar. Öte yandan, entegre bir geliştirme ortamında (IDE) çalışıyorsanız, `System.err` metnini farklı bir renkte basabilirler. Son olarak, kod bir sunucuda çalıştırılıyorsa, `System.err` stream farklı bir günlük dosyasına yazılabilir.

English: Using Logging APIs While `System.out` and `System.err` are incredibly useful for debugging stand-alone or simple applications, they are rarely used in professional software development. Most applications rely on a logging service or API.

Türkçe: `System.out` ve `System.err`, tek başına veya basit uygulamaların hata ayıklaması için inanılmaz derecede yararlı olsa da, profesyonel yazılım geliştirmede nadiren kullanılırlar. Çoğu uygulama bir günlük servisine veya API dayanır.

English: While many logging APIs are available, they tend to share a number of similar attributes. First you create a static logging object in each class. Then you log a message with an appropriate logging level: `debug()`, `info()`, `warn()`, or `error()`. The `debug()` and `info()` methods are useful as they allow developers to log things that aren't errors but may be useful.

Türkçe: Birçok günlükleme API'si mevcut olsa da, bir dizi benzer özelliği paylaşma eğilimindedirler. Önce her sınıfta bir static günlük nesnesi oluşturursunuz. Ardından bir mesajı uygun bir kayıt seviyesiyle kaydedersiniz: `debug()`, `info()`, `warn()` veya `error()`. `debug()` ve `info()` yöntemleri, geliştiricilerin hata olmayan ancak yararlı olabilecek şeyleri kaydetmelerine izin verdikleri için yararlıdır.

Reading Input as an I/O Stream

English: The `System.in` returns an `InputStream` and is used to retrieve text input from the user. It is commonly wrapped with a `BufferedReader` via an `InputStreamReader` to use the `readLine()` method.

Türkçe: `System.in` bir `InputStream` döndürür ve kullanıcıdan metin girişi almak için kullanılır. Genellikle `readLine()` yöntemini kullanmak için bir `InputStreamReader` aracılığıyla bir `BufferedReader` ile sarılır.

```
var reader = new BufferedReader(new InputStreamReader(System.in));
String userInput = reader.readLine();
System.out.println("You entered: " + userInput);
```

English: When executed, this application first fetches text from the user until the user presses the Enter key. It then outputs the text the user entered to the screen.

Türkçe: Çalıştırıldığında, bu uygulama ilk olarak kullanıcı Enter tuşuna basana kadar kullanıcıdan metin alır. Daha sonra kullanıcının ekrana girdiği metni çıkarır.

Closing System Streams

English: You might have noticed that we never created or closed `System.out`, `System.err`, and `System.in` when we used them. In fact, these are the only I/O streams in the entire chapter that we did not use a `try-with-resources` block on!

Türkçe: Bunları kullandığımızda `System.out`, `System.err` ve `System.in` oluşturmadığımızı veya kapatmadığımızı fark etmiş olabilirsiniz. Aslında, `try-with-resources` bloğunu kullanmadığımız tüm bölümdeki tek I/O streams bunlar!

English: Because these are static objects, the System streams are shared by the entire application. The JVM creates and opens them for us. They can be used in a `try-with-resources` statement

Türkçe: Bunlar static nesneleri olduğu için, Sistem streams tüm uygulama tarafından paylaşılır. JVM onları bizim için yaratır ve opens oluşturur. `try-with-resources` statement içinde kullanılabilirler.

English: or by calling `close()`, although closing them is not recommended. Closing the System streams makes them permanently unavailable for all threads in the remainder of the program.

Türkçe: veya `close()` olarak çağırarak, kapatma tavsiye edilmese de. streams Sistemin kapatılması, programın geri kalan kısmındaki tüm konular için kalıcı olarak kullanılamaz hale getirir.

Java 17 editör notu: Standard stream'i kapatmamak doğru uygulamadır; ancak “permanently unavailable” mutlak API kuralı değildir. Uygun permission/environment altında `System.setIn()`, `System.setOut()` ve `System.setErr()` ilgili static reference'ı başka stream ile değiştirebilir.

English: What do you think the following code snippet prints?

Türkçe: Aşağıdaki code snippet sizce ne yazdırır?

```
try (var out = System.out) {}  
System.out.println("Hello");
```

English: Nothing. It prints nothing. The methods of `PrintStream` do not throw any checked exceptions and rely on the `checkError()` to report errors, so they fail silently.

Türkçe: Hiçbir şey. Hiçbir şey basmıyor. `PrintStream` yöntemleri herhangi bir checked exceptions atmaz ve hataları bildirmek için `checkError()` 'a güvenmez, bu nedenle sessizce başarısız olurlar.

English: What about this example?

Türkçe: Peki ya bu örnek?

```
try (var err = System.err) {}  
System.err.println("Hello");
```

English: This one also prints nothing. Like `System.out`, `System.err` is a `PrintStream`. Even if it did throw an exception, we'd have a hard time seeing it since our I/O stream for reporting errors is closed! Closing `System.err` is a particularly bad idea, since the stack traces from all exceptions will be hidden.

Türkçe: Bu da bir şey yazmıyor. `System.out` gibi, `System.err` bir `PrintStream` dir. Bir istisna atsa bile, hataları bildirmek için I/O stream kapalı olduğundan bunu görmekte zorlanırdık! `System.err`'u kapatmak özellikle kötü bir fikirdir, çünkü tüm istisnalardan yığın izleri gizlenecektir.

English: Finally, what do you think this code snippet does?

Türkçe: Son olarak, bu kod snippet'inin ne yaptığını düşünüyorsunuz?

```
var reader = new BufferedReader(new InputStreamReader(System.in));  
try (reader) {}  
String data = reader.readLine(); // IOException
```

English: It prints an exception at runtime. Unlike the `PrintStream` class, most `InputStream` implementations will throw an exception if you try to operate on a closed I/O stream.

Türkçe: Çalışma zamanında bir istisna yazdırır. `PrintStream` sınıfından farklı olarak, çoğu `InputStream` uygulaması, kapalı bir I/O stream üzerinde çalışmaya çalışırsanız bir istisna atar.

Acquiring Input with Console

English: The `java.io.Console` class is specifically designed to handle user interactions. After all, `System.in` and `System.out` are just raw streams, whereas `Console` is a class with numerous methods centered around user input.

Türkçe: `java.io.Console` sınıfı, kullanıcı etkileşimlerini ele almak için özel olarak tasarlanmıştır. Sonuçta, `System.in` ve `System.out` sadece ham streams iken, `Console` kullanıcı girişi etrafında merkezlenmiş çok sayıda yöntemle sahip bir sınıftır.

English: The `Console` class is a singleton because it is accessible only from a factory method and only one instance of it is created by the JVM. For example, if you come across code on the exam such as the following, it does not compile, since the constructors are all private:

Türkçe: `Console` sınıfı bir singleton'dur, çünkü yalnızca bir factory method adresinden erişilebilir ve bunun sadece bir örneği JVM tarafından oluşturulur. Örneğin, aşağıdaki gibi sınavda kodla karşılaşırsanız, derleyicilerin hepsi private olduğundan, derlemez:

```
Console c = new Console(); // DOES NOT COMPILE
```

English: The following snippet shows how to obtain a `Console` and use it to retrieve user input:

Türkçe: Aşağıdaki snippet, `Console` nasıl elde edileceğini ve kullanıcı girdisini almak için nasıl kullanılacağını gösterir:

```
Console console = System.console();
if (console != null) {
    String userInput = console.readLine();
    console.writer().println("You entered: " + userInput);
} else {
    System.err.println("Console not available");
}
```

English: The `Console` object may not be available, depending on where the code is being called. If it is not available, `System.console()` returns `null`. It is imperative that you check for a `null` value before attempting to use a `Console` object!

Türkçe: `Console` nesnesi, kodun çağırıldığı yere bağlı olarak mevcut olmayabilir. Eğer mevcut değilse, `System.console()` `null` değerini gönderir. `Console` nesnesini kullanmaya çalışmadan önce `null` değerini kontrol etmeniz zorunludur!

English: This program first retrieves an instance of the `Console` and verifies that it is available, outputting a message to `System.err` if it is not. If it is available, the program retrieves a line of input from the user and prints the result. As you might have noticed, this example is equivalent to our earlier example of reading user input with `System.in` and `System.out`.

Türkçe: Bu program ilk olarak `Console` örneğini alır ve eğer değilse `System.err` iletisi vererek kullanılabilir olduğunu doğrular. Eğer varsa, program kullanıcıdan bir girdi satırı alır ve sonucu yazdırır. Fark etmiş olabileceğiniz gibi, bu örnek kullanıcı girdisini `System.in` ve `System.out` ile okuma örneğimize eşdeğerdir.

Obtaining Underlying I/O Streams

English: The Console class includes access to two streams for reading and writing data.

Türkçe: Console sınıfı, veri okuma ve yazma için iki streams erişim içerir.

```
public Reader reader()
public PrintWriter writer()
```

English: Accessing these classes is analogous to calling System.in and System.out directly, although they use character streams rather than byte streams. In this manner, they are more appropriate for handling text data.

Türkçe: Bu sınıflara erişmek, byte streams yerine character streams kullanmalarına rağmen, doğrudan System.in ve System.out çağrılarına benzer. Bu şekilde, metin verilerini işlemek için daha uygundurlar.

Formatting Console Data

English: In Chapter 4, you learned about the format() method on String; and in Chapter 11, “Exceptions and Localization,” you worked with formatting using locales. Conveniently, each print stream class includes a format() method, which includes an overloaded version that takes a Locale to combine both of these:

Türkçe: Bölüm 4'te, String'deki format () yöntemini öğrendiniz; ve Bölüm 11'de, yerelleri kullanarak biçimlendirme ile çalıştınız. Uygun bir şekilde, her baskı stream sınıfı, her ikisini de birleştirmek için bir Locale alan aşırı yüklenmiş bir sürüm içeren bir format() yöntemi içerir:

```
// PrintStream
public PrintStream format(String format, Object... args)
public PrintStream format(Locale loc, String format, Object... args)
// PrintWriter
public PrintWriter format(String format, Object... args)
public PrintWriter format(Locale loc, String format, Object... args)
```

English: For convenience (as well as to make C developers feel more at home), Java includes printf() methods, which function identically to the format() methods. The only thing you need to know about these methods is that they are interchangeable with format().

Türkçe: Kolaylık için (C geliştiricilerinin kendilerini evde daha fazla hissetmelerini sağlamanın yanı sıra), Java yöntemleri, format() yöntemleriyle aynı işlevi gören printf() yöntemlerini içerir. Bu yöntemler hakkında bilmeniz gereken tek şey format() ile değiştirilebilir olmasıdır.

English: Let's take a look at using multiple methods to print information for the user:

Türkçe: Kullanıcı için bilgi yazdırmak için birden fazla yöntem kullanarak bir göz atalım:

```
Console console = System.console();
if (console == null) {
    throw new RuntimeException("Console not available");
} else {
```

```
console.writer().println("Welcome to Our Zoo!");
console.format("It has %d animals and employs %d people", 391, 25);
console.writer().println();
console.printf("The zoo spans %5.1f acres", 128.91);
}
```

English: Assuming the Console is available at runtime, it prints the following:

Türkçe: Console'un çalışma zamanında mevcut olduğunu varsayarsak, aşağıdakileri yazdırır:

```
Welcome to Our Zoo!
It has 391 animals and employs 25 people
The zoo spans 128.9 acres.
```

English: Using Console with a Locale Unlike the print stream classes, Console does not include an overloaded format() method that takes a Locale instance. Instead, Console relies on the system locale. Of course, you could always use a specific Locale by retrieving the Writer object and passing your own Locale instance, such as in the following example:

Türkçe: Console ile Locale print stream sınıflarının aksine Console, Locale örneğini alan aşırı yüklenmiş bir format() yöntemi içermez. Bunun yerine, Console sistem lokaline dayanır. Tabii ki, Writer nesnesini geri alarak ve aşağıdaki örnekte olduğu gibi kendi Locale örneğini geçirerek her zaman belirli bir Locale kullanabilirsiniz:

```
Console console = System.console();
console.writer().format(new Locale("fr", "CA"), "Hello World");
```

Reading Console Data

English: The Console class includes four methods for retrieving regular text data from the user.

Türkçe: Console sınıfı, kullanıcıdan düzenli metin verilerini almak için dört yöntem içerir.

```
public String readLine()
public String readLine(String fmt, Object... args)
public char[] readPassword()
public char[] readPassword(String fmt, Object... args)
```

English: Like using System.in with a BufferedReader, the Console readLine() method reads input until the user presses the Enter key. The overloaded version of readLine() displays a formatted message prompt prior to requesting input.

Türkçe: Bir BufferedReader ile System.in kullanmak gibi, kullanıcı Enter tuşuna basana kadar Console readLine() yöntemi girdiyi okur. readLine() 'nin aşırı yüklenmiş sürümü, girdi istemeden önce biçimlendirilmiş bir ileti istemi görüntüler.

English: The readPassword() methods are similar to the readLine() method, with two important differences:

Türkçe: readPassword() yöntemleri iki önemli farkla readLine() yöntemine benzer:

English: The text the user types is not echoed back and displayed on the screen as they are typing.

Türkçe: Kullanıcı tiplerindeki metin geriye doğru yankılanmaz ve yazarken ekranda görüntülenir.

English: The data is returned as a `char[]` instead of a `String`.

Türkçe: Veriler `String` yerine `char[]` olarak döndürülür.

English: The first feature improves security by not showing the password on the screen if someone happens to be sitting next to you. The second feature involves preventing passwords from entering the `String` pool.

Türkçe: İlk özellik, yanınızda biri oturuyorsa ekranda şifreyi göstermeyerek güvenliği artırır. İkinci özellik, şifrelerin `String` havuzuna girmesini önlemeyi içerir.

Java 17 editör notu: `char[]` kullanımının temel avantajı `String` pool iddiası değil, password kullanıldıktan sonra array içeriğinin `Arrays.fill(password, '\0')` gibi bir işlemle temizlenebilmesidir.

Reviewing Console Methods

English: The last code sample we present asks the user a series of questions and prints results based on this information using many of various methods we learned in this section:

Türkçe: Sunduğumuz son kod örneği, kullanıcıya bir dizi soru sorar ve bu bölümde öğrendiğimiz çeşitli yöntemleri kullanarak bu bilgilere dayanarak sonuçları yazdırır:

```
Console console = System.console();
if (console == null) {
    throw new RuntimeException("Console not available");
} else {
    String name = console.readLine("Please enter your name: ");
    console.writer().format("Hi %s", name);
    console.writer().println();
    console.format("What is your address? ");
    String address = console.readLine();
    char[] password = console.readPassword("Enter a password "
        + "between %d and %d characters: ", 5, 10);
    char[] verify = console.readPassword("Enter the password again: ");
    console.printf("Passwords "
        + (Arrays.equals(password, verify)? "match": "do not match"));
}
```

English: Assuming the Console is available, the output should resemble the following:

Türkçe: Console kullanılabilir olduğunu varsayarsak, çıktı aşağıdakilere benzemelidir:

```
Please enter your name: Max
Hi Max
What is your address? Spoonerville
Enter a password between 5 and 10 characters:
Enter the password again:
Passwords match
```

Working with Advanced APIs

English: Files, paths, I/O streams: you've worked with a lot this chapter! In this final section, we cover some advanced features of I/O streams and NIO.2 that can be quite useful in practice— and have been known to appear on the exam from time to time!

Türkçe: Files, yollar, I/O streams: bu bölümde çok çalıştınız! Bu son bölümde, uygulamada oldukça yararlı olabilecek I/O streams ve NIO.2'nin bazı gelişmiş özelliklerini ele alıyoruz - ve zaman zaman sınavda görüldüğü biliniyor!

Manipulating Input Streams

English: All input stream classes include the following methods to manipulate the order in which data is read from an I/O stream:

Türkçe: Tüm input stream sınıfları, verilerin bir I/O stream 'den okunduğu sırayı manipüle etmek için aşağıdaki yöntemleri içerir:

```
// InputStream and Reader
public boolean markSupported()
public void mark(int readLimit)
public void reset() throws IOException
public long skip(long n) throws IOException
```

English: The `mark()` and `reset()` methods return an I/O stream to an earlier position. Before calling either of these methods, you should call the `markSupported()` method, which returns `true` only if `mark()` is supported. The `skip()` method is pretty simple; it basically reads data from the I/O stream and discards the contents.

Türkçe: `mark()` ve `reset()` I/O stream'i önceki bir konuma döndürür. Bu method'lardan birini çağırmadan önce, yalnızca `mark()` destekleniyorsa `true` döndüren `markSupported()` çağrılmalıdır. `skip()` ise I/O stream'den data okuyup content'i discard eder.

English: Not all input stream classes support `mark()` and `reset()`. Make sure to call `markSupported()` on the I/O stream before calling these methods, or an exception will be thrown at runtime.

Türkçe: Tüm input stream sınıfları `mark()` ve `reset()`'yi desteklemez. Bu yöntemleri çağırmadan önce I/O stream üzerinde `markSupported()`'ü aradığınızdan emin olun, aksi takdirde çalışma zamanında bir istisna atılacaktır.

Java 17 editör notu: Unsupported durumda base `InputStream.mark()` no-op olabilir; `reset()` ise `IOException` üretir. Exact behavior concrete stream contract'ına bağlıdır. Bu nedenle önce `markSupported()` kontrol edilmelidir.

Marking Data

English: Assume that we have an `InputStream` instance whose next values are LION. Consider the following code snippet:

Türkçe: Bir sonraki değerleri LION olan bir `InputStream` örneğimiz olduğunu varsayalım. Aşağıdaki kod snippet'ini düşünün:

```
public void readData(InputStream is) throws IOException {
    System.out.print((char) is.read()); // L
    if (is.markSupported()) {
        is.mark(100); // Marks up to 100 bytes
        System.out.print((char) is.read()); // I
        System.out.print((char) is.read()); // O
        is.reset(); // Resets stream to position before I
    }
    System.out.print((char) is.read()); // I
    System.out.print((char) is.read()); // O
    System.out.print((char) is.read()); // N
}
```

English: The code snippet will output LIOION if mark() is supported and LION otherwise. It's a good practice to organize your read() operations so that the I/O stream ends up at the same position regardless of whether mark() is supported.

Türkçe: mark() destekleniyorsa kod snippet'i LIOION ve aksi takdirde LION çıktısını alacaktır. read() işlemlerinizi düzenlemek için iyi bir uygulamadır, böylece I/O akışı, mark() desteklenip desteklenmediğine bakılmaksızın aynı konumda sona erer.

English: What about the value of 100 that we passed to the mark() method? This value is called the readLimit. It instructs the I/O stream that we expect to call reset() after at most 100 bytes. If our program calls reset() after reading more than 100 bytes from calling mark(100), it may throw an exception, depending on the I/O stream class.

Türkçe: mark() yöntemine aktardığımız 100 değeri ne olacak? Bu değere readLimit denir. En fazla 100 bytes sonra reset() çağırma beklediğimiz I/O stream komutunu verir. Programımız, mark(100) aramasından 100 bytes 'dan fazla okuduktan sonra reset() çağırırsa, I/O stream sınıfına bağlı olarak bir istisna atabilir.

English: In actuality, mark() and reset() are not putting the data back into the I/O stream but are storing the data in a temporary buffer in memory to be read again. Therefore, you should not call the mark() operation with too large a value, as this could take up a lot of memory.

Türkçe: Gerçekte, mark() ve reset() verileri I/O stream içine geri koymuyor, verileri tekrar okunacak bellekte geçici bir tamponda saklıyor. Bu nedenle, mark() işlemini çok büyük bir değerle aramamalısınız, çünkü bu çok fazla bellek alabilir.

Skipping Data

English: Assume that we have an InputStream instance whose next values are TIGERS. Consider the following code snippet:

Türkçe: Bir sonraki değerleri TIGERS olan bir InputStream örneğimiz olduğunu varsayalım. Aşağıdaki kod snippet'ini düşünün:

```
System.out.print((char)is.read()); // T
is.skip(2); // Skips I and G
is.read(); // Reads E but doesn't output it
System.out.print((char)is.read()); // R
System.out.print((char)is.read()); // S
```

English: This code prints TRS at runtime. We skipped two characters, I and G. We also read E but didn't use it anywhere, so it behaved like calling `skip(1)`.

Türkçe: Bu kod çalışma zamanında TRS yazdırır. İki karakteri atladık, ben ve G. E'yi de okuduk ama hiçbir yerde kullanmadık, bu yüzden `skip(1)` aramak gibi davrandı.

English: The return parameter of `skip()` tells us how many values were skipped. For example, if we are near the end of the I/O stream and call `skip(1000)`, the return value might be 20, indicating that the end of the I/O stream was reached after 20 values were skipped. Using the return value of `skip()` is important if you need to keep track of where you are in an I/O stream and how many bytes have been processed.

Türkçe: `skip()` dönüş parametresi kaç değerin atlandığını söyler. Örneğin, I/O stream'nin sonuna yakınsak ve `skip(1000)`'yi ararsak, dönüş değeri 20 olabilir, bu da I/O stream'nin sonuna 20 değer atlandıktan sonra ulaşıldığını gösterir. `skip()` geri dönüş değerini kullanmak, bir I/O stream içinde bulunduğunuz yeri ve kaç bytes işlendiğini takip etmeniz gerekiyorsa önemlidir.

Reviewing Manipulation APIs

English: TABLE 14.11 reviews these APIs related to manipulating I/O input streams. While you may not have used these in practice, you need to know them for the exam.

Türkçe: TABLE 14.11, I/O input streams ile ilgili bu API'leri gözden geçirir. Bunları pratikte kullanmamış olsanız da, sınav için bunları bilmeniz gerekir.

English: TABLE 14.11 — Common I/O stream methods

Method	Description
<code>public boolean markSupported()</code>	Returns <code>true</code> if the stream class supports <code>mark()</code>
<code>public void mark(int readLimit)</code>	Marks the current stream position
<code>public void reset()</code>	Attempts to reset to the marked position
<code>public long skip(long n)</code>	Reads and discards the specified number of values

Türkçe: TABLO 14.11 — Yaygın I/O stream method'ları. `markSupported()` capability'yi bildirir; `mark()` konumu kaydeder; `reset()` bu konuma dönmeyi dener; `skip()` actual skipped count'u döndürür.

Java 17 editör notu: Kaynak tabloda `mark(int readLimit)` satırının return type'ı basılmamıştır. Burada Java 17 signature'ı olan `public void mark(int readLimit)` kullanılarak açık OCR/kaynak dizgi hatası düzeltilmiştir.

Discovering File Attributes

English: We begin our discussion by presenting the basic methods for reading file attributes. These methods are usable within any file system, although they may have limited meaning in some file systems.

Türkçe: Önce file attribute'ları okumaya yarayan temel method'ları tanıtıyoruz. Bu method'lar bütün file system'lerde kullanılabilir; ancak bazı attribute'ların belirli file system'lerdeki anlamı sınırlı olabilir.

Checking for Symbolic Links

English: Earlier, we saw that the `Files` class has methods called `isDirectory()` and `isRegularFile()`, which are similar to the `isDirectory()` and `isFile()` methods on `File`. While the `File` object can't tell you if a reference is a symbolic link, the `isSymbolicLink()` method on `Files` can.

Türkçe: Daha önce `Files` class'ının, `File` üzerindeki `isDirectory()` ve `isFile()` method'larına benzeyen `isDirectory()` ve `isRegularFile()` method'larını gördük. `File` object'i bir reference'ın symbolic link olup olmadığını söyleyemez; `Files.isSymbolicLink()` bunu denetleyebilir.

English: It is possible for `isDirectory()` or `isRegularFile()` to return true for a symbolic link, as long as the link resolves to a directory or regular file, respectively. Let's take a look at some sample code:

Türkçe: Link sırasıyla directory veya regular file'a resolve oluyorsa `isDirectory()` ya da `isRegularFile()` symbolic link için `true` dönebilir.

```
System.out.print(Files.isDirectory(Paths.get("/canine/fur.jpg")));  
System.out.print(Files.isSymbolicLink(Paths.get("/canine/coyote")));  
System.out.print(Files.isRegularFile(Paths.get("/canine/types.txt")));
```

English: The first example prints true if `fur.jpg` is a directory or a symbolic link to a directory and false otherwise. The second example prints true if `/canine/coyote` is a symbolic link, regardless of whether the file or directory it points to exists. The third example prints true if `types.txt` points to a regular file or a symbolic link that points to a regular file.

Türkçe: İlk örnek `fur.jpg` directory ise veya directory'yi gösteren symbolic link ise `true`, aksi halde `false` yazdırır. İkinci örnek `/canine/coyote` symbolic link ise target'ının varlığından bağımsız olarak `true` yazdırır. Üçüncü örnek `types.txt` regular file ise ya da regular file'ı gösteren symbolic link ise `true` yazdırır.

Checking File Accessibility

English: In many file systems, it is possible to set a boolean attribute to a file that marks it hidden, readable, or executable. The `Files` class includes methods that expose this information: `isHidden()`, `isReadable()`, `isWritable()`, and `isExecutable()`.

Türkçe: Birçok file system'de bir file'ın hidden, readable, writable veya executable olduğunu belirten boolean attribute'lar ayarlanabilir. `Files` class'ı bu bilgileri gösteren `isHidden()`, `isReadable()`, `isWritable()` ve `isExecutable()` method'larını içerir.

English: A hidden file can't normally be viewed when listing the contents of a directory. The readable, writable, and executable flags are important in file systems where the filename can be viewed, but the user may not have permission to open the file's contents, modify the file, or run the file as a program, respectively.

Türkçe: Hidden file, bir directory'nin içeriği listelenirken normalde görünmez. Readable, writable ve executable flag'leri ise filename görünse bile kullanıcının sırasıyla file içeriğini açma, file'ı değiştirme veya program olarak çalıştırma izni bulunmayabilen file system'lerde önemlidir.

English: Here we present an example of each method:

Türkçe: Burada her yöntemin bir örneğini sunuyoruz:

```
System.out.print(Files.isHidden(Paths.get("/walrus.txt")));  
System.out.print(Files.isReadable(Paths.get("/seal/baby.png")));  
System.out.print(Files.isWritable(Paths.get("/dolphin.txt")));  
System.out.print(Files.isExecutable(Paths.get("/whale.png")));
```

English: If the walrus.txt file exists and is hidden within the file system, the first example prints true. The second example prints true if the baby.png file exists and its contents are readable. The third example prints true if the dolphin.txt file can be modified. Finally, the last example prints true if the file can be executed within the operating system. Note that the file extension does not necessarily determine whether a file is executable. For example, an image file that ends in .png could be marked executable in some file systems.

Türkçe: /walrus.txt varsa ve file system'de hidden ise ilk örnek true yazdırır. /seal/baby.png varsa ve içeriği okunabiliyorsa ikinci; dolphin.txt değiştirilebiliyorsa üçüncü örnek true yazdırır. whale.png operating system içinde çalıştırılabilirse son örnek true yazdırır. File extension tek başına file'in executable olup olmadığını belirlemez; örneğin .png ile biten bir image file bazı file system'lerde executable olarak işaretlenebilir.

English: With the exception of the isHidden() method, these methods do not declare any checked exceptions and return false if the file does not exist.

Türkçe: isHidden() dışında bu method'lar checked exception declare etmez; file yoksa false döndürür.

Improving Attribute Access

English: Up until now, we have been accessing individual file attributes with multiple method calls. While this is functionally correct, there is often a cost each time one of these methods is called. Put simply, it is far more efficient to ask the file system for all of the attributes at once rather than performing multiple round trips to the file system. Furthermore, some attributes are file system-specific and cannot be easily generalized for all file systems.

Türkçe: Şimdiye kadar individual file attribute'lara birden fazla method call ile erişildi. Bu işlevsel olarak doğru olsa da her call'ın bir maliyeti vardır. File system'e birden fazla gidiş-geliş yapmak yerine bütün attribute'ları tek seferde istemek daha verimlidir. Ayrıca bazı attribute'lar file-system-specific olduğu için bütün file system'lere kolayca genellenemez.

English: NIO.2 addresses both of these concerns by allowing you to construct views for various file systems with a single method call. A view is a group of related attributes for a particular file system type. That's not to say that the earlier attribute methods that we just finished discussing do not have their uses. If you need to read only one attribute of a file or directory, requesting a view is unnecessary.

Türkçe: NIO.2, tek method call ile farklı file system'ler için view oluşturarak iki sorunu da giderir. View, belirli file system type'ı için ilişkili attribute grubudur. Bu, önceki individual attribute method'larının kullanışsız olduğu anlamına gelmez. File veya directory'nin yalnız tek attribute'u okunacaksa view istemek gereksizdir.

Understanding Attribute and View Types

English: NIO.2 includes two methods for working with attributes in a single method call: a read-only attributes method and an updatable view method. For each method, you need to provide a file system type object, which tells the NIO.2 method which type of view you are requesting. By updatable view, we mean that we can both read and write attributes with the same object.

Türkçe: NIO.2, attribute'larla tek method call'da çalışmak için iki yaklaşım sunar: read-only attributes method'u ve updatable view method'u. Her ikisinde de istenen view type'ını NIO.2 method'una bildiren bir file-system type object'i verilir. Updatable view ile aynı object üzerinden attribute'lar hem okunabilir hem de yazılabilir.

English: TABLE 14.12 lists the commonly used attributes and view types. For the exam, you only need to know about the basic file attribute types. The other views are for managing operating system-specific information.

Türkçe: TABLE 14.12 yaygın attribute ve view type'larını listeler. Sınav için yalnızca basic file attribute type'larını bilmeniz gerekir; diğer view'lar operating-system-specific bilgileri yönetmek içindir.

English: TABLE 14.12 — The attributes and view types

Attributes interface	View interface	Description
<code>BasicFileAttributes</code>	<code>BasicFileAttributeView</code>	Basic attributes supported by all file systems
<code>DosFileAttributes</code>	<code>DosFileAttributeView</code>	Basic attributes plus DOS/Windows attributes
<code>PosixFileAttributes</code>	<code>PosixFileAttributeView</code>	Basic attributes plus POSIX attributes used by Unix, Linux, macOS, etc.

Türkçe: TABLO 14.12 — Attribute ve view type'ları

Attributes interface	View interface	Açıklama
<code>BasicFileAttributes</code>	<code>BasicFileAttributeView</code>	Bütün file system'lerdeki basic attribute'lar
<code>DosFileAttributes</code>	<code>DosFileAttributeView</code>	Basic + DOS/Windows attribute'ları
<code>PosixFileAttributes</code>	<code>PosixFileAttributeView</code>	Basic + POSIX/Unix/Linux/macOS attribute'ları

Retrieving Attributes

English: The Files class includes the following method to read attributes of a class in a readonly capacity:

Türkçe: `Files` class'ı attribute'ları read-only biçimde almak için aşağıdaki method'u içerir:

```
public static <A extends BasicFileAttributes> A readAttributes(
    Path path,
    Class<A> type,
    LinkOption... options) throws IOException
```

English: Applying it requires specifying the Path and BasicFileAttributes.class parameters.

Türkçe: Method çağrısında Path ve BasicFileAttributes.class parameter'ları belirtilir:

```
var path = Paths.get("/turtles/sea.txt");
BasicFileAttributes data = Files.readAttributes(path,
BasicFileAttributes.class);
System.out.println("Is a directory? " + data.isDirectory());
System.out.println("Is a regular file? " + data.isRegularFile());
System.out.println("Is a symbolic link? " + data.isSymbolicLink());
System.out.println("Size (in bytes): " + data.size());
System.out.println("Last modified: " + data.lastModifiedTime());
```

English: The BasicFileAttributes class includes many values with the same name as the attribute methods in the Files class. The advantage of using this method, though, is that all of the attributes are retrieved at once for some operating systems.

Türkçe: BasicFileAttributes, Files class'ındaki individual attribute method'larıyla aynı adları taşıyan birçok değer içerir. Bu yaklaşımın avantajı, bazı operating system'lerde bütün attribute'ların tek seferde alınmasıdır.

Modifying Attributes

English: The following Files method returns an updatable view:

Türkçe: Aşağıdaki Files method'u updatable view döndürür:

```
public static <V extends FileAttributeView> V getFileAttributeView(
Path path,
Class<V> type,
LinkOption... options)
```

English: We can use the updatable view to increment a file's last modified date/time value by 10,000 milliseconds, or 10 seconds.

Türkçe: Bir dosyanın son değiştirilmiş date/time değerini 10.000 milisaniye veya 10 saniye artırmak için updatable görünümünü kullanabiliriz.

```
// Read file attributes
var path = Paths.get("/turtles/sea.txt");
BasicFileAttributeView view = Files.getFileAttributeView(path,
BasicFileAttributeView.class);
BasicFileAttributes attributes = view.readAttributes();
// Modify file last modified time
FileTime lastModifiedTime = FileTime.fromMillis(
```

```
attributes.lastModifiedTime().toMillis() + 10_000);
view.setTimes(lastModifiedTime, null, null);
```

English: After the updatable view is retrieved, we need to call readAttributes() on the view to obtain the file metadata. From there, we create a new FileTime value and set it using the setTimes() method:

Türkçe: Updatable view alındıktan sonra file metadata'sını elde etmek için view üzerinde readAttributes() çağrılır. Ardından yeni bir FileTime value oluşturulur ve setTimes() ile atanır:

```
// BasicFileAttributeView instance method
public void setTimes(FileTime lastModifiedTime,
FileTime lastAccessTime, FileTime createTime)
```

English: This method allows us to pass null for any date/time value that we do not want to modify. In our sample code, only the last modified date/time is changed.

Türkçe: Bu yöntem, değiştirmek istemediğimiz herhangi bir date/time değeri için null değerini geçmemizi sağlar. Örnek kodumuzda, yalnızca son değiştirilmiş date/time değiştirilir.

English: Not all file attributes can be modified with a view. For example, you cannot set a property that changes a file into a directory. Likewise, you cannot change the size of the object without modifying its contents.

Türkçe: Bütün file attribute'ları view üzerinden değiştirilemez. Örneğin bir file'ı directory'ye dönüştüren bir property ayarlanamaz; aynı şekilde content değiştirilmeden object'in size değeri değiştirilemez.

Traversing a Directory Tree

English: While the `Files.list()` method is useful, it traverses the contents of only a single directory. What if we want to visit all of the paths within a directory tree? Before we proceed, we need to review some basic concepts about file systems. Remember that a directory is organized in a hierarchical manner. For example, a directory can contain files and other directories, which can in turn contain other files and directories. Every record in a file system has exactly one parent, with the exception of the root directory, which sits atop everything.

Türkçe: `Files.list()` kullanışlıdır, ancak yalnızca tek bir directory'nin içeriğini dolaşır. Bir directory tree içindeki bütün path'leri ziyaret etmek istersek file system'in hiyerarşik yapısını dikkate almalıyız. Directory; file ve başka directory'ler içerebilir, bunlar da kendi file ve directory'lerini içerebilir. Root directory dışında file system'deki her entry'nin tam bir parent'ı vardır.

English: A file system is commonly visualized as a tree with a single root node and many branches and leaves. In this model, a directory is a branch or internal node, and a file is a leaf node.

Türkçe: File system genellikle tek bir root node ile çok sayıda branch ve leaf içeren bir tree olarak gösterilir. Bu modelde directory branch/internal node, file ise leaf node'dur.

English: A common task in a file system is to iterate over the descendants of a path, either recording information about them or, more commonly, filtering them for a specific set of files. For example, you may want to search a folder and print a list of all of the `.java` files. Furthermore, file systems store file records in a hierarchical manner. Generally speaking, if you want to search for a file, you have to start with a parent directory, read its child elements, then read their children, and so on.

Türkçe: File system'deki yaygın görevlerden biri, bir path'in descendant'ları üzerinde iterate ederek bunlarla ilgili bilgi kaydetmek veya belirli file'ları filter etmektir. Örneğin bir folder'ı arayıp bütün `.java` file'larını listelemek isteyebilirsiniz. File entry'leri hiyerarşik tutulduğundan arama genellikle parent directory'den başlar; önce child element'ler, ardından onların child'ları okunur ve bu biçimde devam edilir.

English: Traversing a directory, also referred to as walking a directory tree, is the process by which you start with a parent directory and iterate over all of its descendants until some condition is met or there are no more elements over which to iterate. For example, if we're searching for a single file, we can end the search when the file is found or we've checked all files and come up empty. The starting path is usually a specific directory; after all, it would be timeconsuming to search the entire file system on every request!

Türkçe: Directory traversal, diğer adıyla directory tree walking, bir parent directory'den başlayıp bir koşul sağlanana veya iterate edilecek element kalmayana kadar bütün descendant'ları dolaşma işlemidir. Tek bir file aranıyorsa file bulunduğu ya da bütün adaylar sonuçsuz denetlendiğinde arama biter. Başlangıç path'i genellikle belirli bir directory'dir; her istekte bütün file system'i aramak zaman alır.

English: Don't Use DirectoryStream and FileVisitor While browsing the NIO.2 Javadocs, you may come across methods that use the DirectoryStream and FileVisitor classes to traverse a directory. These methods predate the existence of the Stream API and were even required knowledge for older Java certification exams.

Türkçe: NIO.2 Javadocs'a göz atarken DirectoryStream ve FileVisitor sınıflarını bir dizinden geçmek için kullanan yöntemlerle karşılaşabilirsiniz. Bu yöntemler Stream API'in varlığından önce gelir ve hatta daha eski Java sertifikasyon sınavları için gerekli bilgidir.

English: The best advice we can give you is to not use them. The newer Stream API-based methods are superior and accomplish the same thing, often with much less code.

Türkçe: Size verebileceğimiz en iyi tavsiye, onları kullanmamanızdır. Daha yeni Stream API'tabanlı yöntemler daha üstündür ve genellikle çok daha az kodla aynı şeyi gerçekleştirir.

Selecting a Search Strategy

English: Two common strategies are associated with walking a directory tree: a depth-first search and a breadth-first search. A depth-first search traverses the structure from the root to an arbitrary leaf and then navigates back up toward the root, traversing fully any paths it skipped along the way. The search depth is the distance from the root to current node. To prevent endless searching, Java includes a search depth that is used to limit how many levels (or hops) from the root the search is allowed to go.

Türkçe: Directory tree walking için iki yaygın strategy vardır: depth-first search ve breadth-first search. Depth-first search root'tan seçilen bir leaf'e kadar ilerler, sonra geri dönerek yol boyunca atlanan branch'leri tamamen dolaşır. Search depth, root ile current node arasındaki uzaklıktır. Java, sınırsız aramayı önlemek için root'tan kaç level/hop ilerlenebileceğini belirleyen bir depth limit kullanılmasına izin verir.

English: Alternatively, a breadth-first search starts at the root and processes all elements of each particular depth before proceeding to the next depth level. The results are ordered by depth, with all nodes at depth 1 read before all nodes at depth 2, and so on. While a breadth-first search tends to be balanced and predictable, it also requires more memory since a list of visited nodes must be maintained.

Türkçe: Breadth-first search ise root'tan başlar ve bir sonraki depth level'a geçmeden önce current depth'teki bütün element'leri işler. Sonuçlar depth'e göre sıralanır; depth 1'deki bütün node'lar depth 2'dekilerden önce okunur ve bu düzen sürer. Bu strategy dengeli ve öngörülebilirdir; ancak ziyaret edilen node'ların listesi tutulduğu için daha fazla memory gerektirir.

English: For the exam, you don't have to understand the details of each search strategy that Java employs; you just need to be aware that the NIO.2 Stream API methods use depth-first searching with a depth limit, which can be optionally changed.

Türkçe: Sınav için, Java'nın kullandığı her arama stratejisinin ayrıntılarını anlamak zorunda değilsiniz; sadece NIO.2 Stream API yöntemlerinin isteğe bağlı olarak değiştirilebilen bir derinlik sınırı olan derinlik-öncelikli arama kullandığını bilmeniz gerekir.

Walking a Directory

English: That's enough background information; let's get to more Stream API methods. The Files class includes two methods for walking the directory tree using a depth-first search.

Türkçe: Bu yeterli arka plan bilgisi; daha fazla Stream API yöntemine geçelim. Files sınıfı, derinlik-ilk arama kullanarak dizin ağacını yürümek için iki yöntem içerir.

```
public static Stream<Path> walk(Path start,
    FileVisitOption... options) throws IOException
public static Stream<Path> walk(Path start, int maxDepth,
    FileVisitOption... options) throws IOException
```

English: Like our other stream methods, walk() uses lazy evaluation and evaluates a Path only as it gets to it. This means that even if the directory tree includes hundreds or thousands of files, the memory required to process a directory tree is low. The first walk() method relies on a default maximum depth of Integer.MAX_VALUE, while the overloaded version allows the user to set a maximum depth. This is useful in cases where the file system might be large and we know the information we are looking for is near the root.

Türkçe: Diğer stream method'ları gibi walk() da lazy evaluation kullanır ve bir Path'i ancak o path'e ulaştığında değerlendirir. Directory tree yüzlerce veya binlerce file içerse bile processing için gereken memory düşüktür. İlk walk() overload'u default maximum depth olarak Integer.MAX_VALUE kullanır; diğer overload maximum depth'in kullanıcı tarafından belirlenmesini sağlar. Büyük file system'lerde aranan bilgi root'a yakınsa bu sınır kullanışlıdır.

English: Rather than just printing the contents of a directory tree, we can again do something more interesting. The following getPathSize() method walks a directory tree and returns the total size of all the files in the directory:

Türkçe: Bir dizin ağacının içeriğini yazdırmak yerine, yine daha ilginç bir şey yapabiliriz. Aşağıdaki getPathSize() yöntemi bir dizin ağacında yürür ve dizindeki tüm dosyaların toplam boyutunu döndürür:

```
private long getSize(Path p) {
    try {
        return Files.size(p);
    } catch (IOException e) {
        throw new UncheckedIOException(e);
    }
}

public long getPathSize(Path source) throws IOException {
    try (var s = Files.walk(source)) {
        return s.parallel()
            .filter(p -> !Files.isDirectory(p))
            .mapToLong(this::getSize)
            .sum();
    }
}
```

English: The `getSize()` helper method is needed because `Files.size()` declares `IOException`, and we'd rather not put a try /catch block inside a lambda expression. Instead, we wrap it in the unchecked exception class `UncheckedIOException`. We can print the data using the `format()` method:

Türkçe: `Files.size()` `IOException` declare ettiği ve lambda içine `try/catch` koymak istemediğimiz için `getSize()` helper method'u gerekir. Exception bunun yerine unchecked `UncheckedIOException` ile wrap edilir. Data `format()` ile yazdırılabilir:

```
var size = getPathSize(Path.of("/fox/data"));
System.out.format("Total Size: %.2f megabytes", (size/1000000.0));
```

English: Depending on the directory you run this on, it will print something like this:

Türkçe: Bunu çalıştırdığınız dizine bağlı olarak, şöyle bir şey yazdıracaktır:

```
Total Size: 15.30 megabytes
```

Applying a Depth Limit

English: Let's say our directory tree is quite deep, so we apply a depth limit by changing one line of code in our `getPathSize()` method.

Türkçe: Dizin ağacımızın oldukça derin olduğunu varsayalım, bu yüzden `getPathSize()` yöntemimizde bir kod satırı değiştirerek bir derinlik sınırı uyguluyoruz.

```
try (var s = Files.walk(source, 5)) {
```

English: This new version checks for files only within 5 steps of the starting node. A depth value of 0 indicates the current path itself. Since the method calculates values only on files, you'd have to set a depth limit of at least 1 to get a nonzero result when this method is applied to a directory tree.

Türkçe: Bu yeni sürüm, dosyaları yalnızca başlangıç düğümünden 5 adım içinde kontrol eder. 0'ın derinlik değeri, mevcut yolun kendisini gösterir. Yöntem yalnızca dosyalardaki değerleri hesapladığından, bu yöntem bir dizin ağacına uygulandığında sıfır olmayan bir sonuç elde etmek için en az 1 derinlik sınırı belirlemeniz gerekir.

Avoiding Circular Paths

English: Many of our earlier NIO.2 methods traverse symbolic links by default, with a `NOFOLLOW_LINKS` used to disable this behavior. The `walk()` method is different in that it does not follow symbolic links by default and requires the `FOLLOW_LINKS` option to be

Türkçe: Önceki NIO.2 method'larının birçoğu symbolic link'leri varsayılan olarak takip eder; bu davranış `NOFOLLOW_LINKS` ile kapatılabilir. `walk()` ise symbolic link'leri varsayılan olarak takip etmez ve bunun için `FOLLOW_LINKS` option'ının

English: enabled. We can alter our `getPathSize()` method to enable following symbolic links by adding the `FileVisitOption`:

Türkçe: etkinleştirilmesi gerekir. Symbolic link takibini açmak için `getPathSize()` method'una `FileVisitOption` ekleyebiliriz:

```
try (var s = Files.walk(source,
    FileVisitOption.FOLLOW_LINKS)) {
```

English: When traversing a directory tree, your program needs to be careful of symbolic links, if enabled. For example, if our process comes across a symbolic link that points to the root directory of the file system, every file in the system will be searched!

Türkçe: Directory tree dolaşılırken symbolic link takibi etkinse dikkatli olunmalıdır. Örneğin process, file system root'unu gösteren symbolic link ile karşılaşırsa sistemdeki her file aranır.

English: Worse yet, a symbolic link could lead to a cycle in which a path is visited repeatedly. A cycle is an infinite circular dependency in which an entry in a directory tree points to one of its ancestor directories. Let's say we had a directory tree as shown in FIGURE 14.7 with the symbolic link `/birds/robin/allBirds` that points to `/birds`.

Türkçe: Symbolic link, path'in tekrar tekrar ziyaret edildiği cycle'a yol açabilir. Cycle, directory tree'deki bir entry'nin ancestor directory'lerinden birini gösterdiği sonsuz circular dependency'dir. FIGURE 14.7'deki `/birds/robin/allBirds` path'inin `/birds` symbolic link olduğunu varsayalım.

English: FIGURE 14.7 — File system with cycle

```
birds\
├─ robin\                □— start
├─ pictures\
│   └─ nest.png
│   └─ wings.gif
└─ allBirds\ —symbolic link—□ /birds
```

Türkçe: **ŞEKİL 14.7 — Cycle içeren file system.** `/birds/robin/allBirds`, ancestor `/birds` directory'sine symbolic link'tir.

English: What happens if we try to traverse this tree and follow all symbolic links, starting with `/birds/robin`? TABLE 14.13 shows the paths visited after walking a depth of 3. For simplicity, we walk the tree in a breadth-first ordering, although a cycle occurs regardless of the search strategy used.

Türkçe: `/birds/robin`den başlayıp bütün symbolic link'leri izlersek ne olur? TABLE 14.13, maximum depth 3 ile ziyaret edilen path'leri gösterir. Kaynak, kolaylık amacıyla breadth-first order kullanır; ancak search strategy ne olursa olsun cycle oluşur.

English: **TABLE 14.13 — Walking a directory with a cycle using breadth-first search**

Depth	Path reached
0	<code>/birds/robin</code>
1	<code>/birds/robin/pictures</code>
1	<code>/birds/robin/allBirds</code> → <code>/birds</code>
2	<code>/birds/robin/pictures/nest.png</code>
2	<code>/birds/robin/pictures/wings.gif</code>

Türkçe: **TABLO 14.13 — Cycle içeren directory'nin kaynakta breadth-first varsayımıyla yürünmesi.**
Root depth 0'dır; symbolic link'in çözüldüğü target okla gösterilir.

Java 17 editör notu: Kaynak tablo cycle'ı açıklamak için açıkça breadth-first ordering varsayar. Bunu `Files.walk()` için genel ordering contract'ı saymayın; Java 17 API, directory tree'nin **depth-first** yüründüğünü belirtir.

English: TABLE 14.13 — Continued

Depth	Path reached
2	<code>/birds/robin/allBirds/robin</code> → <code>/birds/robin</code>
3	<code>/birds/robin/allBirds/robin/pictures</code> → <code>/birds/robin/pictures</code>
3	<code>/birds/robin/allBirds/robin/pictures/allBirds</code> → <code>/birds/robin/allBirds</code> → <code>/birds</code>

Türkçe: TABLO 14.13 — Devam. Depth 2'de daha önce ziyaret edilen `/birds/robin`e yeniden ulaşılır; cycle burada ortaya çıkar.

English: □ After walking a distance of 1 from the start, we hit the symbolic link `/birds/robin/allBirds` and go back to the top of the directory tree `/birds`. That's okay because we haven't visited `/birds` yet, so there's no cycle yet!

Türkçe: Start'tan depth 1 ilerleyince `/birds/robin/allBirds` symbolic link'ine ulaşır ve directory tree'nin root'u `/birds`e döneriz. `/birds` henüz ziyaret edilmediğinden bu noktada cycle yoktur.

English: Unfortunately, at depth 2, we encounter a cycle. We've already visited the `/birds/robin` directory on our first step, and now we're encountering it again. If the process continues, we'll be doomed to visit the directory over and over again.

Türkçe: Depth 2'de cycle oluşur. İlk adımda `/birds/robin` directory'si zaten ziyaret edilmiştir ve şimdi yeniden karşılaşılr. Process sürerse aynı directory tekrar tekrar ziyaret edilir.

English: Be aware that when the `FOLLOW_LINKS` option is used, the `walk()` method will track all of the paths it has visited, throwing a `FileSystemLoopException` if a path is visited twice.

Türkçe: `FOLLOW_LINKS` kullanıldığında `walk()`, ziyaret ettiği bütün path'leri izler ve aynı path ikinci kez ziyaret edilirse `FileSystemLoopException` fırlatır.

Searching a Directory

English: In the previous example, we applied a filter to the `Stream<Path>` object to filter the results, although there is a more convenient method.

Türkçe: Önceki örnekte sonuçları filter etmek için `Stream<Path>` object'ine `filter()` uyguladık; ancak bunun için daha kullanışlı bir method vardır.

```
public static Stream<Path> find(Path start,
int maxDepth,
BiPredicate<Path, BasicFileAttributes> matcher,
FileVisitOption... options) throws IOException
```

English: The `find()` method behaves in a similar manner as the `walk()` method, except that it takes a `BiPredicate` to filter the data. It also requires a depth limit to be set. Like `walk()`, `find()` also supports the `FOLLOW_LINK` option.

Türkçe: `find()`, data'yı filter etmek için `BiPredicate` alması dışında `walk()` a benzer davranır. Ayrıca bir depth limit verilmesini gerektirir. `walk()` gibi `find()` da kaynak metinde `FOLLOW_LINK` adıyla anılan link-following option'ını destekler.

Java 17 editör notu: Kaynak prose'daki `FOLLOW_LINK` yazımı tekildir. Gerçek Java 17 enum constant'ı `FileVisitOption.FOLLOW_LINK` tir.

English: The two parameters of the `BiPredicate` are a `Path` object and a `BasicFileAttributes` object, which you saw earlier in the chapter. In this manner, Java automatically retrieves the basic file information for you, allowing you to write complex lambda expressions that have direct access to this object. We illustrate this with the following example:

Türkçe: `BiPredicate` in iki parameter'ı daha önce gördüğümüz bir `Path` ve bir `BasicFileAttributes` object'idir. Java basic file bilgisini otomatik olarak alır; böylece lambda expression içinde bu attribute object'ine doğrudan erişebilirsiniz. Aşağıdaki örnek bunu gösterir:

```
Path path = Paths.get("/bigcats");
long minSize = 1_000;
try (var s = Files.find(path, 10,
(p, a) -> a.isRegularFile()
```

```
&& p.toString().endsWith(".java")
&& a.size() > minSize)) {
s.forEach(System.out::println);
}
```

English: This example searches a directory tree and prints all `.java` files with a size of at least 1,000 bytes, using a depth limit of 10. While we could have accomplished this using the `walk()` method along with a call to `readAttributes()`, this implementation is a lot shorter and more convenient than those would have been. We also don't have to worry about any methods within the lambda expression declaring a checked exception, as we saw in the `getPathSize()` example.

Türkçe: Bu örnek directory tree'yi depth limit 10 ile arar ve en az 1.000 byte boyutundaki bütün `.java` file'larını yazdırır. Aynı işlem `walk()` ile `readAttributes()` birlikte kullanılarak yapılabilse de bu implementation daha kısa ve kullanışlıdır. Ayrıca `getPathSize()` örneğindeki gibi lambda içindeki method'ların checked exception declare etmesiyle uğraşmak gerekmez.

Review of Key APIs

English: The key APIs that you need to know for the exam are listed in TABLE 14.14. We know some of the classes look similar. You need to know this table really well before taking the exam.

Türkçe: Sınav için bilmeniz gereken anahtar API'ler TABLE 14.14'te listelenmiştir. Bazı sınıfların benzer göründüğünü biliyoruz. Sınava girmeden önce bu masayı çok iyi bilmeniz gerekir.

English: TABLE 14.14 — Key APIs

Class	Purpose
<code>File</code>	I/O representation of a location in a file system
<code>Files</code>	Helper methods for working with <code>Path</code>
<code>Path</code>	NIO.2 representation of a location in a file system
<code>Paths</code>	Factory methods for obtaining <code>Path</code>
<code>URI</code>	Uniform resource identifier for files, URLs, etc.
<code>FileSystem</code>	NIO.2 representation of a file system
<code>FileSystems</code>	Factory methods for obtaining <code>FileSystem</code>
<code>InputStream</code>	Superclass for reading byte-based files
<code>OutputStream</code>	Superclass for writing byte-based files
<code>Reader</code>	Superclass for reading character-based files
<code>Writer</code>	Superclass for writing character-based files

Türkçe: TABLO 14.14 — Temel API'ler. `File` legacy I/O, `Path/Files` NIO.2 location ve operation modelini; dört abstract stream class'ı da byte/character ile input/output eksenlerini temsil eder.

English: Additionally, FIGURE 14.8 shows all of the I/O stream classes that you should be familiar with for the exam, with the exception of the filter streams. `FilterInputStream` and `FilterOutputStream` are high-level superclasses that filter or transform data. They are rarely used directly.

Türkçe: Ek olarak, FIGURE 14.8, streams filtresi hariç, sınav için aşina olmanız gereken I/O stream sınıflarının tümünü gösterir. `FilterInputStream` ve `FilterOutputStream`, verileri filtreleyen veya dönüştüren üst düzey süper sınıflardır. Nadiren doğrudan kullanılırlar.

English: FIGURE 14.8 — Diagram of I/O stream classes

```

InputStream (abstract)
├── FileInputStream                [low-level]
├── FilterInputStream             [high-level superclass]
│   └── BufferedInputStream       [high-level]
└── ObjectInputStream            [high-level]

Reader (abstract)
├── BufferedReader               [high-level]
├── InputStreamReader            [low-level bridge]
└── FileReader                  [low-level]

OutputStream (abstract)
├── FileOutputStream            [low-level]
├── FilterOutputStream         [high-level superclass]
│   ├── BufferedOutputStream    [high-level]
│   └── PrintStream             [high-level]
└── ObjectOutputStream         [high-level]

Writer (abstract)
├── BufferedWriter              [high-level]
├── OutputStreamWriter          [low-level bridge]
│   └── FileWriter              [low-level]
└── PrintWriter                 [high-level]
  
```

Türkçe: ŞEKİL 14.8 — I/O stream class'ları. Dört abstract base class'ın altında low-level source/sink stream'leri ile başka stream'leri wrap eden high-level stream'ler gösterilir. `InputStreamReader` ve `OutputStreamWriter` byte–character bridge'leridir.

English: The `InputStreamReader` and `OutputStreamWriter` are incredibly convenient and are also unique in that they are the only I/O stream classes to have both `InputStream` /`OutputStream` and `Reader` /`Writer` in their name.

Türkçe: `InputStreamReader` ve `OutputStreamWriter` kullanışlı ve benzersizdir: Adlarında hem `InputStream`/`OutputStream` hem de `Reader`/`Writer` geçen tek I/O stream class'larıdır.

Summary

English: This chapter is all about reading and writing data. We started by showing you how to create File from I/O and Path from NIO.2. We then covered the functionality that works with both I/O and NIO.2 before getting into NIO.2-specific APIs. You should be familiar with how to combine or resolve Path objects with other Path objects. Additionally, NIO.2 includes Stream API methods that can be used to process files and directories. We discussed methods for listing a directory, walking a directory tree, searching a directory tree, and reading the lines of a file.

Türkçe: Bu bölüm data okuma ve yazma üzerinedir. Önce legacy I/O API'sinde File, NIO.2'de ise Path oluşturmayı gösterdik. Ardından NIO.2-specific API'lere geçmeden önce I/O ve NIO.2'nin ortak functionality'sini ele aldık. Path object'lerini başka Path object'leriyle combine etme ve resolve() etme davranışını bilmelisiniz. NIO.2, file ve directory'leri işlemek için Stream API method'ları da sunar. Directory listeleme, directory tree walking/searching ve file line'larını okuma method'larını inceledik.

English: We spent time reviewing various methods available in the Files helper class. As discussed, the name of the function often tells you exactly what it does. We explained that most of these methods are capable of throwing an IOException, and many take optional varargs enum values.

Türkçe: Files helper class'ındaki çeşitli method'ları gözden geçirdik. Method adı çoğu zaman ne yaptığını doğrudan açıklar. Bu method'ların çoğu IOException fırlatabilir ve birçoğu isteğe bağlı varargs enum value'ları alır.

English: We then introduced I/O streams and explained how they are used to read or write large quantities of data. While there are a lot of I/O streams, they differ on some key points:

Türkçe: Ardından I/O stream'leri tanıtip büyük miktarda data okumak veya yazmak için nasıl kullanıldıklarını açıkladık. Çok sayıda I/O stream olsa da temel ayırım noktaları şunlardır:

English: Byte vs.character streams Input vs.output streams Low-level vs.high-level streams Often, the name of the I/O stream can tell you a lot about what it does. We visited many of the I/O stream classes that you will need to know for the exam in increasing order of complexity. A common practice is to start with a low-level resource or file stream and wrap it in a buffered I/O stream to improve performance. You can also apply a high-level stream to manipulate the data, such as an object or print stream. We described what it means to be serializable in Java, and we showed you how to use the object stream classes to persist objects directly to and from disk.

Türkçe: Byte stream–character stream, input–output ve low-level–high-level ayrımları önemlidir. I/O stream adı çoğu zaman ne yaptığını açıklar. Sınav için gereken class'lar, artan complexity sırasıyla incelendi. Yaygın pattern, low-level resource veya file stream ile başlayıp performance için buffered I/O stream ile wrap etmektir. Object stream ya da print stream gibi high-level stream'ler data'yı işlemek için eklenebilir. Ayrıca Java serialization kavramı ve object stream class'larıyla object'leri disk'e persist etme ve disk'ten geri okuma anlatıldı.

English: We explained how to read input data from the user using both the system stream objects and the Console class. The Console class has many useful features, such as built-in support for passwords and formatting.

Türkçe: Kullanıcı input'unun hem system stream object'leri hem de Console class'ı ile nasıl okunacağını açıkladık. Console, password okuma ve formatting için built-in support gibi yararlı özellikler sunar.

English: We also discussed how NIO.2 provides methods for reading and writing file metadata. NIO.2 includes two methods for retrieving all of the file system attributes for a path in a single call without numerous round trips to the operating system. One method requires a read-only attribute type, while the second method requires an updatable view type. It also allows NIO.2 to support operating system–specific file attributes.

Türkçe: NIO.2'nin file metadata'sını okumak ve yazmak için sunduğu method'ları da ele aldık. NIO.2, operating system'e çok sayıda round trip yapmadan bir path'in bütün file-system attribute'larını tek call'da almak için iki yaklaşım içerir. Biri read-only attribute type, diğeri updatable view type gerektirir. Bu yapı NIO.2'nin operating-system-specific file attribute'larını da desteklemesini sağlar.

Exam Essentials

English: Understand files and directories. Files are records that store data within a persistent storage device, such as a hard disk drive, that is available after the application has finished executing. Files are organized within a file system in directories, which in turn may contain other directories. The root directory is the topmost directory in a file system.

Türkçe: File ve directory'leri anlayın. File, hard disk gibi kalıcı storage device üzerinde data tutan ve application sona erdikten sonra da varlığını sürdüren bir record'dur. File'lar file system içinde directory'lerde düzenlenir; directory'ler başka directory'ler de içerebilir. Root directory, file system'deki en üst directory'dir.

English: Be able to use File and Path. An I/O File instance is created by calling the constructor. It contains a number of instance methods for creating and manipulating a file or directory. An NIO.2 Path instance is an immutable object that is commonly created from the factory method Paths.get() or Path.of(). It can also be created from FileSystem, java.net.URI, or java.io.File instances. The Path interface includes many instance methods for reading and manipulating the abstract path value.

Türkçe: File ve Path kullanabilin. I/O File instance'ı constructor çağrısıyla oluşturulur ve file/directory oluşturup yönetmeye yarayan instance method'lar içerir. NIO.2 Path, çoğunlukla Paths.get() veya Path.of() factory method'larıyla oluşturulan immutable bir object'tir; FileSystem, java.net.URI veya java.io.File instance'larından da elde edilebilir. Path interface'i abstract path value'yu okumak ve değiştirilmiş yeni değerler üretmek için birçok instance method içerir.

English: Distinguish between types of I/O streams. I/O streams are categorized by byte/character, input/output, and low-level/high-level. Byte streams operate on binary data and have names that end with Stream, while character streams operate on text data and have names that end in Reader or Writer. The InputStream and Reader classes are the topmost abstract classes that receive data, while the OutputStream and Writer classes are the topmost abstract classes that send data. A low-level stream is one that operates directly on the underlying resource, such as a file or network connection. A high-level stream operates on a lowlevel or other high-level stream to filter data, convert data, or improve performance.

Türkçe: I/O stream türlerini ayırt edin. I/O stream'ler byte/character, input/output ve low-level/high-level eksenlerinde sınıflandırılır. Byte stream'ler binary data üzerinde çalışır ve adları Stream ile; character stream'ler text data üzerinde çalışır ve adları Reader veya Writer ile biter. InputStream ile Reader data alan, OutputStream ile Writer data gönderen en üst abstract class'lardır. Low-level stream file veya network connection gibi alttaki resource'a doğrudan bağlanır. High-level stream ise data'yı filter/convert etmek ya da performance'ı artırmak için low-level veya başka bir high-level stream üzerinde çalışır.

English: Understand how to use Java serialization. A class is considered serializable if it implements the `java.io.Serializable` interface and contains instance members that are either serializable or marked `transient`. All Java primitives and the `String` class are serializable. The `ObjectInputStream` and `ObjectOutputStream` classes can be used to read and write a `Serializable` object from and to an I/O stream, respectively.

Türkçe: Java serialization kullanımını anlayın. Bir class, `java.io.Serializable` interface'ini implement ediyor ve her instance member'ı serializable ya da `transient` ise class serializable kabul edilir. Bütün Java primitive'leri ve `String` serializable'dır. `ObjectInputStream` ve `ObjectOutputStream`, `Serializable` object'i sırasıyla bir I/O stream'den okumak ve stream'e yazmak için kullanılır.

English: Be able to interact with the user. Be able to interact with the user using the system streams (`System.out`, `System.err`, and `System.in`) as well as the `Console` class. The `Console` class includes special methods for formatting data and retrieving complex input such as passwords.

Türkçe: Kullanıcıyla etkileşim kurabilin. `System.out`, `System.err` ve `System.in` system stream'leriyle ve `Console` class'ıyla kullanıcı etkileşimini bilin. `Console`, data formatting ve password gibi karmaşık input'ları alma amacıyla özel method'lar içerir.

English: Manage file attributes. The `NIO.2 Files` class includes many methods for reading single file attributes, such as its size or whether it is a directory, a symbolic link, hidden, etc. `NIO.2` also supports reading all of the attributes in a single call. An attribute type is used to support operating system-specific views. Finally, `NIO.2` supports updatable views for modifying selected attributes.

Türkçe: File attribute'ları yönetebilin. `NIO.2 Files` class'ı size, directory olma, symbolic link olma veya hidden olma gibi individual attribute'ları okumak için birçok method içerir. `NIO.2` bütün attribute'ları tek call'da okumayı da destekler. Attribute type'ları operating-system-specific view'lar için kullanılır; updatable view'lar ise seçili attribute'ların değiştirilmesini sağlar.

Review Questions

English: The answers to the chapter review questions can be found in the Appendix.

Türkçe: Bölüm inceleme sorularının cevapları Ek'te bulunabilir.

English: 1. Which class would be best to use to read a binary file into a Java object?

Türkçe: 1. Binary bir dosyayı okuyup Java nesnesine dönüştürmek için en uygun sınıf hangisidir?

English: A. `BufferedStream`

Türkçe: A. `BufferedStream`

English: B. `FileReader`

Türkçe: B. `FileReader`

English: C. `ObjectInputStream`

Türkçe: C. `ObjectInputStream`

English: D. `ObjectReader`

Türkçe: D. `ObjectReader`

English: E. `ObjectOutputStream`

Türkçe: E. `ObjectOutputStream`

English: F. `ObjectWriter`

Türkçe: F. `ObjectWriter`

English: G. None of the above

Türkçe: G. Yukarıdakilerin hiçbiri

English: 2. Assuming that `/` is the root directory within the file system, which of the following are true statements? (Choose all that apply.)

Türkçe: 2. `/` işaretinin file system içindeki root directory olduğunu varsayarsak, aşağıdaki ifadelerden hangileri doğrudur? (Uygun olanların tümünü seçin.)

English: A. `/home/parrot` is an absolute path.

Türkçe: A. `/home/parrot` bir absolute path'tir.

English: B. `/home/parrot` is a directory.

Türkçe: B. `/home/parrot` bir directory'dir.

English: C. `/home/parrot` is a relative path.

Türkçe: C. `/home/parrot` bir relative path'tir.

English: D. `new File("/home")` will throw an exception if `/home` does not exist.

Türkçe: D. `/home` yoksa `new File("/home")` bir exception fırlatır.

English: E. `new File("/home").delete()` will throw an exception if `/home` does not exist.

Türkçe: E. `/home` yoksa `new File("/home").delete()` bir exception fırlatır.

English: F. A `Reader` offers character encoding, making it more useful when working with `String` data than an `InputStream`.

Türkçe: F. Bir `Reader`, character encoding sunduğu için `String` verileriyle çalışırken `InputStream`'den daha kullanışlıdır.

English: G. A `Reader` offers multithreading support, making it more useful than an `InputStream`.

Türkçe: G. Bir `Reader`, multithreading desteği sunduğu için `InputStream`'den daha kullanışlıdır.

English: 3. What are possible results of executing the following code? (Choose all that apply.)

Türkçe: 3. Aşağıdaki kodu çalıştırmamanın olası sonuçları nelerdir? (Tüm geçerli olanları seçin.)

```
public static void main(String[] args) throws IOException {
    String line;
    var c = System.console();
    Writer w = c.writer();
    try (w) {
        if ((line = c.readLine("Enter your name: ")) != null)
            w.append(line);
        w.flush();
    }
}
```

English: A. The code runs, but nothing is printed.

Türkçe: A. Kod çalışıyor, ama hiçbir şey basılmıyor.

English: B. The code prints what was entered by the user.

Türkçe: B. Kod, kullanıcı tarafından girilenleri yazdırır.

English: C. The code behaves the same if throws IOException is removed.

Türkçe: C. `throws IOException` kaldırılırsa kod aynı şekilde davranır.

English: D. A NullPointerException may be thrown.

Türkçe: D. Bir NullPointerException atılabilir.

English: E. A NullPointerException will always be thrown.

Türkçe: E. Her zaman bir NullPointerException atılacaktır.

English: F. A NullPointerException will never be thrown.

Türkçe: F. Bir NullPointerException asla atılmaz.

English: G. The code does not compile.

Türkçe: G. Kod derlenmiyor.

English: 4. For which values of path sent to this method would it be possible for the following code to output Success? (Choose all that apply.)

Türkçe: 4. Bu method'a gönderilen `path` hangi değerleri aldığında kodun `Success` yazdırması mümkün olur? (Uygun olanların tümünü seçin.)

```
public void removeBadFile(Path path) {
    if(Files.isDirectory(path))
        System.out.println(Files.deleteIfExists(path)
            ? "Success": "Try Again");
}
```

English: A. path refers to a regular file in the file system.

Türkçe: A. `path`, file system içindeki regular file'ı gösterir.

English: B. path refers to a symbolic link in the file system.
Türkçe: B. `path`, file system içindeki bir symbolic link'i gösterir.

English: C. path refers to an empty directory in the file system.
Türkçe: C. `path`, file system içindeki empty directory'yi gösterir.

English: D. path refers to a directory with content in the file system.
Türkçe: D. `path`, file system içinde içeriği bulunan bir directory'yi gösterir.

English: E. path does not refer to a record that exists within the file system.
Türkçe: E. `path`, file system içinde var olan hiçbir entry'yi göstermez.

English: F. The code does not compile.
Türkçe: F. Kod derlenmiyor.

English: 5. Assume that the directory `/animals` exists and is empty. What is the result of executing the following code?
Türkçe: 5. `/animals` directory'sinin var ve boş olduğunu varsayın. Aşağıdaki kod çalıştırıldığında ne olur?

```
Path path = Path.of("/animals");
try (var z = Files.walk(path)) {
    boolean b = z
        .filter((p,a) -> a.isDirectory() &&!path.equals(p)) // x
        .findFirst().isPresent(); // y
    System.out.print(b? "No Sub": "Has Sub");
}
```

English: A. It prints No Sub.
Türkçe: A. `No Sub` yazdırılır.

English: B. It prints Has Sub.
Türkçe: B. `Has Sub` yazdırılır.

English: C. The code will not compile because of line x.
Türkçe: C. Kod, `x` satırı nedeniyle derlenmez.

English: D. The code will not compile because of line y.
Türkçe: D. Kod, `y` satırı nedeniyle derlenmez.

English: E. The output cannot be determined.
Türkçe: E. Çıktısı belirlenemiyor.

English: F. It produces an infinite loop at runtime.

Türkçe: F. Çalışma zamanında sonsuz bir döngü üretir.

English: 6. What would be the value of name if the instance of Eagle created in the main() method were serialized and then deserialized?

Türkçe: 6. `main()` method'unda oluşturulan `Eagle` instance'ı serialize edilip ardından deserialize edilirse `name` değerinin sonucu ne olur?

```
import java.io.Serializable;
class Bird {
    protected transient String name;
```

```
    public void setName(String name) { this.name = name; }
    public String getName() { return name; }
    public Bird() {
        this.name = "Matt";
    }
}

public class Eagle extends Bird implements Serializable {
    { this.name = "Olivia"; }
    public Eagle() {
        this.name = "Bridget";
    }
    public static void main(String[] args) {
        var e = new Eagle();
        e.name = "Adeline";
    }
}
```

English: A. Adeline

Türkçe: A. Adeline

English: B. Bridget

Türkçe: B. Bridget

English: C. Matt

Türkçe: C. Matt

English: D. Olivia

Türkçe: D. Olivia

English: E. null

Türkçe: E. null

English: F. The code does not compile.

Türkçe: F. Kod derlenmiyor.

English: G. The code compiles but throws an exception at runtime.
Türkçe: G. Kod derlenir ancak çalışma zamanında bir istisna atar.

English: 7. Assume that `/kang` exists as a symbolic link to the directory `/mammal/kangaroo` within the file system. Which of the following statements are correct about this code snippet?

Türkçe: 7. `/kang` path'inin file system içindeki `/mammal/kangaroo` directory'sine symbolic link olduğunu varsayın. Bu kod parçasıyla ilgili ifadelerden hangileri doğrudur?

English: (Choose all that apply.)
Türkçe: (Tüm geçerli olanları seçin.)

```
var path = Paths.get("/kang");  
if(Files.isDirectory(path) && Files.isSymbolicLink(path))  
Files.createDirectory(path.resolve("joey"));
```

English: A. A new directory will always be created.
Türkçe: A. Her zaman yeni bir dizin oluşturulacaktır.

English: B. A new directory may be created.
Türkçe: B. Yeni bir dizin oluşturulabilir.

English: C. If the code creates a directory, it will be reachable at `/kang/joey`.
Türkçe: C. Kod bir directory oluşturursa bu directory'ye `/kang/joey` üzerinden erişilebilir.

English: D. If the code creates a directory, it will be reachable at `/mammal/joey`.
Türkçe: D. Kod bir directory oluşturursa bu directory'ye `/mammal/joey` üzerinden erişilebilir.

English: E. The code does not compile.
Türkçe: E. Kod derlenmiyor.

English: F. The code will compile but will always throw an exception at runtime.
Türkçe: F. Kod derlenecek ancak çalışma zamanında her zaman bir istisna atacaktır.

English: 8. Assuming that the `/fox/food-schedule.csv` file exists with the specified contents, what is the expected output of calling `printData()` on it?
Türkçe: 8. `/fox/food-schedule.csv` file'ının belirtilen içerikle var olduğunu varsayarsak, bu file için `printData()` çağrısının beklenen çıktısı nedir?

```
/fox/food-schedule.csv
6am,Breakfast
9am,SecondBreakfast
12pm,Lunch
6pm,Dinner
void printData(Path path) throws IOException {
Files.readAllLines(path) // r1
.flatMap(p -> Stream.of(p.split(","))) // r2
.map(q -> q.toUpperCase()) // r3
.forEach(System.out::println);
}
```

English: A. The code will not compile because of line r1.

Türkçe: A. Kod, **r1** satırı nedeniyle derlenmez.

English: B. The code will not compile because of line r2.

Türkçe: B. Kod, **r2** satırı nedeniyle derlenmez.

English: C. The code will not compile because of line r3.

Türkçe: C. Kod, **r3** satırı nedeniyle derlenmez.

English: D. It throws an exception at runtime.

Türkçe: D. Çalışma zamanında bir istisna atar.

English: E. It does not print anything at runtime.

Türkçe: E. Çalışma zamanında hiçbir şey yazdırmaz.

English: F. None of the above

Türkçe: F. Yukarıdakilerin hiçbirisi

English: 9. Given the following method, which statements are correct? (Choose all that apply.)

Türkçe: 9. Aşağıdaki method için hangi ifadeler doğrudur? (Uygun olanların tümünü seçin.)

```
public void copyFile(File file1, File file2) throws Exception {
var reader = new InputStreamReader(new FileInputStream(file1));
try (var writer = new FileWriter(file2)) {
char[] buffer = new char[10];
while(reader.read(buffer) != -1) {
writer.write(buffer);
// n1
}
}
}
```

English: A. The code does not compile because reader is not a buffered stream.

Türkçe: A. **reader** bir buffered stream olmadığı için kod derlenmez.

English: B. The code does not compile because writer is not a buffered stream.

Türkçe: B. `writer` bir buffered stream olmadığı için kod derlenmez.

English: C. The code compiles and correctly copies the data between some files.

Türkçe: C. Kod derlenir ve bazı file'lar arasında veriyi doğru biçimde kopyalar.

English: D. The code compiles and correctly copies the data between all files.

Türkçe: D. Kod derlenir ve bütün file'lar arasında veriyi doğru biçimde kopyalar.

English: E. If we check file2 on line n1 within the file system after five iterations of the while loop, it may be empty.

Türkçe: E. `while` loop'unun beş iteration'ından sonra file system içindeki `file2`, `n1` satırında kontrol edilirse boş olabilir.

English: F. If we check file2 on line n1 within the file system after five iterations, it will contain exactly 50 characters.

Türkçe: F. Beş iteration'dan sonra file system içindeki `file2`, `n1` satırında kontrol edilirse tam olarak 50 character içerir.

English: G. This method contains a resource leak.

Türkçe: G. Bu yöntem bir kaynak sızıntısı içerir.

English: 10. Which of the following correctly create Path instances? (Choose all that apply.)

Türkçe: 10. Aşağıdakilerden hangileri `Path` instance'ını doğru biçimde oluşturur? (Uygun olanların tümünü seçin.)

English: A. `new Path("jaguar.txt")`

Türkçe: A. `new Path("jaguar.txt")`

English: B. `FileSystems.getDefault().getPath("puma.txt")`

Türkçe: B. `FileSystems.getDefault().getPath("puma.txt")`

English: C. `Path.get("cats","lynx.txt")`

Türkçe: C. `Path.get("cats","lynx.txt")`

English: D. `new java.io.File("tiger.txt").toPath()`

Türkçe: D. `new java.io.File("tiger.txt").toPath()`

English: E. `new FileSystem().getPath("lion")`

Türkçe: E. `new FileSystem().getPath("lion")`

English: F. `Paths.getPath("ocelot.txt")`

Türkçe: F. `Paths.getPath("ocelot.txt")`

English: G. Path.of(Path.of(".").toUri())

Türkçe: G. Path.of(Path.of(".").toUri())

English: 11. Which classes will allow the following to compile? (Choose all that apply.)

Türkçe: 11. Hangi sınıflar aşağıdakilerin derlenmesine izin verecek? (Tüm geçerli olanları seçin.)

```
var is = new BufferedInputStream(new FileInputStream("z.txt"));
InputStream wrapper = new(is);
try (wrapper) {}
```

English: A. BufferedInputStream

Türkçe: A. BufferedInputStream

English: B. BufferedReader

Türkçe: B. BufferedReader

English: C. BufferedWriter

Türkçe: C. BufferedWriter

English: D. FileInputStream

Türkçe: D. FileInputStream

English: E. ObjectInputStream

Türkçe: E. ObjectInputStream

English: F. ObjectOutputStream

Türkçe: F. ObjectOutputStream

English: G. None of the above, as the first line does not compile

Türkçe: G. İlk satır derlenmediği için yukarıdakilerin hiçbirisi

English: 12. What is the result of executing the following code? (Choose all that apply.)

Türkçe: 12. Aşağıdaki kodun çalıştırılmasının sonucu nedir? (Tüm geçerli olanları seçin.)

```
var p = Paths.get("sloth.schedule");
var a = Files.readAttributes(p, BasicFileAttributes.class);
Files.mkdir(p.resolve(".backup"));
if(a.size()>0 && a.isDirectory()) {
    a.setTimes(null,null,null);
}
```

English: A. It compiles and runs without issue.

Türkçe: A. Kod derlenir ve sorunsuz çalışır.

English: B. The code will not compile because of line 5.
Türkçe: B. Kod, 5. satır nedeniyle derlenmeyecek.

English: C. The code will not compile because of line 6.
Türkçe: C. Kod, 6. satır nedeniyle derlenmez.

English: D. The code will not compile because of line 7.
Türkçe: D. Kod, 7. satır nedeniyle derlenmeyecek.

English: E. The code will not compile because of line 8.
Türkçe: E. Kod, 8. satır nedeniyle derlenmeyecek.

English: F. None of the above
Türkçe: F. Yukarıdakilerin hiçbirisi

English: 13. Which of the following are true statements about serialization in Java? (Choose all that apply.)
Türkçe: 13. Aşağıdakilerden hangileri Java serialization hakkında doğru ifadelerdir? (Uygun olanların tümünü seçin.)

English: A. All non-null instance members of the class must be serializable or marked transient.
Türkçe: A. Class'ın `null` olmayan bütün instance member'ları serializable olmalı veya `transient` olarak işaretlenmelidir.

English: B. Records are automatically serializable.
Türkçe: B. Record'lar otomatik olarak serializable'dır.

English: C. Serialization involves converting data into Java objects.
Türkçe: C. Serialization, veriyi Java object'lerine dönüştürmeyi içerir.

English: D. Serializable is a functional interface.
Türkçe: D. `Serializable` bir functional interface'tir.

English: E. The class must declare a static serialVersionUID variable.
Türkçe: E. Class, static bir `serialVersionUID` variable'ı declare etmelidir.

English: F. The class must extend the Serializable class.
Türkçe: F. Class, `Serializable` class'ını extend etmelidir.

English: G. The class must implement the Serializable interface.
Türkçe: G. Class, `Serializable` interface'ini implement etmelidir.

English: 14. What is the output of the following code? (Choose three.)

Türkçe: 14. Aşağıdaki kodun çıktısı nedir? (Üç seçeneği işaretleyin.)

```
22: var p1 = Path.of("/zoo/./bear", "../food.txt");
23: p1.normalize().relativize(Path.of("/lion"));
24: System.out.println(p1);
25:
26: var p2 = Paths.get("/zoo/animals/bear/koala/food.txt");
27: System.out.println(p2.subpath(1,3).getName(1));
28:
29: var p3 = Path.of("/pets/./cat.txt");
30: var p4 = Paths.get("./dog.txt");
31: System.out.println(p4.resolve(p3));
```

English: A. .././lion

Türkçe: A. .././lion

English: B. /zoo/./bear/./food.txt

Türkçe: B. /zoo/./bear/./food.txt

English: C. animal

Türkçe: C. animal

English: D. bear

Türkçe: D. bear

English: E. /pets/./cat.txt

Türkçe: E. /pets/./cat.txt

English: F. /pets/./cat.txt/./dog.txt

Türkçe: F. /pets/./cat.txt/./dog.txt

English: 15. Suppose that the working directory is /weather and the absolute path /weather/winter/snow.dat represents a file that exists within the file system. Which of the following lines of code create an object that represents the file? (Choose all that apply.)

Türkçe: 15. Working directory'nin /weather olduğunu ve absolute path /weather/winter/snow.dat değerinin file system içinde var olan bir file'ı temsil ettiğini varsayalım. Aşağıdaki code line'larından hangileri bu file'ı temsil eden bir object oluşturur? (Uygun olanların tümünü seçin.)

English: A. new File("/weather", "winter", "snow.dat")

Türkçe: A. new File("/weather", "winter", "snow.dat")

English: B. new File("/weather/winter/snow.dat")

Türkçe: B. new File("/weather/winter/snow.dat")

English: C. `new File("/weather/winter", new File("snow.dat"))`

Türkçe: C. `new File("/weather/winter", new File("snow.dat"))`

English: D. `new File("weather", "/winter/snow.dat")`

Türkçe: D. `new File("weather", "/winter/snow.dat")`

English: E. `new File(new File("/weather/winter"), "snow.dat")`

Türkçe: E. `new File(new File("/weather/winter"), "snow.dat")`

English: F. `Path.of("/weather/winer/snow.dat").toFile();`

Türkçe: F. `Path.of("/weather/winer/snow.dat").toFile();`

English: G. None of the above

Türkçe: G. Yukarıdakilerin hiçbiri

Java 17 editör notu: F seçeneğindeki `winer` yazımı kaynakta bu şekildedir; sessizce `winter` olarak düzeltilmemiştir. Bu literal kod var olan `/weather/winter/snow.dat` file'ını değil başka bir path'i temsil eder. Bu nedenle kaynak Appendix anahtarı **B, E, F** dese de, basılı seçenekler Java 17'e göre değerlendirildiğinde savunulabilir cevap **B, E**'dir.

English: 16. Assuming `zoo-data.txt` exists and is not empty, what statements about the following method are correct? (Choose all that apply.)

Türkçe: 16. `zoo-data.txt` file'ının var ve boş olmadığını varsayarsak, aşağıdaki method hakkında hangi ifadeler doğrudur? (Uygun olanların tümünü seçin.)

```
private void echo() throws IOException {
    var o = new FileWriter("new-zoo.txt");
    try (var f = new FileReader("zoo-data.txt"); o) {
        var b = new BufferedReader(f);
        o.write(b.readLine());
    }
    o.write("");
}
```

English: A. When run, the method creates a new file with one line of text in it.

Türkçe: A. Çalıştırıldığında, yöntem içinde bir satır metin bulunan yeni bir dosya oluşturur.

English: B. When run, the method creates a new file with two lines of text in it.

Türkçe: B. Çalıştırıldığında, yöntem içinde iki satırlık metin bulunan yeni bir dosya oluşturur.

English: C. When run, the method creates a new file with the same number of lines as the original file.

Türkçe: C. Çalıştırıldığında, yöntem orijinal dosyayla aynı sayıda satır içeren yeni bir dosya oluşturur.

English: D. The method compiles but will produce an exception at runtime.

Türkçe: D. Yöntem derlenir, ancak çalışma zamanında bir istisna üretecektir.

English: E. The method does not compile.

Türkçe: E. Method derlenmez.

English: F. The method uses byte stream classes.

Türkçe: F. Method, byte stream class'larını kullanır.

English: 17. Which are true statements? (Choose all that apply.)

Türkçe: 17. Hangileri doğru ifadelerdir? (Tüm geçerli olanları seçin.)

English: A. NIO.2 includes a method to delete an entire directory tree.

Türkçe: A. NIO.2 tüm dizin ağacını silmek için bir yöntem içerir.

English: B. NIO.2 includes a method to traverse a directory tree.

Türkçe: B. NIO.2 bir dizin ağacını geçmek için bir yöntem içerir.

English: C. NIO.2 includes methods that are aware of symbolic links.

Türkçe: C. NIO.2, symbolic link'lerin farkında olan method'lar içerir.

English: D. Files.readAttributes() cannot access file-system dependent attributes.

Türkçe: D. Files.readAttributes() dosya sistemine bağımlı özniteliklere erişemez.

English: E. Files.readAttributes() is often more performant since it reads multiple attributes rather than accessing individual attributes.

Türkçe: E. `Files.readAttributes()`, ayrı ayrı attribute'lara erişmek yerine birden fazla attribute okuduğu için genellikle daha yüksek performance sağlar.

English: F. Files.readAttributes() works with the File object.

Türkçe: F. `Files.readAttributes()`, `File` object'ile çalışır.

English: 18. Assume that reader is a valid stream whose next characters are PEACOCKS. What is true about the output of the following code snippet? (Choose all that apply.)

Türkçe: 18. `reader`ın sıradaki character'ları `PEACOCKS` olan geçerli bir stream olduğunu varsayın.

Aşağıdaki kod parçasının çıktısı hakkında hangileri doğrudur? (Uygun olanların tümünü seçin.)

```
var sb = new StringBuilder();
sb.append((char) reader.read());
reader.mark(10);
for(int i=0; i<2; i++) {
    sb.append((char) reader.read());
    reader.skip(2);
}
reader.reset();
reader.skip(0);
```

```
sb.append((char) reader.read());
System.out.println(sb.toString());
```

English: A. The code may print PEAE.

Türkçe: A. Kod **PEAE** yazdırabilir.

English: B. The code may print PEOA.

Türkçe: B. Kod **PEOA** yazdırabilir.

English: C. The code may print PEOE.

Türkçe: C. Kod **PEOE** yazdırabilir.

English: D. The code may print PEOS.

Türkçe: D. Kod **PEOS** yazdırabilir.

English: E. The code will always print PEAE.

Türkçe: E. Kod her zaman **PEAE** yazdırır.

English: F. The code will always print PEOA.

Türkçe: F. Kod her zaman **PEOA** yazdırır.

English: G. The code will always print PEOE.

Türkçe: G. Kod her zaman **PEOE** yazdırır.

English: H. The code will always print PEOS.

Türkçe: H. Kod her zaman **PEOS** yazdırır.

English: 19. Assuming that the directories and files referenced exist and are not symbolic links, what is the result of executing the following code?

Türkçe: 19. Referans verilen directory ve file'ların var olduğunu ve symbolic link olmadığını varsayarsak, aşağıdaki kodun sonucu nedir?

```
var p1 = Path.of("/lizard",".").resolve(Path.of("walking.txt"));
var p2 = new File("/lizard/./actions/./walking.txt").toPath();
System.out.print(Files.isSameFile(p1,p2));
System.out.print(" ");
System.out.print(p1.equals(p2));
System.out.print(" ");
System.out.print(Files.mismatch(p1,p2));
```

English: A. true true -1

Türkçe: A. **true true -1**

English: B. true true 0

Türkçe: B. **true true 0**

English: C. true false -1

Türkçe: C. `true false -1`

English: D. true false 0

Türkçe: D. `true false 0`

English: E. false true -1

Türkçe: E. `false true -1`

English: F. false true 0

Türkçe: F. `false true 0`

English: G. The code does not compile.

Türkçe: G. Kod derlenmiyor.

English: H. The result cannot be determined.

Türkçe: H. Sonuç belirlenemez.

English: 20. Assume that `monkey.txt` is a file that exists in the current working directory. Which statements about the following code snippet are correct? (Choose all that apply.)

Türkçe: 20. `monkey.txt` file'inin current working directory'de var olduğunu varsayalım. Aşağıdaki kod parçasıyla ilgili hangi ifadeler doğrudur? (Uygun olanların tümünü seçin.)

```
Files.move(Path.of("monkey.txt"), Paths.get("/animals"),
StandardCopyOption.ATOMIC_MOVE,
LinkOption.NOFOLLOW_LINKS);
```

English: A. If `/animals/monkey.txt` exists, it will be overwritten at runtime.

Türkçe: A. `/animals/monkey.txt` varsa çalışma zamanında üzerine yazılır.

English: B. If `/animals` exists as an empty directory, `/animals/monkey.txt` will be the new location of the file.

Türkçe: B. `/animals` empty directory olarak varsa file'in yeni konumu `/animals/monkey.txt` olur.

English: C. If `monkey.txt` is a symbolic link, the file it points to will be moved at runtime.

Türkçe: C. `monkey.txt` bir symbolic link ise işaret ettiği file çalışma zamanında taşınır.

English: D. If the move is successful and another process is monitoring the file system, it will not see an incomplete file at runtime.

Türkçe: D. Move başarılı olur ve başka bir process file system'i izlerse çalışma zamanında incomplete bir file görmez.

English: E. None of the above

Türkçe: E. Yukarıdakilerin hiçbirisi

English: 21. Assume that /monkeys exists as a directory containing multiple files, symbolic links, and subdirectories. Which statement about the following code is correct?

Türkçe: 21. /monkeys path'inin birden fazla file, symbolic link ve subdirectory içeren bir directory olduğunu varsayın. Aşağıdaki kodla ilgili hangi ifade doğrudur?

```
var f = Path.of("/monkeys");
try (var m =
Files.find(f, 0, (p,a) -> a.isSymbolicLink())) { // y1
m.map(s -> s.toString())
.collect(Collectors.toList())
.stream()
.filter(s -> s.toString().endsWith(".txt")) // y2
.forEach(System.out::println);
}
```

English: A. It will print all symbolic links in the directory tree ending in.txt.

Türkçe: A. Directory tree'de .txt ile biten bütün symbolic link'leri yazdırır.

English: B. It will print the target of all symbolic links in the directory ending in.txt.

Türkçe: B. Directory'de .txt ile biten bütün symbolic link'lerin target'ını yazdırır.

English: C. It will print nothing.

Türkçe: C. Hiçbir şey basmayacak.

English: D. It does not compile because of line y1.

Türkçe: D. Kod, y1 satırı nedeniyle derlenmez.

English: E. It does not compile because of line y2.

Türkçe: E. Kod, y2 satırı nedeniyle derlenmez.

English: F. It compiles but throws an exception at runtime.

Türkçe: F. Kod derlenir ancak çalışma zamanında bir exception fırlatır.

English: 22. Which of the following fields will be null after an instance of the class created on line 17 is serialized and then deserialized using ObjectOutputStream and ObjectInputStream? (Choose all that apply.)

Türkçe: 22. 17. satırda oluşturulan class'ın instance'ı ObjectOutputStream ile serialize, ardından ObjectInputStream ile deserialize edildikten sonra aşağıdaki field'lardan hangileri null olur? (Uygun olanların tümünü seçin.)

```
import java.io.Serializable;
import java.util.List;
public class Zebra implements Serializable {
    private transient String name = "George";
    private static String birthPlace = "Africa";
    private transient Integer age;
    List<Zebra> friends = new java.util.ArrayList<>();
    private Object stripes = new Object();
    { age = 10;}
    public Zebra() {
        this.name = "Sophia";
    }
    static Zebra writeAndRead(Zebra z) {
```

```
// Implementation omitted
}
public static void main(String[] args) {
    var zebra = new Zebra();
    zebra = writeAndRead(zebra);
}
```

English: A. age

Türkçe: A. *age*

English: B. birthplace

Türkçe: B. *birthplace*

English: C. friends

Türkçe: C. *friends*

English: D. name

Türkçe: D. *name*

English: E. stripes

Türkçe: E. *stripes*

English: F. The code does not compile.

Türkçe: F. Kod derlenmiyor.

English: G. The code compiles but throws an exception at runtime.

Türkçe: G. Kod derlenir ancak çalışma zamanında bir istisna atar.

English: 23. What are some possible results of executing the following code? (Choose all that apply.)

Türkçe: 23. Aşağıdaki kodu çalıştırmanın bazı olası sonuçları nelerdir? (Tüm geçerli olanları seçin.)

```
var x = Path.of("/animals/fluffy/..");
Files.walk(x.toRealPath().getParent()) // u1
.map(p -> p.toAbsolutePath().toString()) // u2
.filter(s -> s.endsWith(".java"))
.forEach(System.out::println);
```

English: A. It prints some files in the root directory.

Türkçe: A. Kök dizinindeki bazı dosyaları yazdırır.

English: B. It prints all files in the root directory.

Türkçe: B. Kök dizinindeki tüm dosyaları yazdırır.

English: C. FileSystemLoopException is thrown at runtime.

Türkçe: C. FileSystemLoopException çalışma zamanında atılır.

English: D. Another exception is thrown at runtime.

Türkçe: D. Başka bir istisna da çalışma zamanında atılır.

English: E. The code will not compile because of line u1.

Türkçe: E. Kod, **u1** satırı nedeniyle derlenmez.

English: F. The code will not compile because of line u2.

Türkçe: F. Kod, **u2** satırı nedeniyle derlenmez.

English: 24. Assume that the source instance passed to the following method represents a file that exists.

Türkçe: 24. Aşağıdaki method'a geçirilen **source** instance'ının var olan bir file'ı temsil ettiğini varsayalım.

English: Also assume that /flip/sounds.txt exists as a file prior to executing this method. When this method is executed, which statement correctly copies the file to the path specified by /flip/sounds.txt?

Türkçe: Ayrıca method çalıştırılmadan önce /flip/sounds.txt path'inde bir file bulunduğunu varsayın. Method çalıştırıldığında hangi statement file'ı doğru biçimde /flip/sounds.txt path'ine kopyalar?

```
void copyIntoFlipDirectory(Path source) throws IOException {
    var dolphinDir = Path.of("/flip");
    dolphinDir = Files.createDirectories(dolphinDir);
    var n = Paths.get("sounds.txt");
    /* INSERT CODE HERE */
}
```

English: A. Files.copy(source, dolphinDir)

Türkçe: A. **Files.copy(source, dolphinDir)**

English: B. Files.copy(source, dolphinDir.resolve(n), StandardCopyOption.REPLACE_EXISTING)

Türkçe: B. **Files.copy(source, dolphinDir.resolve(n), StandardCopyOption.REPLACE_EXISTING)**

English: C. `Files.copy(source, dolphinDir, StandardCopyOption.REPLACE_EXISTING)`

Türkçe: C. `Files.copy(source, dolphinDir, StandardCopyOption.REPLACE_EXISTING)`

English: D. `Files.copy(source, dolphinDir.resolve(n))`

Türkçe: D. `Files.copy(source, dolphinDir.resolve(n))`

English: E. The method does not compile, regardless of what is placed in the blank.

Türkçe: E. Yöntem, boşluğa ne yerleştirildiğinden bağımsız olarak derlemez.

English: F. The method compiles but throws an exception at runtime, regardless of what is placed in the blank.

Türkçe: F. Boşluğa ne yazılırsa yazılsın method derlenir ancak çalışma zamanında exception fırlatır.

English: 25. Suppose that you need to read text data from a file and want the data to be performant on large files. Which two java.io stream classes can be chained together to best achieve this result? (Choose two.)

Türkçe: 25. Bir file'dan text data okumanız ve büyük file'larda yüksek performance elde etmeniz gerekiyor. Bu sonucu en iyi sağlayacak hangi iki `java.io` stream class'ı birbirine zincirlenebilir? (İki seçeneği işaretleyin.)

English: A. `BufferedInputStream`

Türkçe: A. `BufferedInputStream`

English: B. `BufferedReader`

Türkçe: B. `BufferedReader`

English: C. `FileInputStream`

Türkçe: C. `FileInputStream`

English: D. `FileReader`

Türkçe: D. `FileReader`

English: E. `PrintInputStream`

Türkçe: E. `PrintInputStream`

English: F. `ObjectInputStream`

Türkçe: F. `ObjectInputStream`

English: G. `PrintReader`

Türkçe: G. `PrintReader`

Appendix · Official Review Question Answers / Resmî Cevaplar

Aşağıdaki cevaplar kaynak Appendix bölümündeki sıra ve gerekçeleri korur. Türkçe bloklar doğal teknik çeviridir.

Official Answer 1

English: 1. C. Since the question asks about putting data into a structured object, the best class would be one that deserializes the data. Therefore, `ObjectInputStream` is the best choice, which is option C. `ObjectWriter`, `BufferedStream`, and `ObjectReader` are not I/O stream classes. `ObjectOutputStream` is an I/O class but is used to serialize data, not deserialize it. `FileReader` can be used to read text file data and construct an object, but the question asks what would be the best class to use for binary data.

Türkçe: 1. C. Soru, data'yı structured object'e yerleştirmeyi istediği için en uygun class deserialize işlemi yapan class'tır; bu nedenle C'deki `ObjectInputStream` en iyi seçimdir. `ObjectWriter`, `BufferedStream` ve `ObjectReader` I/O stream class'ları değildir. `ObjectOutputStream` bir I/O class'ıdır fakat deserialize değil serialize işlemi yapar. `FileReader` text file okuyup object oluşturmada kullanılabilir; ancak soru binary data için en uygun class'ı sorar.

Official Answer 2

English: 2. A, F. Paths that begin with the root directory are absolute paths, so option A is correct, and option C is incorrect. Option B is incorrect because the path could be a file or directory within the file system. There is no rule that files have to end with a file extension. Option D is incorrect, as it is possible to create a File reference to files and directories that do not exist.

Türkçe: 2. A, F. Root directory ile başlayan path'ler absolute olduğundan A doğru, C yanlıştır. Bir path file system içinde file veya directory gösterebildiği için B yanlıştır; file adının extension ile bitme zorunluluğu yoktur. Var olmayan file ve directory için de `File` reference'ı oluşturulabildiğinden D yanlıştır.

English: Option E is also incorrect. The `delete()` method returns false if the file or directory cannot be deleted. Character stream classes often include built-in convenience methods for working with String data, so option F is correct. There is no such optimization for multi-threading, making option G incorrect.

Türkçe: E de yanlıştır; `delete()` file veya directory silinemezse `false` döndürür. Character stream class'ları çoğunlukla `String` data için convenience method'lar içerdiğinden F doğrudur. Multithreading için böyle bir optimization bulunmadığı için G yanlıştır.

Official Answer 3

English: 3. B, D. If the console is unavailable, `System.console()` will return null, making option D correct and options E and F incorrect. The writer methods throw a checked `IOException`, making option C incorrect. The code works correctly, prompting for input and printing it. Therefore, option A is incorrect and option B is correct.

Türkçe: 3. B, D. Console kullanılamıyorsa `System.console()` null döndürür; D doğru, E ve F yanlıştır. `Writer` method'ları checked `IOException` fırlatabildiğinden C yanlıştır. Console mevcutsa kod input ister ve girilen değeri yazdırır; A yanlış, B doğrudur.

Official Answer 4

English: 4. F. The code does not compile, as `Files.deleteIfExists()` declares the checked `IOException` that must be handled or declared. Remember, most `Files` methods declare `IOException`, especially the ones that modify a file or directory. For this reason, option F is correct. If the method were corrected to declare the appropriate exceptions, option C would be correct. Option B would also be correct if the method were provided a symbolic link that pointed to an empty directory. Options A and E would not print anything, as `Files.isDirectory()` returns false for both. Finally, option D would throw a `DirectoryNotEmptyException` at runtime.

Türkçe: 4. F. `Files.deleteIfExists()` handle veya declare edilmesi gereken checked `IOException` bildirdiğinden kod derlenmez; F doğrudur. Özellikle file veya directory değiştiren pek çok `Files` method'u `IOException` declare eder. Method uygun exception'ı declare etseydi C doğru olurdu. Empty directory'yi gösteren symbolic link verilirse B de doğru olurdu. `Files.isDirectory()` A ve E için false döndürdüğünden çıktı oluşmaz. D ise çalışma zamanında `DirectoryNotEmptyException` fırlatır.

Official Answer 5

English: 5. C. The `filter()` operation applied to a `Stream<Path>` takes only one parameter, not two, so the code does not compile, and option C is correct. If the code were rewritten to use the `Files.find()` method with the `BiPredicate` as input (along with a `maxDepth` value), the output would be option B, `Has Sub`, since the directory is given to be empty. For fun, we reversed the expected output of the ternary operation.

Türkçe: 5. C. `Stream<Path>` üzerindeki `filter()` operation'ı iki değil tek parameter alır; kod derlenmez ve C doğrudur. Kod, `maxDepth` ile birlikte `BiPredicate` alan `Files.find()` kullanacak biçimde yazılıysaydı directory boş olduğundan B'deki `Has Sub` yazdırılırdı. Ternary operation'ın beklenen output'u özellikle ters çevrilmiştir.

Official Answer 6

English: 6. C. The code compiles and runs without issue, so options F and G are incorrect. The key here is that while `Eagle` is serializable, its parent class, `Bird`, is not. Therefore, none of the members of `Bird` will be serialized. Even if you didn't know that, you should know what happens on deserialization. During deserialization, Java calls the constructor of the first non-serializable parent. In this case, the `Bird` constructor is called, with `name` being set to `Matt`, making option C correct. Note that none of the constructors or instance initializers in `Eagle` are executed as part of deserialization.

Türkçe: 6. C. Kod sorunsuz derlenip çalıştığı için F ve G yanlıştır. `Eagle` serializable olsa da superclass'ı `Bird` değildir; bu nedenle `Bird` member'ları serialize edilmez. Deserialization sırasında Java ilk non-serializable superclass'ın constructor'ını çağırır. Burada `Bird` constructor'ı çalışıp `name`i `Matt` yapar; C doğrudur. `Eagle` içindeki constructor ve instance initializer'lar deserialization kapsamında çalışmaz.

Official Answer 7

English: 7. B, C. The code snippet will attempt to create a directory if the target of the symbolic link exists and is a directory. If the directory already exists, though, it will throw an exception.

Türkçe: 7. B, C. Symbolic link target'ı var ve directory ise kod yeni bir directory oluşturmaya çalışır. Target directory zaten varsa exception fırlatılır.

English: For this reason, option A is incorrect, and option B is correct. It will be created in `/mammal/kangaroo/joey` and also reachable at `/kang/joey` because of the symbolic link, making option C correct.

Türkçe: Bu nedenle A yanlış, B doğrudur. Directory `/mammal/kangaroo/joey` konumunda oluşturulur ve symbolic link nedeniyle `/kang/joey` üzerinden de erişilebilir; C de doğrudur.

Official Answer 8

English: 8. B. The `readAllLines()` method returns a `List`, not a `Stream`. Therefore, the call to `flatMap()` is invalid, and option B is correct. If the `Files.lines()` method were used instead, it would print the contents of the file one capitalized word at a time with the commas removed.

Türkçe: 8. B. `readAllLines()` `Stream` değil `List` döndürür; dolayısıyla `flatMap()` çağrısı geçersizdir ve B doğrudur. Bunun yerine `Files.lines()` kullanılsaydı comma'lar kaldırılarak file content'i her satırda bir uppercase word olacak şekilde yazdırılırdı.

Official Answer 9

English: 9. C, E, G. First, the method does compile, so options A and B are incorrect. Methods to read/write `byte[]` values exist in the abstract parent of all I/O stream classes. This implementation is not correct, though, as the return value of `read(buffer)` is not used properly.

Türkçe: 9. C, E, G. Method derlendiği için A ve B yanlıştır. `byte[]` okuyan ve yazan method'lar bütün I/O stream class'larının abstract parent'ında bulunur. Bununla birlikte `read(buffer)` return value'su doğru kullanılmadığından implementation hatalıdır.

English: It will only correctly copy files whose character count is a multiple of 10, making option C correct and option D incorrect. Option E is also correct as the data may not have made it to disk yet. Option F would be correct if the `flush()` method were called after every write.

Türkçe: Yalnız character sayısı 10'un katı olan file'ları doğru kopyalar; C doğru, D yanlıştır. Data henüz disk'e ulaşmamış olabileceğinden E de doğrudur. Her `write()` sonrasında `flush()` çağrılıysaydı F doğru olurdu.

English: Finally, option G is correct as the reader stream is never closed.

Türkçe: Son olarak `reader` stream hiç kapatılmadığından G doğrudur.

Official Answer 10

English: 10. B, D, G. Options A and E are incorrect because `Path` and `FileSystem`, respectively, are abstract types that should be instantiated using a factory method. Option C is incorrect because the static method in the `Path` interface is `of()`, not `get()`. Option F is incorrect because the static method in the `Paths` class is `get()`, not `getPath()`. Options B and D are correct ways to obtain a `Path` instance. Option G is also correct, as there is an overloaded static method in `Path` that takes a `URI` instead of a `String`.

Türkçe: 10. B, D, G. `Path` ve `FileSystem` factory method ile instance oluşturulması gereken abstract type'lar olduğundan A ve E yanlıştır. `Path` interface'indeki static method `get()` değil `of()` olduğu için C; `Paths` class'ındaki static method `getPath()` değil `get()` olduğu için F yanlıştır. B ve D geçerli `Path` oluşturma yollarıdır. `Path`te `String` yerine `URI` alan overload bulunduğundan G de doğrudur.

Official Answer 11

English: 11. A, E. The code will compile if the correct classes are used, so option G is incorrect.

Türkçe: 11. A, E. Doğru class'lar kullanılırsa kod derlenir; G yanlıştır.

English: Remember, a try-with-resources statement can use resources declared before the start of the statement. The reference type of wrapper is `InputStream`, so we need a class that inherits `InputStream`. We can eliminate `BufferedWriter`, `ObjectOutputStream`, and `BufferedReader` since their names do not end in `InputStream`. Next, we see the class must take another stream as input, so we need to choose the remaining streams that are high-level streams. `BufferedInputStream` is a high-level stream, so option A is correct. Even though the instance is already a `BufferedInputStream`, there's no rule that it can't be

Türkçe: try-with-resources statement başlamadan önce declare edilmiş resource'ları kullanabilir.

wrapper'ın reference type'ı `InputStream` olduğundan `InputStream`den türeyen bir class gerekir.

Adları `InputStream` ile bitmeyen `BufferedWriter`, `ObjectOutputStream` ve `BufferedReader` elenir. Constructor'ı başka bir stream aldığı için kalan high-level stream'ler seçilmelidir.

`BufferedInputStream` high-level olduğundan A doğrudur. Instance zaten `BufferedInputStream` olsa bile tekrar bir high-level stream ile wrap edilmesini engelleyen kural yoktur.

English: wrapped multiple times by a high-level stream. Option D is incorrect, as `FileInputStream` operates on a file, not another stream. Finally, option E is correct— an `ObjectInputStream` is a high-level stream that operates on other streams.

Türkçe: `FileInputStream` başka stream üzerinde değil file üzerinde çalıştığından D yanlıştır.

`ObjectInputStream` diğer stream'ler üzerinde çalışan high-level bir stream olduğundan E doğrudur.

Official Answer 12

English: 12. C, E. The method to create a directory in the `Files` class is `createDirectory()`, not `mkdir()`. For this reason, line 6 does not compile, and option C is correct. In addition, the `setTimes()` method is available only on `BasicFileAttributeView`, not the read-only `BasicFileAttributes`, so line 8 will also not compile, making option E correct.

Türkçe: 12. C, E. `Files` class'ında directory oluşturan method `mkdir()` değil `createDirectory()` dir; bu nedenle 6. satır derlenmez ve C doğrudur. Ayrıca `setTimes()` read-only `BasicFileAttributes`ta değil yalnız `BasicFileAttributeView`da bulunur. 8. satır da derlenmez; E doğrudur.

Official Answer 13

English: 13. A, G. For a class to be serialized, it must implement the `Serializable` interface and contain instance members that are serializable or marked `transient`. For these reasons, options A and G are correct and option F is incorrect. Option B is incorrect because even records are required to implement `Serializable` to be serialized. Option C is incorrect because it describes deserialization. The `Serializable` interface is a marker interface that does not contain any abstract methods, making option D incorrect. While it is a good practice for a serializable class to include a static `serialVersionUID` variable, it is not required. Therefore, option E is incorrect as well.

Türkçe: 13. A, G. Bir class'ın serialize edilebilmesi için `Serializable` interface'ini implement etmesi ve serializable ya da `transient` instance member'lar içermesi gerekir; A ve G doğru, F yanlıştır. Record'lar bile serialize edilmek için `Serializable` implement etmelidir; B yanlıştır. C deserialization'ı anlattığından yanlıştır. `Serializable`, abstract method içermeyen marker interface'tir; D yanlıştır. Static `serialVersionUID` eklemek iyi practice olsa da zorunlu değildir; E de yanlıştır.

Official Answer 14

English: 14. B, D, E. Path is immutable, so line 23 is ignored. If it were assigned to p1, option A would be correct. Since it is not assigned, the original value is still present, which is option B.

Türkçe: 14. B, D, E. `Path` immutable olduğu için 23. satırın `result`'ı kullanılmaz. `Result` `p1`e atansaydı A doğru olurdu. Atama yapılmadığından original value korunur; dolayısıyla B doğrudur.

English: Moving on to the second section, the `subpath()` method on line 27 is applied to the absolute path, which returns the relative path `animals/bear`. Next, the `getName()` method is applied to the relative path, and since this is indexed from 0, it returns the relative path `bear`. Therefore, option D is correct. Finally, remember calling `resolve()` with an absolute path as a parameter returns the absolute path, so option E is correct.

Türkçe: İkinci bölümde 27. satırdaki `subpath()` absolute path'e uygulanır ve relative path `animals/bear` döner. Ardından zero-based `getName()` bu relative path'e uygulanınca `bear` döner; bu nedenle D doğrudur. Son olarak `resolve()` a absolute path verildiğinde bu absolute path aynen döndüğü için E de doğrudur.

Official Answer 15

English: 15. B, E, F. Option A does not compile, as there is no `File` constructor that takes three parameters. Option B is correct and is the proper way to create a `File` instance with a single `String` parameter. Option C is incorrect, as there is no constructor that takes a `String` followed by a `File`. There is a constructor that takes a `File` followed by a `String`, making option E correct. Option D is incorrect because the first parameter is missing a slash (/) to indicate it is an absolute path. Since it's a relative path, it is correct only when the user's current directory is the root directory. Finally, option F is correct as it creates a `File` from a `Path`.

Türkçe: 15. B, E, F. Üç parameter alan bir `File` constructor'ı bulunmadığından A derlenmez. Tek `String` parameter'la `File` instance'ı oluşturan B doğrudur. `String` ardından `File` alan bir constructor bulunmadığı için C yanlıştır; `File` ardından `String` alan constructor bulunduğundan E doğrudur. D'nin ilk parameter'ında absolute path'i belirten / yoktur; bu relative path yalnız current directory root ise doğru file'ı gösterir. Son olarak kaynak, `Path`ten `File` oluşturan F'yi de doğru kabul eder.

Kaynak tutarlılığı notu: Appendix, F seçeneğini doğru kabul eder; ancak soru gövdesindeki literal `/weather/winer/snow.dat` path'inde `winter` yerine `winer` yazılıdır. `Path.toFile()` file'ın varlığını aramadan bir `File` object'i oluştursa da bu object soruda verilen mevcut file'ı temsil etmez. Basılı soru için Java 17'e göre cevap **B, E**; olası dizgi hatası `winter` olarak düzeltilirse kaynak anahtarı **B, E, F** olur.

Official Answer 16

English: 16. A, D. The method compiles, so option E is incorrect. The method creates a `new-zoo.txt` file and copies the first line from `zoo-data.txt` into it, making option A correct. The `try-with-resources` statement closes all of the declared resources, including the `FileWriter` o.

Türkçe: 16. A, D. Method derlenir; dolayısıyla E yanlıştır. `new-zoo.txt` oluşturulur ve `zoo-data.txt` file'ının ilk line'ı buraya kopyalanır; bu nedenle A doğrudur. `try-with-resources` statement, `FileWriter` o dahil declare edilen bütün resource'ları kapatır.

English: For this reason, the `Writer` is closed when the last `o.write()` is called, resulting in an `IOException` at runtime and making option D correct. Option F is incorrect because this implementation uses the character stream classes, which inherit from `Reader` or `Writer`.

Türkçe: Bu nedenle son `o.write()` çağrıldığında `Writer` zaten kapalıdır; çalışma zamanında `IOException` oluşur ve D doğru olur. Uygulama `Reader` veya `Writer`dan türeyen character stream class'larını kullandığından F yanlıştır.

Official Answer 17

English: 17. B, C, E. Options B and C are properties of NIO.2 and are good reasons to use it over the `java.io.File` class. Option A is incorrect as both APIs can delete only empty directories, not a directory tree. Using a view to read multiple attributes leads to fewer round trips between the process and the file system and better performance, making option E correct.

Türkçe: 17. B, C, E. B ve C, NIO.2'nin `java.io.File` API'sine tercih edilmesi için geçerli özellikleridir. İki API de tek çağrıyla yalnız empty directory silebildiği, directory tree silemediği için A yanlıştır. Birden fazla attribute'u view üzerinden birlikte okumak process ile file system arasındaki gidiş-geliş sayısını azaltır ve performance'ı yükseltir; bu nedenle E doğrudur.

English: Views can be used to access file system-specific attributes that are not available in `Files` methods; therefore, option D is correct. `Files` is part of NIO.2, whereas `File` is part of `java.io`, which means option F is incorrect.

Türkçe: View'lar, `Files` method'larında bulunmayan file-system-specific attribute'lara erişmek için kullanılabilir; kaynak açıklama bu yüzden D'yi doğru sayar. `Files` NIO.2'nin, `File` ise `java.io`unun parçasıdır; dolayısıyla F yanlıştır.

Kaynak tutarlılığı notu: Resmî anahtar **B, C, E**'dir ve seçeneklerin Java 17 değerlendirmesi de bunu destekler. Açıklamanın sonraki paragrafındaki “therefore, option D is correct” cümlesi kendi anahtarıyla çelişen bir kaynak hatasıdır. `Files.readAttributes()` hem typed attribute class'ları hem de string attribute-view syntax'ı üzerinden file-system-dependent attributes'a erişebilir; bu yüzden D yanlıştır.

Official Answer 18

English: 18. C. Since a `Reader` may or may not support `mark()`, we can rule out options E, F, G, and

Türkçe: 18. C. Bir `Reader`, `mark()`ı destekleyebilir veya desteklemeyebilir; bu nedenle E, F, G ve

English: H. Assuming `mark()` is supported, P is added to the `StringBuilder` first. Next, the position in the stream is marked before E. The E is added to the `StringBuilder`, with AC being skipped, and then the O is added to the `StringBuilder`, with CK being skipped. The stream is then `reset()` to the position before the E. The call to `skip(0)` doesn't do anything since there are no characters to skip, so E is added onto the `StringBuilder` in the next `read()`

Türkçe: H elenir. `mark()` in desteklendiğini varsayarsak önce P, `StringBuilder`a eklenir. Ardından stream position'ı E den önce mark edilir. E eklenir, AC skip edilir; sonra O eklenir ve CK skip edilir. Stream, `reset()` ile E den önceki position'a döner. `skip(0)` hiçbir character atlamadığından sonraki `read()` ile E, `StringBuilder`a eklenir.

English: call. The value PEOE is printed, and option C is correct.

Türkçe: çağırısı yapılır. PEOE yazdırılır ve C seçeneği doğru olur.

Java 17 editör notu: Appendix cevabı, `skip(2)` çağrılarının her seferinde tam iki character atladığını varsayarak **C** sonucuna ulaşır. `Reader.skip()` contract'ı ise istenenden daha az, hatta 0, atlanmasına izin verir. `mark()` destekleniyor ve `reset()` başarılıysa basılı seçeneklerden **A** da mümkün olabilir; örneğin ilk `skip(2)` çağırısı 0 döndürürse PEAЕ yazdırılır. Dolayısıyla contract düzeyinde savunulabilir “may print” cevapları **A, C**'dir.

Official Answer 19

English: 19. C. The code compiles and runs without issue, so option G is incorrect. If you simplify the redundant path symbols, p1 and p2 represent the same path, /lizard/walking.txt.

Türkçe: 19. C. Kod sorunsuz derlenip çalıştığı için G yanlıştır. Redundant path symbol'ları sadeleştirildiğinde p1 ile p2, aynı /lizard/walking.txt path'ini temsil eder.

English: Therefore, `isSameFile()` returns true. The second output is false, because `equals()`

Türkçe: Bu nedenle `isSameFile()` true döndürür. İkinci çıktı false'tur; çünkü `equals()`

English: checks only if the path values are the same, without reducing the path symbols. Finally, `mismatch()` sees that the contents are the same and returns -1. For these reasons, option C is correct.

Türkçe: yalnız path value'larının aynı olup olmadığını, path symbol'larını sadeleştirmeden denetler. Son olarak `mismatch()`, content'in aynı olduğunu görüp -1 döndürür. Bu nedenlerle C doğrudur.

Official Answer 20

English: 20. D. The target path of the file after the `move()` operation is /animals, not /animals/monkey.txt, so options A and B are both incorrect. Both will throw an exception at runtime since /animals already exists and is a directory. Next, the `NOFOLLOW_LINKS` option means that if the source is a symbolic link, the link itself and not the target will be copied at runtime, so option C is also incorrect. The option `ATOMIC_MOVE` means that any process monitoring the file system will not see an incomplete file during the move, so option D is correct.

Türkçe: 20. D. `move()` sonrasında target path /animals/monkey.txt değil /animals'dır; dolayısıyla A ve B yanlıştır. /animals zaten var olan bir directory olduğundan her iki durumda da çalışma zamanında exception fırlatılır. `NOFOLLOW_LINKS`, source symbolic link ise target yerine link'in kendisinin işleme alınacağı anlamına gelir; bu yüzden C de yanlıştır. `ATOMIC_MOVE`, file system'i izleyen başka bir process'in move sırasında incomplete file görmemesini sağlar; D doğrudur.

Official Answer 21

English: 21. C. The code compiles and runs without issue, so options D, E, and F are incorrect. The most important thing to notice is that the depth parameter specified as the second argument to `find()` is 0, meaning the only record that will be searched is the top-level directory. Since we know that the top directory is a directory and not a symbolic link, no other paths will be visited, and nothing will be printed. For these reasons, option C is the correct answer.

Türkçe: 21. C. Kod sorunsuz derlenip çalıştığı için D, E ve F yanlıştır. `find()` in ikinci argument'ı olan depth değeri 0'dır; dolayısıyla yalnız top-level directory aranır. Bu entry'nin directory olduğunu ve symbolic link olmadığını bildiğimizden başka hiçbir path ziyaret edilmez, hiçbir şey yazdırılmaz. Bu nedenle C doğrudur.

Official Answer 22

English: 22. G. The code compiles, so option F is incorrect. To be serializable, a class must implement the `Serializable` interface, which `Zebra` does. It must also contain instance members that either are marked `transient` or are serializable. The instance member `stripes` is of type `Object`, which is not serializable. If `Object` implemented `Serializable`, all objects would be serializable by default, defeating the purpose of having the `Serializable` interface. Therefore, the `Zebra` class is not serializable, with the program throwing an exception at runtime if serialized and making option G correct. If `stripes` were removed from the class, options A and D would be the correct answers, as `name` and `age` are both marked `transient`.

Türkçe: 22. G. Kod derlendiği için F yanlıştır. `Serializable` olmak için class, `Zebra` nın yaptığı gibi `Serializable` interface'ini implement etmelidir. Instance member'ları da ya `transient` olmalı ya da serializable value göstermelidir. `stripes`, serializable olmayan `Object` type'ındadır. `Object`, `Serializable` implement etseydi her object varsayılan olarak serializable olur ve marker interface'in amacı ortadan kalkardı. Bu nedenle `Zebra` serialize edilmeye çalışılırsa çalışma zamanında exception fırlatılır; G doğrudur. `stripes` kaldırılırsa `name` ile `age` `transient` olduğundan A ve D doğru olurdu.

Official Answer 23

English: 23. A, D. The code compiles without issue, so options E and F are incorrect. The `toRealPath()` method will simplify the path to `/animals` and throw an exception if it does not exist, making option D correct. If the path does exist, calling `getParent()` on it returns the root directory. Walking the root directory with the filter expression will print all `.java` files in the root directory (along with all `.java` files in the directory tree), making option A correct. Option B is incorrect because it will skip files and directories that do not end in the `.java` extension. Option C is also incorrect as `Files.walk()` does not follow symbolic links by default. Only if the `FOLLOW_LINKS` option is provided and a cycle is encountered will the exception be thrown.

Türkçe: 23. A, D. Kod sorunsuz derlendiğinden E ve F yanlıştır. `toRealPath()`, path'i `/animals` a sadeleştirir; path yoksa exception fırlatır, bu yüzden D doğrudur. Path varsa `getParent()` root directory'yi döndürür. Root üzerinde `walk()` ve filter çalıştırıldığında root ile directory tree'deki bütün `.java` file'ları yazdırılır; A doğrudur. `.java` ile bitmeyen file ve directory'ler elendiği için B yanlıştır. `Files.walk()` varsayılan olarak symbolic link izlemediğinden C de yanlıştır. Yalnız `FOLLOW_LINKS` verilmiş ve cycle oluşmuşsa exception fırlatılır.

Official Answer 24

English: 24. B. The method compiles without issue, so option E is incorrect. Option F is also incorrect.

Türkçe: 24. B. Method sorunsuz derlenir; dolayısıyla E ve F yanlıştır.

English: Even though `/flip` exists, `createDirectories()` does not throw an exception if the path already exists. If `createDirectory()` were used instead, option F would be correct.

Türkçe: `/flip` var olsa bile `createDirectories()` existing path için exception fırlatmaz. Bunun yerine `createDirectory()` kullanılsaydı F doğru olurdu.

English: Next, the `copy()` command takes a target that is the path to the new file location, not the directory to be copied into. Therefore, the target path should be `/flip/sounds.txt`, not `/flip`. For this reason, options A and C are incorrect. Since the question says the file already exists, the `REPLACE_EXISTING` option must be specified or an exception will be thrown at runtime, making option B the correct answer.

Türkçe: `copy()` target olarak içine kopyalanacak directory'yi değil, yeni file location'ının tam path'ini alır. Bu nedenle target `/flip` değil `/flip/sounds.txt` olmalıdır; A ve C yanlıştır. File zaten var olduğundan `REPLACE_EXISTING` verilmezse çalışma zamanında exception fırlatılır. Bu yüzden B doğrudur.

Official Answer 25

English: 25. B, D. Since you need to read characters, the Reader classes are appropriate. Therefore, you can eliminate options A, C, and F. Additionally, options E and G are incorrect, as they reference classes that do not exist. Options B and D are correct since they read from a file and buffer for performance.

Türkçe: 25. B, D. Character okunacağı için `Reader` class'ları uygundur; A, C ve F elenir. E ve G var olmayan class'ları gösterdiğinden yanlıştır. B ile D birlikte bir file'dan character okur ve performance için buffer sağlar; ikisi de doğrudur.

Coverage ledger

- Chapter body marker'ları: 785–862
- Appendix answer marker'ları: 955–959
- Resmî cevap hedefi: 1–25
- Kod blokları özgün dilinde tutulmuştur.
- Çeviri ayrıntıları ünite vocabulary ve grammar kaynaklarıyla desteklenir.